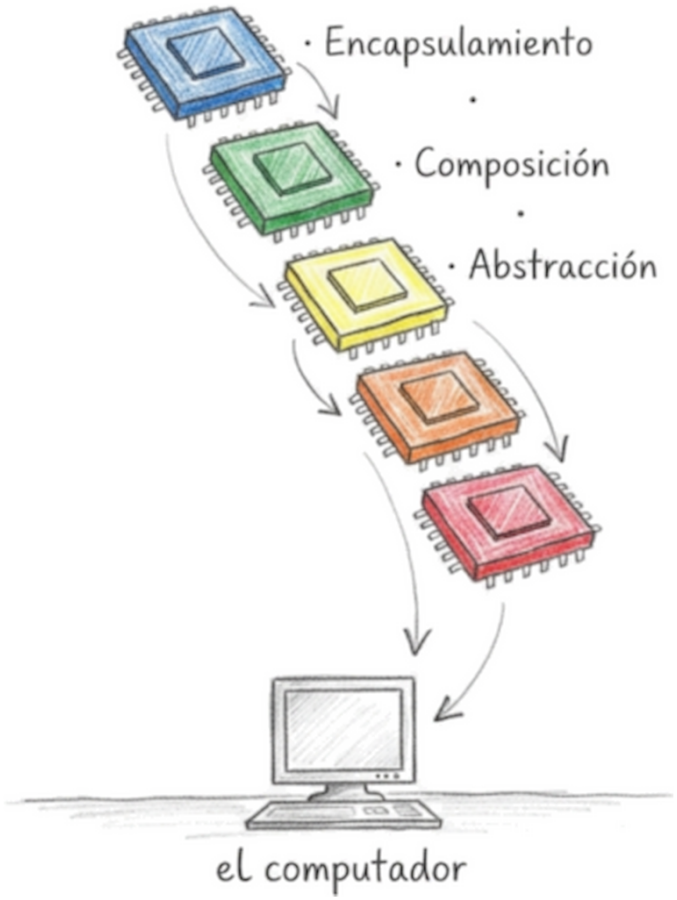
INVADIENDO ESPACIOS

Orientación a objetos con Space Invaders, para programadores y no programadores

II



Luis Alfonso Salazar Vaca

27 de agosto de 2026

Aviso legal y de propiedad intelectual

Invadiendo Espacios es una obra educativa e independiente cuyo propósito es enseñar conceptos de programación, orientación a objetos, diseño de software y uso de herramientas de inteligencia artificial mediante ejemplos prácticos.

Space Invaders se utiliza en esta obra exclusivamente como referencia cultural, histórica y caso de estudio educativo, por tratarse de un videojuego ampliamente conocido cuyas mecánicas permiten ilustrar de manera accesible los conceptos desarrollados a lo largo del libro.

Space Invaders y las marcas, nombres, personajes, diseños, elementos gráficos y demás activos asociados al videojuego original pertenecen a sus respectivos titulares. Esta obra no está afiliada, patrocinada, autorizada ni respaldada por TAITO Corporation ni por los titulares de los derechos de Space Invaders.

El juego desarrollado a lo largo de este libro constituye una implementación original creada con fines educativos. Sus reglas, estructuras de software, ejemplos, código fuente y representaciones utilizadas para explicar los conceptos del libro han sido desarrollados específicamente para esta obra y no pretenden reproducir el código fuente, sprites, recursos gráficos, sonidos, música ni otros activos protegidos del videojuego original.

Las referencias al videojuego original se realizan únicamente cuando resultan necesarias para explicar, contextualizar o analizar los conceptos tratados. La mención de nombres comerciales o marcas registradas no implica propiedad sobre dichas marcas ni relación comercial con sus titulares.

El código fuente incluido en esta obra tiene fines educativos y demostrativos. Su disponibilidad, licencia y condiciones de reutilización se especifican en el repositorio oficial asociado al libro.

© 2026 Luis Alfonso Salazar Vaca
Invadiendo Espacios
Todos los derechos sobre el texto y los elementos originales de esta obra pertenecen a su autor, salvo aquellos expresamente identificados como propiedad de terceros.



Código desarrollado en este libro

Invadiendo Espacios

Orientación a objetos con Space Invaders, para programadores y no programadores

Luis Alfonso Salazar Vaca

27 de agosto de 2026

Contenidos

Prólogo — El siguiente espacio por conquistar **1**

I Definiendo espacios, un lugar para cada cosa y cada cosa en su lugar **2**

1 Del escritorio a una pantalla **3**
1.1 Dónde nos quedamos: un estado, una decisión, un turno 4
1.2 Lo que Space Invaders trae y el tres en raya no tenía 4
1.3 El juego en una frase: qué hay que conseguir 5
1.4 Por qué este juego y no otro 6
1.5 Por qué Python, dicho como decisión 7
1.6 Por qué implementar el modo auto-play 7

2 Cuando todo lo sabe una sola función **9**
2.1 La primera versión que todos escribimos 9
2.2 El bucle que lo hace todo 10
2.3 Las variables que hay que tener en la cabeza a la vez 11
2.4 Tres cambios pedidos, y lo que cuesta cada uno 11
2.5 La desproporción, otra vez, y en otro sitio 12
2.6 Y aquí no se puede recorrer el juego entero 13

II Los objetos **15**

3 Clases y objetos **16**
3.1 La pregunta de este libro: ¿quién debería saber esto? 16
3.2 El molde y el objeto 16
3.3 Cuántos invasores hay y cuántas clases hacen falta 17
3.4 Qué no es un objeto: la trampa de la clase por cada sustantivo 18
3.5 La prueba de la frase: decir qué hace cada objeto sin usar «y» 19

4 Atributos: lo que un objeto sabe **22**
4.1 Dato, información y conocimiento 22
4.2 Estado propio frente a estado prestado 23
4.3 Los atributos del invasor, del cañón y de la bala 23
4.4 Lo que parece un atributo y es una cuenta 25

4.5 Datos desnudos: por qué un entero no dice qué significa 25

5 Métodos: lo que un objeto sabe hacer 28
5.1 Pedir, no manipular 28
5.2 Preguntar y ordenar son cosas distintas 29
5.3 Los métodos del invasor, uno a uno 29
5.4 Métodos que devuelven un objeto nuevo en vez de cambiar el actual 31

6 Constructores: nacer ya válido 33
6.1 Objetos que no deberían poder existir 33
6.2 Invariantes de clase: lo que se garantiza en el nacimiento 34
6.3 El constructor como contrato de entrada 35
6.4 Lo que no va en un constructor 36

III Lo que no se muestra no se vende 38

7 Encapsulamiento 39
7.1 La frontera: qué se ve desde fuera 39
7.2 Quién puede romper una regla del juego, y por qué no debería poder 40
7.3 Cambiar por dentro sin que nadie se entere 41
7.4 Lo que el encapsulamiento *no* es: poner guiones bajos 41

8 Abstracción 44
8.1 Llevar un problema real al programa 44
8.2 Quedarse con lo que decide y descartar lo demás 45
8.3 Entidad: lo que el bucle necesita saber, y nada más 46
8.4 La prueba que separa una abstracción de una metáfora 47
8.5 Abstraer de menos y abstraer de más 47

IV Parecerse es diferente de SER 49

9 Herencia 50
9.1 Tres tipos de invasor que se mueven igual 50
9.2 Qué se hereda de verdad: comportamiento, no parecido 51
9.3 El OVNI no es un invasor, aunque lo parezca 51
9.4 La tentación: una subclase por cada habilidad 52
9.5 Por qué las jerarquías profundas salen caras 54

10 Polimorfismo 56
10.1 Una sola orden para entidades distintas 56
10.2 Varias naves agresoras, un solo bucle 56
10.3 El bucle que no sabe con quién habla 58

10.4 Condicionales que desaparecen . . . 58
10.5 Lo que un sustituto tiene que poder hacer . . . 60

11 Composición 62
11.1 Tener algo no define un nuevo ser . . . 62
11.2 El cañón, su arma y sus vidas . . . 62
11.3 La oleada como objeto, no como lista . . . 63
11.4 Una habilidad se tiene, no se es . . . 64

12 Interfaces y clases abstractas 67
12.1 Un contrato es un encargo bien dado . . . 67
12.2 Colisionable: dame tu caja y no me cuentes más . . . 67
12.3 Cuándo un contrato y cuándo una base con trabajo hecho . . . 68
12.4 Contratos que piden de más y contratos que mienten . . . 69

V Los cinco principios 72

13 Responsabilidad única 73
13.1 El enunciado: solo puede haber una única razón para cambiar . . . 73
13.2 La razón para cambiar: quién pide la modificación . . . 73
13.3 La clase que movía, dibujaba, puntuaba y sonaba . . . 74
13.4 Partir por motivo de cambio, no por tamaño . . . 75
13.5 Partir de más . . . 76

14 Abierto/cerrado 78
14.1 El enunciado . . . 78
14.2 El cuarto tipo de invasor . . . 78
14.3 Extensión frente a modificación, con una prueba objetiva . . . 80
14.4 Lo que hay que dejar abierto y lo que conviene dejar cerrado . . . 81
14.5 Adivinar el futuro sale caro . . . 82

15 Sustitución de Liskov 84
15.1 El enunciado, en la versión que se entiende . . . 84
15.2 El invasor indestructible que rompe el bucle . . . 84
15.3 Precondiciones, poscondiciones e invariantes: qué se le puede exigir . . . 85
15.4 El caso clásico —y por qué aquí no lo uso— . . . 87
15.5 Cuando la jerarquía era el error, no el sustituto . . . 87

16 Segregación de interfaces 90
16.1 El enunciado . . . 90
16.2 La interfaz que obliga al marcador a saber dibujarse . . . 90
16.3 Interfaces pequeñas: cuántas y de qué tamaño . . . 91
16.4 Segregar de más: la interfaz de un solo método por objeto . . . 92

17 Inversión de dependencias 94
17.1 El enunciado . . . 94
17.2 De quién depende el juego, dibujado . . . 94
17.3 Depender de lo que se promete, no de quién lo cumple . . . 96
17.4 DIP no es inyección de dependencias, y confundirlas es lo normal . . . 97
17.5 Invertir de más: la abstracción con una sola implementación . . . 97

VI Que funcione no es suficiente 100

18 El juego que nació apretado 101
18.1 Las condiciones de 1978 . . . 101
18.2 Optimizar no era una virtud: era la condición para que el juego existiera . . . 101
18.3 La pregunta incómoda: hoy sobran recursos, ¿para qué preocuparse? . . . 102
18.4 La respuesta: sí, y por cuatro motivos que convergen . . . 102
18.5 No se trata solo de resolver la tarea, sino de hacerlo bien . . . 103
18.6 La tensión que no vamos a esquivar . . . 104

19 Dimensionar el dato 106
19.1 Un entero de propósito general para guardar un número del 0 al 5 . . . 106
19.2 En Python el ancho lo decide el contenedor, no la variable . . . 106
19.3 Los tipos que Python sí te da, y cuál corresponde a cada dato . . . 106
19.4 Lo que cuesta un objeto antes de guardar nada . . . 107
19.5 Medido: una oleada de invasores en cuatro representaciones . . . 108
19.6 Cuándo dimensionar es prematuro, y cómo se sabe . . . 109

20 Factory: crear sin saber qué se crea 111
20.1 El dolor: montar cada oleada a mano . . . 111
20.2 Pedir la oleada 7 . . . 112
20.3 Qué se gana y qué se paga . . . 113

21 Strategy: cambiar el comportamiento sin tocar el código 115
21.1 El dolor: tres formas de disparar y un condicional que crece . . . 115
21.2 La estrategia de disparo, sustituible en caliente . . . 116
21.3 El principio abierto/cerrado, hecho código . . . 117
21.4 Lo que cuesta . . . 118

22 Observer: enterarse sin preguntar 119
22.1 El dolor: el invasor tiene que conocer al marcador . . . 119
22.2 Quien lo publica no sabe quién lo escucha . . . 120
22.3 El precio: un flujo que ya no se lee de arriba abajo . . . 121

23 Adapter: hablar con lo que no se diseñó para ti 123
23.1 El dolor: cada mando habla su idioma . . . 123

23.2 Una sola puerta de entrada . . . 123
23.3 Dónde acaba el adaptador y empieza el remiendo . . . 125

24 Repository: guardar y recuperar 126
24.1 El dolor: los marcadores atan el juego a un archivo . . . 126
24.2 Pedir sin saber dónde está guardado . . . 127
24.3 Lo que este juego apenas necesita, y por qué se cuenta igual . . . 128

25 Singleton: uno solo, y lo que cuesta 130
25.1 El dolor: dos relojes en la misma partida . . . 130
25.2 Cómo se hace . . . 130
25.3 Por qué es el patrón más criticado, y con razón . . . 131
25.4 Qué usar en su lugar casi siempre . . . 131
25.5 Cuándo sí . . . 132

26 Inyección de dependencias: recibir en vez de fabricar 134
26.1 El dolor: cada habilidad y cada azar, fabricados dentro . . . 134
26.2 Otorgar habilidades desde fuera: al agresor, al defensor y al búnker . . . 135
26.3 La semilla que entra desde fuera . . . 135
26.4 Dobles de prueba: probar el comportamiento, no la suerte . . . 136
26.5 La técnica y el principio: qué la separa del capítulo 17 . . . 137
26.6 Lo que esto le regala al capítulo 31 . . . 137

27 MVC: lo que el juego es y cómo se ve 138
27.1 El dolor: el juego sabe dibujarse . . . 138
27.2 Modelo, vista y control, elemento por elemento . . . 138
27.3 La misma partida en consola y en pantalla . . . 139
27.4 Dónde MVC se queda corto . . . 140

VII El jugador humano entra en el juego 142

28 El jugador humano 143
28.1 Hasta aquí jugaba una máquina, y eso escondía una pregunta . . . 143
28.2 Un jugador es un contrato, no una persona . . . 143
28.3 Leer teclas y traducir teclas son dos oficios . . . 144
28.4 El ritmo: un turno que dura lo que dura un parpadeo . . . 145
28.5 Los dos defectos que solo aparecen con el teclado delante . . . 146
28.6 Lo que costó, contado en líneas . . . 147

29 Recompensas que caen 149
29.1 El jugador también se diseña . . . 149
29.2 ¿Quién sabe que esa nave lleva algo dentro? . . . 149
29.3 Una mejora es un arma que envuelve a otra . . . 150

29.4 Cada mejora cambia una sola cosa, y por eso se apilan . . . 151
29.5 La bala que atraviesa: quién decide si un proyectil sigue vivo . . . 152
29.6 El orden no importa, y eso es una cláusula que se prueba . . . 153
29.7 Dos defectos que aparecen al ejecutarlo . . . 154
29.8 Lo que costó la vuelta siguiente . . . 154

30 Oleadas que no se repiten 156
30.1 El dolor: una clase por nivel . . . 156
30.2 La progresión, en una tabla y en un solo sitio . . . 156
30.3 Artillero: disparar sin esperar turno . . . 157
30.4 Zigzag: salirse de la retícula . . . 158
30.5 Combinada: escudo y regeneración sin una clase nueva . . . 158
30.6 Lo que costó subir la dificultad, medido . . . 159
30.7 Y lo que encareció, que también hay que decirlo . . . 159

VIII Lo que realmente cambió 161

31 Comprobarlo cuando no se puede recorrer todo 162
31.1 El espacio de estados de este juego, calculado . . . 162
31.2 Lo que se pierde al perder la exhaustividad . . . 163
31.3 Determinismo: fijar el azar para poder repetir . . . 163
31.4 Propiedades en lugar de casos . . . 164
31.5 Invariantes que no pueden romperse . . . 164
31.6 Lo que este capítulo *no* demuestra . . . 165

32 El diseño se mide por el cambio siguiente 167
32.1 Las cifras, medidas y no estimadas . . . 167
32.2 Los tres cambios del capítulo 2, ahora . . . 168
32.3 Lo que el reparto costó en recursos . . . 168
32.4 Qué se pudo borrar . . . 169
32.5 Defender un diseño explicándolo . . . 170
32.6 Lo que el reparto no hizo . . . 171

33 ¿Hacía falta la orientación a objetos? 172
33.1 La objeción del lector curioso . . . 172
33.2 Tres versiones del mismo juego . . . 172
33.3 La respuesta a la pregunta incómoda de este capítulo: para el juego tal como se pidió . . . 174
33.4 Y sin embargo, por qué insisto en recomendar la orientación a objetos . . . 174
33.5 Un reparto parcial no rinde . . . 175

34 Cuándo esto no funciona 177
34.1 Clases que solo son funciones con sombrero . . . 177

34.2 Repartir de más: cuando cada decisión está en otro archivo 178
34.3 Optimizar de más: el byte que costó una tarde y ahorró un kilobyte 178
34.4 SOLID aplicado a rajatabla, y lo que produce 179
34.5 Lo que este juego no sirve para enseñar 179
34.6 El riesgo de ver patrones donde no los hay 180
34.7 Cómo saber que te has equivocado 181

IX Capacidades aumentadas con Inteligencia Artificial 183

35 Un juego entero en un prompt 184
35.1 Lo que la herramienta hace extraordinariamente bien 184
35.2 El error de diseño que un programa que funciona esconde 184
35.3 Por qué aquí el riesgo es mayor que en el primer libro 185
35.4 Lo que sigue siendo humano 186

36 Prompts que reparten 187
36.1 No pidas el juego, pide el reparto 187
36.2 Que cuestione el escenario antes de escribir nada 187
36.3 Que haga lo mínimo pedido 188
36.4 El agente inteligente eres tú: la planeación y el objetivo son tuyos 188
36.5 Anatomía de un prompt de diseño 189
36.6 Cómo pedirle que critique tu diseño 189
36.7 Verificar lo que te responde cuando todo compila 190

Apéndices 191

A Glosario 191
Los diez conceptos 191
Los tres escalones del dato 192
Los cinco principios 192
Los ocho patrones 192
Términos del método 193

Referencias 194

Prólogo — El siguiente espacio por conquistar

En *Pensando fuera de la caja* aprendimos a mirar los problemas desde otro ángulo. El tres en raya nos enseñó que, antes de escribir una instrucción, conviene comprender lo que queremos resolver. Nos animamos a programar, a experimentar y a descubrir que una buena idea puede abrir caminos que no habíamos imaginado.

Ahora llega el siguiente paso.

No porque lo que hicimos antes estuviera mal, sino porque aprender también significa evolucionar. Ya sabemos que podemos construir programas. En este libro vamos a preguntarnos cómo construirlos pensando en lo que realmente necesitan: qué responsabilidades deben existir, cómo organizar cada pieza, cuándo una solución sencilla es suficiente y cuándo merece la pena invertir en un diseño capaz de crecer.

Queremos programas que funcionen, desde luego. Pero también que podamos entender, cambiar y mantener sin perder de vista el tiempo, la memoria y los recursos que utilizan. La eficiencia no es solo hacer que una máquina trabaje más rápido: también es evitar complejidades que no aportan valor.

Te invito a convertirte en el invasor de un espacio nuevo: el que se encuentra al otro lado de la pantalla. No para destruirlo, sino para conquistarlo con tus propias ideas, procesos y necesidades. Serás el comandante de tu flota. No necesitas hacerlo todo personalmente: necesitas saber qué quieres conseguir, qué debe hacer cada pieza y cómo comprobar que avanzas en la dirección correcta.

I

Definiendo espacios, un lugar para cada cosa y cada cosa en su lugar

Capítulo 1

Del escritorio a una pantalla

Levanta la vista un momento y mira dónde estás.

Ahí está tu escritorio, y encima lo que tu oficio ha ido dejando: papeles ordenados como lo decidiste en su momento, libros que abres siempre por las mismas páginas, una libreta con lo que no cabe en ningún otro sitio. Y alrededor, lo que no se ve y es igual de importante: el procedimiento que hay que seguir para pedir algo, la lista que repasas cada lunes, la cuenta que rehaces cada mes porque no está escrita en ninguna parte de la que no dependa tu memoria. Tuyo o de quienes trabajan contigo, da igual: es un entorno, tiene sus reglas, y funciona porque hay alguien que lo hace funcionar.

Buena parte de eso podría hacerlo una herramienta. Y tú lo sabes, porque cada vez que repites el mismo trabajo por tercera vez se te ocurre.

Ahora mira el otro objeto de la mesa. El monitor, el teclado, el ratón: eso no es una máquina, es un portal. Al otro lado hay un mundo entero que funciona en paralelo al tuyo y que lo complementa de mil maneras. Está gobernado, tiene territorio y no está vacío: lo que hay dentro son las rutinas que otros definieron. Por eso tantas veces acabas haciendo las cosas como el programa quiere y no como tú las harías.

¿Y si invades ese espacio y lo conquistas? Este libro te invita precisamente a lograrlo.

A partir de aquí eres **el comandante de tu propia flota**. Tu tarea es entrar en ese espacio digital y ocuparlo con lo tuyo: tu entorno, tus procesos, tus rutinas, tus costumbres —no una imitación aproximada de ellos, sino ellos—, hasta que lo que hay al otro lado de la pantalla te sirva de verdad. Y no te detienes cuando el programa funciona. Te detienes cuando el resultado te satisface, que es bastante más profundo y es de lo que se ocupan los capítulos que vienen.

Ese espacio que vas a invadir tiene fronteras, y son las reglas del computador: la memoria que hay, el tiempo que cabe entre dos fotogramas, los tipos de dato que admite, los recursos que nunca sobran. Programar es ocuparlo con lo que tú necesitas, hasta que la máquina se someta a tus necesidades en lugar de al revés. Se hace pieza a pieza, nunca de un salto.

Y ahora el juego que da título al libro, y lo posiciona como segundo de esta serie. En el Space Invaders de las máquinas recreativas manejas el cañón: defiendes. El espacio es tuyo y unos bichos bajan a quitártelo, fila a fila, cada vez más deprisa. Aquí no. Aquí la flota que baja es la tuya.

Y hay una cosa más que un comandante hace, y que conviene recordar: **un comandante no dispara: lidera.** Es consciente de qué sabe hacer cada nave, reparte responsabilidades y recursos, y lleva a la flota al objetivo. Su trabajo no es hacerlo todo: es que cada pieza cumpla con lo suyo y el conjunto llegue donde tiene que llegar. Todo lo demás —incluido cómo se lo pides a un asistente de inteligencia artificial— sale de ahí.

1.1 Dónde nos quedamos: un estado, una decisión, un turno

El libro anterior, *Pensando fuera de la caja*, se ocupaba del tres en raya. Terminaba con un algoritmo de cuatro reglas que no puede perder, y con una comprobación que casi ningún libro de programación puede ofrecer: recorrí el juego entero, todas las partidas posibles, y conté cero derrotas.

Aquello fue posible por una propiedad del problema que entonces apenas mencioné, y que ahora hay que mirar de frente. El tres en raya es **estático**:

- Hay **un estado**: quién ocupa qué casilla.
- Hay **una decisión**: dónde jugar.
- Y hay **un turno**: mientras tú piensas, el mundo no se mueve.

Un problema así cabe entero en una función. No porque yo escribiera un algoritmo especialmente compacto, sino porque no había nada más que gobernar: el mérito era del problema tanto como del algoritmo.

Detengámonos a pensar · Lo que aquel libro no podía enseñar

Un caso pequeño enseña bien lo que enseña, y por eso lo elegí. Pero hay cosas que sencillamente no estaban allí.

No había dos cosas ocurriendo a la vez. No había nada que se moviera solo. No había pantalla, así que no hacía falta decidir qué parte del programa sabe dibujar. Y como todo cabía en una función, nunca surgió la pregunta de *quién* debería saber cada cosa: solo había uno que lo sabía todo, y era suficiente.

La orientación a objetos existe para gobernar exactamente lo que allí faltaba. Enseñarla con el tres en raya habría sido inventar una necesidad.

1.2 Lo que Space Invaders trae y el tres en raya no tenía

Cambiar de juego nos permite cambiar de clase de problema.

Cuadro 1.1: Los nuevos desafíos que plantea este escenario.

En el tres en raya	**En Space Invaders**
Un estado	Decenas de entidades vivas a la vez, cada una con su estado particular
Una decisión	Muchas decisiones simultáneas, y cada una es de una entidad específica
Un turno	El reloj no se detiene, y todo sigue ocurriendo independientemente de tus decisiones
Nada que dibujar	Una frontera entre lo que el juego *es* y cómo se ve
El juego entero recorrible	Un espacio de estados demasiado amplio

Las cuatro primeras son evidentes. La quinta es la que más me costó aceptar.

Detengámonos a pensar · La quinta fila es la que nos invita a conquistar este espacio

En el libro anterior, cada vez que afirmé algo pude recorrer el juego completo y demostrarlo. Cero derrotas no era una estimación: era un recuento.

Aquí eso no es factible. Ni con toda la paciencia del mundo: el capítulo 31 calcula el tamaño del espacio de estados de este juego en lugar de afirmarlo, y la conclusión es que no se puede recorrer.

Lo que hay en su lugar —fijar el azar para poder repetir una partida, comprobar propiedades en vez de casos, exigir invariantes que no puedan romperse—. Y eso es lo que hace interesante este desafío.

1.3 El juego en una frase: qué hay que conseguir

Antes de programar nada hace falta saber qué se quiere. Este es el encargo, enunciado como lo enunciaría un cliente:

> Una formación de invasores avanza hacia abajo. El jugador maneja un cañón que se desplaza a lo ancho y dispara. **Gana quien llegue antes**: el jugador, si despeja la formación; los invasores, si tocan el suelo o acaban con las vidas del jugador.

Y estas son las reglas concretas del juego que vamos a construir. No son las del arcade de 1978: son las nuestras, y las cito a continuación.

1. La formación se mueve en bloque, a lo ancho. Cuando alguno de sus miembros vivos toca un lado, **toda** la formación baja una fila y cambia de sentido.

2. Cuantos menos invasores quedan, más deprisa se mueve la formación. El juego se endurece solo.

3. El cañón tiene una bala en el aire cada vez. Hasta que no se resuelve (colisiona o se pierde fuera de la pantalla), no hay otra.
4. Los invasores sueltan bombas, y solo puede haber unas pocas cayendo a la vez.
5. Hay búnkeres que aguantan varios impactos, de los dos bandos.
6. De vez en cuando cruza un OVNI por arriba, y si el jugador lo destruye otorga muchos puntos.
7. Al despejar la formación empieza otra oleada, un poco más abajo.

Inténtalo primero · Antes de seguir

Busca un papel y responde a tres preguntas. Tardas cinco minutos y el capítulo siguiente se lee de otra manera si lo haces.

Uno. Enumera todo lo que hay que recordar en cada instante para poder dibujar la pantalla. Todo: posiciones, marcadores, lo que sea.

Dos. ¿Quién decide que un invasor ha muerto? ¿La bala, el invasor, o alguien que los mire a los dos?

Tres. La regla 1 dice que la formación baja cuando *alguno* toca el lado. ¿Quién sabe eso? Ningún invasor conoce a los demás.

1.4 Por qué este juego y no otro

La elección tiene la misma exigencia que en el libro anterior: el ejemplo tiene que ser lo bastante conocido como para que ninguna dificultad venga del enunciado, y lo bastante completo como para que las dificultades que aparezcan sean retadoras.

Porque lo conoces. No voy a gastar una página explicándote el juego. Lo que te cueste será lo que quiero que te cueste.

Porque cada concepto tiene aquí un lugar concreto. Esto es lo que decidió la elección. No hay un solo capítulo de este libro que ilustre un concepto con un ejemplo inventado.

Porque se puede terminar. Al cerrar el último capítulo existirá un juego que se ejecuta, no una colección de fragmentos. Y lo ejecutaremos, que es distinto que afirmar que funciona.

El recurso que vas a ver en todo el libro

En el juego

Cada concepto de este libro aparece donde el juego lo necesita: un invasor que no deja que le cambien la posición desde fuera, una formación que sabe algo que sus miembros no saben, una habilidad que se otorga en lugar de heredarse.

En la vida real

El mismo concepto funcionando en tu día a día: el cajero que no te deja tocar el saldo, el líder que gestiona información que su equipo desconoce.

No es adorno pedagógico. **Abstraer es llevar un problema real al programa.**

Sin programar · Qué vas a hacer tú si no sabes programar

Prácticamente todo lo que importa de este libro se decide sin escribir una línea de código. Quién sabe qué, quién le pide qué a quién, qué pasa cuando hay que cambiar algo: eso es diseño, y se hace hablando, dibujando y llegando a acuerdos.

Vas a tener dos apoyos en cada capítulo. El recuadro que acabas de ver, que traduce cada concepto a algo que ya sabes. Y los recuadros *prompt*, con los que puedes pedirle a tu asistente que construya el componente que tú has decidido.

La prueba de que este libro te sirve no es que acabes escribiendo código. Es que acabes pudiendo decidir el cómo, el qué y defenderlo.

1.5 Por qué Python, dicho como decisión

Todo el código de este libro está en Python, y lo elegí **por sencillez de lectura**. No por rendimiento, no porque sea el mejor lenguaje para escribir juegos, y desde luego no porque sea el más riguroso de los disponibles.

Lo que gano: un listado de Python se lee sin saber Python. Quien no programa puede seguir un método de doce líneas y entenderlo.

Lo que pierdo, y hay que decirlo: **Python no obliga a casi nada**. No exige declarar tipos, no impide tocar por fuera lo que debería ser privado, y no avisa cuando una clase incumple el contrato que dijo cumplir.

Dos condiciones más, y las dos son deliberadas.

Solo biblioteca estándar. Este libro se fundamenta en sostener que el consumo de recursos importa; pedirte que instales media docena de paquetes para mover unos bichos por una pantalla sería contradecirlo. El juego se dibuja en el terminal y se ejecuta en cualquier sitio donde haya un Python.

Y con la interfaz más sencilla que se puede tener: Un invasor es una letra. Una bala, una barra. No hay sprites, ni animación, ni color.

1.6 Por qué implementar el modo auto-play

Y una decisión más, que no es de estilo y es la que más trabajo me ha ahorrado: **este juego debe poder jugar en modo automático.**

Son las dos mitades de una misma decisión.

La primera mitad: hay un jugador automático. Además de poder jugar tú —que llega en el capítulo 28—, el juego trae dentro alguien que mueve el cañón y dispara. Juega de una manera sencilla y siempre de la misma manera. No está ahí para lucirse: está para que una partida no necesite a nadie delante.

La segunda mitad: la semilla. Cada partida se juega a partir de un número, y de ese número sale todo lo que en el juego es azar: qué invasor suelta la bomba siguiente, cuándo cruza el OVNI, por qué lado entra. Con la misma semilla, la misma partida, hasta la última bomba.

Ahora el motivo. Si no puedo recorrer el juego entero —y no puedo—, lo único que me queda es jugarlo muchísimo. Y jugarlo muchísimo es imposible mientras haga falta una persona delante: y eso es un desafío que resolveremos para poder hacer pruebas y buscar calidad.

Con las dos mitades puestas, tres cosas que de otra manera no podría hacer:

- **Cientos de partidas seguidas, sin nadie mirando.** Se lanzan con un número al lado y devuelven un resumen. Ahí aparecen los defectos que necesitan mucha repetición para asomar, y aparecen temprano y se corrigen antes de que sea tarde y costoso.
- **Varias partidas a la vez, y a la vista.** El tablero se dibuja con letras y ocupa cuarenta y seis columnas, así que caben cuatro o seis ventanas una al lado de otra en la pantalla, cada una con su semilla, jugando en paralelo delante de ti. Cada ventana es un programa aparte y ninguna se entera de las demás. Un comportamiento raro que aparezca una vez cada veinte partidas lo ves en un minuto con seis tableros delante; jugando tú, puede que no lo veas nunca.
- **Y la partida que falle se puede repetir.** Esto es el valor que buscamos. Sin semilla, un fallo visto una vez está perdido: no sabes cómo volver a él, y acabas mirando el código a ciegas preguntándote si de verdad pasó. Con semilla, apuntas el número y vuelves a esa partida exacta tantas veces como quieras, con el fallo esperándote en el mismo turno. Eso es lo que convierte «he visto algo raro» en un defecto localizado.

Y las tres juntas mueven los defectos hacia el principio, que es donde se resuelven barato. Un fallo que encuentro con el juego a medio construir lo arreglo en el sitio donde lo puse; el mismo fallo encontrado al final me obliga a averiguar antes cuál de las veintitantas piezas lo provoca.

Detengámonos a pensar · Principio de calidad de software

Que muchas partidas salgan bien no demuestra que el juego esté bien. Demuestra que en *esas* partidas no falló, y no garantiza que no exista al menos una donde exista un error. Un defecto que solo aparezca con una combinación que ninguna de esas semillas llegó a producir sigue ahí, esperando a que alguien encuentre esas condiciones particulares que lo revelan.

Ya sabemos qué hay que construir, con qué, y con qué reglas. En el siguiente capítulo tomaremos acción, y vamos a identificar nuevos desafíos. Nuestra aventura en esta invasión comienza ahora. ¡A la carga!

Capítulo 2

Cuando todo lo sabe una sola función

Vamos a escribir el juego.

No la versión definitiva, sino la primera. La que sale cuando uno tiene las reglas delante y quiere ver los bichos moverse cuanto antes. Es como se escribe cuando no se ha decidido todavía nada sobre el diseño, que es exactamente el punto de partida de este libro.

Funciona. Se juega. Y revela nuevos requisitos que requieren ser atendidos.

2.1 La primera versión que todos escribimos

El razonamiento es directo y no tiene nada de malo. Hay que recordar dónde está cada invasor, así que hago una lista de posiciones. Hay que saber cuáles siguen vivos: otra lista. La puntuación es un número, las vidas otro, la bala tiene su posición. Todo eso son variables, y las variables viven en una función.

Después hay un bucle: mientras la partida siga, muevo lo que haya que mover, compruebo lo que haya que comprobar y dibujo.

Código 2.1: El estado de la partida, tal como sale la primera vez: una variable por cosa que hay que recordar.

```
def jugar(semilla=0, ver=False, turnos_maximos=TURNOS_MAXIMOS):
    azar = random.Random(semilla)

    canon_x = ANCHO // 2
    vidas = 3
    bala_x = -1
    bala_y = -1

    inv_x = []
    inv_y = []
    inv_fila = []
    inv_vivo = []
    for fila in range(FILAS_INVASORES):
        for col in range(COLUMNAS_INVASORES):
            inv_x.append(2 + col * 3)
            inv_y.append(1 + fila * 2)
            inv_fila.append(fila)
            inv_vivo.append(True)
    inv_dir = 1
    inv_paso = 0
    ...
```

Fíjate en las cuatro listas de invasores, porque van a dar guerra. El invasor número 7 no está en ningún sitio: está *repartido* entre `inv_x[7]`, `inv_y[7]`, `inv_fila[7]` y `inv_vivo[7]`. Se llaman listas paralelas, y son la forma más natural de guardar muchas cosas iguales cuando todavía no existe la idea de objeto.

Sin programar · Qué son las listas paralelas

Imagina una clase con treinta alumnos. El profesor cuenta con cuatro hojas de papel: en una, los nombres en orden; en otra, las notas en el mismo orden; en otra, las faltas; en otra, observaciones de cada uno.
Funciona perfectamente mientras las cuatro hojas estén en el mismo orden.
El alumno número 7 no existe como tal en ninguna parte. Existe la línea 7 de cuatro hojas distintas, y es el profesor quien las mantiene alineadas.

2.2 El bucle que lo hace todo

Con el estado montado, el bucle es la parte fácil de escribir y la difícil de leer. Va haciendo, por orden, todo lo que pasa en un tick del reloj.

Código 2.2: El bucle de la partida, con los comentarios que puse para orientarme. Que fueran necesarios, ya nos debe llamar la atención.

```
    while turno < turnos_maximos:
        turno += 1
        # ---------- cuantos quedan, y a que ritmo avanzan ----------
        # ---------- decision del jugador --------------------------
        # ---------- la bala propia sube ---------------------------
        # ---------- la formacion avanza ---------------------------
        # ---------- el rival dispara ------------------------------
        # ---------- las bombas bajan ------------------------------
        # ---------- el ovni ---------------------------------------
        # ---------- dibujar ---------------------------------------
```

Ocho apartados. Los comentarios los escribí mientras programaba, no después: sin ellos me perdía. Y esa es la primera señal, aunque todavía no sea una prueba de nada.

Detengámonos a pensar · Un comentario que hace de índice

Cuando un bloque de código necesita comentarios para saber por dónde vas, el comentario no está explicando algo difícil: está **sustituyendo a una estructura que es necesaria**. «La formación avanza» no es una aclaración. Es el nombre de algo que debería poder llamarse, y que aquí no se puede llamar porque no existe: es un tramo de función, no una cosa.

2.3 Las variables que hay que tener en la cabeza a la vez

Aquí es donde el capítulo deja de opinar y empieza a medir.

Todas las cifras que siguen salen de un programa, no de mi impresión.

Inténtalo primero · Adivina antes de mirar

La función `jugar()` declara variables locales: la puntuación, las vidas, las cuatro listas de invasores, la posición de la bala, los índices de los bucles, los acumuladores.

¿Cuántas variables dirías que hay que considerar a la vez?

Di un número antes de pasar a la tabla. En voz alta o por escrito, como en el libro anterior: solo así funciona.

Cuadro 2.1: El estado que hay que sostener para leer la función. Se cuentan los nombres distintos que la función asigna, más sus parámetros; se descartan los de bucle que no guardan nada.

Medida	Valor
Variables vivas a la vez en `jugar()`	52

Cincuenta y dos.

Y no son cincuenta y dos repartidas por el programa: son cincuenta y dos *en el mismo sitio*. Cualquier línea de esa función puede leer y escribir cualquiera de ellas. Cuando en la línea 200 pregunto por `inv_dir`, tengo que saber que ciento cuarenta líneas antes alguien pudo cambiarlo.

Qué es de verdad una función de 265 líneas

Lo que ves al leerlo

Se lee de arriba abajo. Cada tramo hace algo comprensible: mover, comprobar, dibujar. Ninguna línea es difícil.

Si te sientas con tiempo, entiendes lo que hace. No hay trucos.

Lo que eso significa

Para entender *una* línea cualquiera hay que conocer el estado de las 52 variables en ese punto.

No es que el código sea difícil: es que **no se puede leer por partes**. La unidad mínima de comprensión es la función entera, y la función entera no cabe de una vez en la cabeza de nadie.

2.4 Tres cambios pedidos, y lo que cuesta cada uno

Un programa no se juzga por lo que costó escribirlo. Se juzga por lo que cuesta el cambio siguiente, porque siempre hay uno.

Así que vamos a pedir tres cambios, que cualquiera hubiera pensado pedir tras jugar un rato. Los implementé sobre esta misma versión, los tres funcionan, y los tres están en el

repositorio para que puedas mirarlos.

1. **Un cuarto tipo de invasor**: uno acorazado, que aguanta dos impactos y otorga más puntos.
2. **Dos balas en el aire** en lugar de una.
3. **Los diez mejores marcadores**, guardados entre partidas.

Inténtalo primero · Antes de la tabla

Los tres cambios suenan pequeños. Elige uno —el de las dos balas es el más instructivo— y responde:
¿Cuántos **sitios distintos** del programa habría que tocar? No líneas: sitios. Tramos separados, que hay que encontrar uno por uno y ninguno se puede olvidar.

Cuadro 2.2: Coste de cada cambio sobre la versión de una sola pieza. «Sitios» son tramos separados del archivo: se cuentan sin contexto, y cada bloque contiguo de cambios es un sitio. Medido.

Cambio pedido	+líneas	−líneas	Sitios
Un cuarto tipo de invasor	16	3	6
Dos balas en el aire	43	39	4
Los diez mejores marcadores	27	0	6

Añadir un tipo de invasor obliga a tocar **seis** tramos separados de la función: donde se crean, donde la bala impacta, donde se reinicia la oleada, donde se dibuja. Ninguno de los seis está cerca de los otros. Y si te olvidas del de la oleada, el juego funciona perfectamente durante la primera oleada y falla en la segunda.

Detengámonos a pensar · Por qué «sitios» y no «líneas»

Cuarenta y tres líneas se escriben en un rato. Encontrar los cuatro tramos donde hay que escribirlas, no.
El trabajo de un cambio casi nunca está en teclear. Está en **acordarse de todos los rincones**, que requieren ser impactados.

2.5 La desproporción, otra vez, y en otro sitio

Quien haya leído el libro anterior reconocerá la escena, aunque el escenario sea distinto.

Allí, nueve casillas producían cuatro mil quinientos casos, y la desproporción estaba entre **el problema y su descripción**.

Aquí las reglas del juego caben en pocas líneas —están en el capítulo 1— y sostenerlas exige llevar 52 variables a la vez, y encontrar entre cuatro y seis sitios cada vez que alguien

pide un cambio. La desproporción ahora está entre **lo que el problema pide y lo que hay que recordar para atenderlo**.

Es la misma señal y significa lo mismo: la complicación no estaba en el juego. La hemos puesto nosotros al escribirlo así.

2.6 Y aquí no se puede recorrer el juego entero

Queda una diferencia con el libro anterior que conviene dejar sentada antes de seguir, porque cambia lo que este libro puede prometerte.

Allí, cuando dije que el algoritmo no perdía nunca, lo había comprobado jugando todas las partidas posibles. Aquí no puedo hacer eso, y no por falta de ganas: el capítulo 31 calcula cuántos estados tiene este juego, y el número no deja lugar a discusión.

De momento basta con una comprobación modesta y honesta, y es exactamente para esto para lo que decidí en el apartado «por qué implementar el modo auto-play» del capítulo 1 que las partidas no necesitaran a nadie delante. Jugué 500, con semillas del 0 al 499:

Cuadro 2.3: 500 partidas del juego de una sola pieza, semillas 0 a 499, con el jugador automático. Medido; reproducible con la misma orden.

Resultado	**Valor**
Puntuación media	1 515
Partidas que despejan al menos una oleada	46,8 %
Oleada más lejana alcanzada	5
Partidas perdidas porque la formación llegó abajo	497

Quinientas partidas no demuestran que el juego sea correcto. Demuestran que en esas quinientas no falló.

Detengámonos a pensar · Y sin embargo esa tanda encontró un defecto

Merece la pena contarlo, porque es la primera vez que este libro mide algo y ya sirvió para algo.

Al lanzar las primeras tandas, **ninguna** de las 500 partidas despejaba una oleada. Nunca. Lo cual significaba que el tramo de código que pasa a la oleada siguiente —crear la formación otra vez, bajarla una fila, sumar el bono— no se había ejecutado ni una sola vez.

La causa no era el juego: era que la velocidad de la bala estaba escrita en **dos sitios** que no se ven el uno al otro. En la física, que la hace subir; y en la puntería del jugador automático, que calcula cuánto tarda en llegar. Cambié uno y no el otro, y el jugador empezó a fallar todos los tiros sin que nada indicara por qué.

Sacar ese número a una constante —un cambio de tres líneas— pasó las partidas que despejan oleada del 0 % al 33 %.

Detengámonos a pensar · Y uno que la tanda no podía encontrar

Las quinientas partidas se juegan **sin dibujar**, porque nadie mira quinientas partidas. Ese detalle, que parece de comodidad, dejó fuera de la comprobación todo el segmento de código que pinta la pantalla.

Puse cuatro partidas en la pantalla para verlas jugar, y el programa se rompió. Las cuatro. Con cualquier semilla, y siempre en el mismo sitio: al pintar el cañón.

La causa. Cuando cruza el OVNI, el jugador automático apunta tres columnas por delante de él, para no llegar tarde. Si el OVNI se va por el borde, ese punto cae **fuera del tablero**, y el cañón lo persigue igual, porque nadie le había dicho dónde se acaba el tablero. Sin dibujar, salir del tablero no le molesta a nadie: el cañón se pasea por columnas que no existen y la partida sigue su curso. Al dibujar hay que pintarlo en una casilla que no está, y ahí se acaba todo.

Pasaba en **452 de las 500 partidas** —hasta tres columnas fuera— y ninguna lo delató. Esa cifra es de entonces y ya no se puede reproducir: el arreglo, que es topar el objetivo al ancho del tablero, cabe en media línea. Lo que sí se puede repetir es la comprobación, que quedó dentro del medidor de este capítulo y hoy dice cero.

Y como el cañón dejó de perseguir al OVNI fuera del tablero, la tabla de arriba cambió con él: antes daba 1 593 puntos de media y 49,6 % de partidas que despejan. Los datos de la tabla son medidos con código corregido.

La moraleja que destaco es —un tramo que nunca se ejecuta no está bien, está sin comprobar— **hay defectos que solo se ven mirando.**

Con esto tenemos un juego que funciona, unas cifras que lo describen y una señal que ya vimos en el libro anterior: la desproporción entre lo que el problema pide y lo que hay que sostener para atenderlo.

II

Los objetos

Capítulo 3

Clases y objetos

En el capítulo 1 te dejé tres preguntas y te pedí que guardaras las respuestas. Este capítulo empieza por ahí, porque las tres apuntan al mismo sitio.

3.1 La pregunta de este libro: ¿quién debería saber esto?

Las tres preguntas eran estas, y las contesto por orden.

Uno: ¿qué hay que recordar en cada instante? Lo contamos, y salieron 52 variables. Si tu cuenta es más corta, probablemente agrupaste —dijiste «los invasores» en lugar de enumerar sus cuatro listas paralelas—, y agrupar es exactamente lo que este capítulo va a hacer.

Dos: ¿quién decide que un invasor ha muerto? Casi todo el mundo responde «la bala» o «el invasor», y las dos respuestas son defendibles. La mía, después de darle vueltas, fue que **ninguno de los dos**: ni la bala sabe contra qué chocó, ni el invasor sabe que había una bala. Quien puede decidirlo es quien ve a los dos a la vez.

Tres: ¿quién sabe que la formación tocó el borde? Esta no tiene dos respuestas defendibles: tiene una. Ningún invasor conoce a los demás, así que ninguno puede saber si *alguno* tocó el borde. Lo sabe el conjunto. Y si el conjunto sabe algo que sus miembros no saben, el conjunto **es alguien**, no una lista.

Detengámonos a pensar · La pregunta, enunciada de una vez

Todo este libro es una sola pregunta repetida:

¿quién debería saber esto?

No «¿cómo lo programo?», que ya sabemos. No «¿dónde lo pongo?», que es la misma pregunta formulada distinto. Sino de quién es cada dato y de quién es cada decisión.

En el libro anterior la pregunta era si estábamos mirando el problema desde el sitio correcto. Esta es la misma un escalón más abajo, y hay que hacérsela muchas veces por capítulo en lugar de una vez por problema.

3.2 El molde y el objeto

Empiezo por lo más básico, que además es lo que más se confunde al explicarlo deprisa.

Definición 3.1 (Clase)**.** Una *clase* es la descripción de una familia de cosas: qué sabe cada una de ellas, qué sabe hacer, y qué tiene que cumplir para ser válida. Es su descripción.

Definición 3.2 (Objeto)**.** Un *objeto* es una de esas cosas, ya existiendo, con sus valores propios. Dos objetos de la misma clase comparten lo que deben saber y qué saben hacer, y no comparten lo que cada uno sabe.

El molde y el objeto

En el juego

`Invasor` es la clase: dice que un invasor tiene un sitio, un tipo y una resistencia, y que sabe avanzar, disparar y encajar un impacto.
En la pantalla hay 55 invasores. Cincuenta y cinco objetos, una sola clase. Cuando el de la esquina muere, no muere la clase: sigue habiendo cincuenta y cuatro.

En la vida real

Un automóvil es la clase: dice lo que un auto debe hacer y lo que debe tener para ser un auto.
Sin embargo, cada auto circulando en la vía es un objeto de esa clase: una instancia particular que tiene sus características propias, ocupa un espacio físico, y le corresponde un identificador único que lo distingue de los demás.

Sin programar · Cómo distinguirlos siempre

Hay una prueba rápida: **cuenta cuántos hay**.
Si de algo hay muchos a la vez y todos funcionan igual, eso son objetos y su descripción es la clase. Si de algo hay uno solo y siempre habrá uno solo, sospecha: puede que no necesites una clase, y el capítulo 25 se ocupa de lo que pasa cuando alguien insiste en que sea única.

3.3 Cuántos invasores hay y cuántas clases hacen falta

Cincuenta y cinco invasores, y una clase. Esa desproporción es el primer regalo, y conviene verla despacio porque explica para qué sirve todo esto.

En el capítulo anterior, los 55 invasores estaban repartidos entre cuatro listas paralelas: el invasor número 7 no existía en ninguna parte, existía la posición 7 de cuatro listas distintas. Aquí no.

Código 3.1: La clase. Lo que sigue es la descripción de los 55, no de uno.

```
class Invasor(Entidad):
    __slots__ = ("_tipo", "_fila", "_columna", "_resistencia", "_habilidad")

    def __init__(self, tipo, fila, columna, caja, habilidad=SIN_HABILIDAD):
        if fila < 0 or columna < 0:
            raise ValueError("un invasor nace en un sitio de la formacion")
        super().__init__(caja)
        self._tipo = tipo
        self._fila = fila
        self._columna = columna
        self._resistencia = tipo.resistencia
```

```
12          self._habilidad = habilidad
```

Detengámonos a pensar · Lo que se ha ganado, y no es escribir menos

Es tentador medir esto en líneas. No es ahí donde está la ganancia.

Lo que se ha ganado es que **el invasor número 7 ahora existe**. Es una cosa que se puede tomar, pasar a una función, usarlo en una lista, comparar y preguntar. Antes era una coordenada compartida por cuatro listas que alguien tenía que mantener alineadas, y ese alguien eras tú.

Y hay un efecto secundario que se nota enseguida: gobierno y gestión, con menor probabilidad de error.

3.4 Qué no es un objeto: la trampa de la clase por cada sustantivo

Aquí es donde se equivoca casi todo el mundo la primera vez, y me incluyo.

La receta que se enseña —subraya los sustantivos del enunciado y haz una clase con cada uno— produce, aplicada a nuestro juego: `Invasor`, `Formación`, `Cañón`, `Bala`, `Bomba`, `Búnker`, `OVNI`, `Pantalla`, `Puntuación`, `Vida`, `Oleada`, `Turno`, `Partida`, `Jugador`, `Fila`, `Columna`.

Dieciséis clases. La mitad sobran.

Inténtalo primero · Antes de que te lo diga

De esa lista de dieciséis, tacha las que **no** deberían ser una clase.

Y para cada una que taches, di por qué. La razón importa más que el acierto: hay al menos tres motivos distintos por los que un sustantivo no merece una clase, y encontrarlos es el ejercicio.

Ahora, te presento mis tachones y sus motivos:

Cuadro 3.1: Sustantivos que no merecen una clase, y por qué.

Sustantivo	Motivo	Qué es en realidad
Puntuación, Vida, Turno, Oleada	Es un **dato**, no una cosa	Un número que alguien guarda. La puntuación es de la partida, las vidas son del cañón
Fila, Columna	Es una **coordenada**	Parte de dónde está un invasor, no una entidad con vida propia
Pantalla	Es **otro oficio**	No pertenece al juego: pertenece a quien lo dibuja, más adelante lo abordaremos.
Jugador	**No existe dentro del juego**	Quien juega está fuera. Lo que existe dentro es el cañón, que lleva a cabo lo que decides

Quedan siete, y son las que sobreviven en esta primera incursión: `Invasor`, `Formacion`, `Canon`, `Bunker`, `Ovni`, los proyectiles y `Partida`.

Detengámonos a pensar · El criterio

Un sustantivo merece una clase cuando cumple las dos:

Tiene estado propio que nadie más debería tocar. Un invasor tiene su posición y su resistencia, y son suyas.

Y tiene comportamiento que depende de ese estado. Un invasor decide si un impacto lo mata, y lo decide mirando sus atributos propios.

Si solo cumple la primera, es un dato de otro. Si solo cumple la segunda, es una función y no le hace falta disfraz —y el capítulo 34 enseña lo que pasa cuando se le pone uno—.

3.5 La prueba de la frase: decir qué hace cada objeto sin usar «y»

Termino con la herramienta más útil de este capítulo, y cabe en una regla de juego.

Describe lo que hace cada clase en una frase. Si necesitas la palabra «y», tienes dos clases.

Sobre las nuestras:

Cuadro 3.2: La prueba de la frase, aplicada. La última fila es la que no pasa, y está ahí a propósito.

Clase	Qué hace, en una frase
`Invasor`	Guarda dónde está y decide si un impacto lo mata
`Formacion`	Mueve el bloque y sabe cuándo tocó el borde
`Canon`	Obedece órdenes de movimiento y de disparo
`Bunker`	Encaja impactos hasta que se deshace
`Partida`	Hace avanzar el turno, resuelve los impactos, lleva la puntuación, decide cuándo se acaba *y* cuándo empieza otra oleada

Fíjate en las tres primeras: todas llevan una «y». No he hecho trampa retirándolas, y prefiero explicar por qué las dejo.

Un invasor guarda su sitio *y* decide sobre su muerte, pero las dos cosas son la misma: decide sobre su muerte **mirando lo suyo**. La «y» que delata no es cualquier «y»: es la que une dos cosas que cambiarían por motivos distintos.

La última fila, en cambio, no tiene defensa. `Partida` lleva cuatro «y», y son cuatro «y» de las malas. Lo sé mientras la escribo, y aun así la dejo así durante buena parte del libro, porque repartirla antes de que duela sería enseñarte la respuesta antes del reto.

Si esto fuera un equipo · La persona que lo sabe todo

Cambia «clase» por «persona» y la prueba de la frase sigue funcionando.

Si al describir el puesto de alguien necesitas cuatro «y» —«lleva la facturación *y* atiende el teléfono *y* conoce a los proveedores *y* es la única que sabe cerrar el mes»—, no tienes un puesto: tienes cuatro, apilados sobre una persona.

Y lo que ocurre es exactamente lo que le ocurre a `Partida`: funciona perfectamente hasta que hay que cambiar algo, y entonces resulta que no se puede tocar una de las cuatro cosas sin tocar las otras tres. Cuando esa persona falta, no falta una: faltan cuatro.

Y aquí el aviso, porque este libro no vende humo: que las dos cosas se *describan* igual no demuestra que se *comporten* igual. Un equipo no es un programa, las personas no son objetos y el parecido sirve para pensar, no para deducir. Lo uso como lo que es: una ilustración.

Prompt · Repartir, no programar

```
Voy a construir un Space Invaders y quiero decidir el reparto de
responsabilidades ANTES de escribir código.

Estas son las reglas del juego: [pégalas aquí].

No escribas código todavía. Haz esto, en este orden:

1. Cuestióname el planteamiento. ¿Hay alguna regla ambigua, contradictoria o
   que dependa de algo que no he dicho? Pregúntamelo antes de seguir.

2. Cuando lo hayamos aclarado, propón la lista MÍNIMA de clases. Para cada
   una, una frase de qué hace, sin usar la palabra "y".

3. Señálame las que no pasen esa prueba, y por qué.

No añadas nada que no te haya pedido. Si crees que falta algo, dímelo en lugar
de añadirlo.
```

Compruébalo · La prueba de la frase sobre algo tuyo

Piensa en un programa y haz la lista de sus componentes.

Escribe una frase por componente. Cuenta las «y».

Después, para cada «y», hazte la única pregunta que importa: *¿podrían pedirme que cambie una de las dos cosas sin la otra?* Si la respuesta es sí, ahí hay dos componentes, no solo uno.

Ya tenemos siete clases y una pregunta con la que decidir si merecen serlo. Lo que todavía no tenemos es qué guarda cada una: dije que un invasor «tiene un sitio, un tipo y una

resistencia».

El capítulo siguiente separa tres cosas que casi siempre se llaman igual —dato, información y conocimiento— y de esa distinción sale el criterio para decidir qué guarda un objeto y qué no debería guardar nunca.

Capítulo 4

Atributos: lo que un objeto sabe

Decidir mal lo que una clase guarda es la forma más silenciosa de estropear un diseño: un atributo de más no rompe nada, no da error y no se nota hasta que hay que cambiar algo. Este capítulo separa primero tres cosas que casi siempre se llaman igual, porque de esa separación se manifiesta el criterio.

4.1 Dato, información y conocimiento

Los tres escalones, sobre el mismo juego.

Definición 4.1 (Dato)**.** Un *dato* es un valor sin contexto. No significa nada por sí solo y no permite decidir nada.

Definición 4.2 (Información)**.** La *información* es un dato con su contexto: qué es, de quién, en qué unidad. Permite responder preguntas, pero no dice qué hacer.

Definición 4.3 (Conocimiento)**.** El *conocimiento* es lo que permite *decidir*: relaciona varias informaciones y produce una acción.

Cuadro 4.1: Los tres escalones sobre el mismo número. Solo el tercero permite jugar.

Escalón	Ejemplo	Qué permite
Dato	`2`	Nada. Podría ser vidas, columnas, puntos o tener cualquier significado
Información	«al cañón le quedan 2 vidas»	Responder «¿cuántas vidas le quedan?»
Conocimiento	«con 2 vidas y la formación en la fila 15, conviene no perder el búnker izquierdo»	Decidir la jugada

Detengámonos a pensar · Y esto gobierna el libro entero

La distinción no es filosofía: reparte el trabajo.

Los atributos guardan datos. Un número, un estado, una posición.

El objeto los convierte en información, porque les pone nombre y dueño: ese `2` deja de ser un `2` y pasa a ser *las vidas de este cañón*.

Y el conocimiento vive en el comportamiento, no en los atributos. La regla de que la formación baja al tocar el borde no es un dato de nadie: es algo que la formación *sabe hacer*.
Corolario práctico, y para el diseño de software se usa habitualmente: **si algo se puede deducir, no se guarda**. Guardarlo lo convierte en un segundo sitio donde la verdad puede estar equivocada.

Los tres escalones

En el juego

El `3` que guarda un búnker es un dato. «A este búnker le quedan 3 impactos» es información: ya tiene dueño y unidad. «Quedan 3 impactos y la oleada va por la fila 15, así que ese búnker no llega al final» es conocimiento, y no está escrito en ninguna variable: se deduce.

En la vida real

El número de una historia clínica es un dato.
«El paciente tiene 38,2 de temperatura» es información.
«Con 38,2 y esos síntomas, hay que hacer una prueba» es conocimiento, y por eso lo tiene el médico y no la hoja.
La hoja guarda; quien decide es quien sabe. Confundirlo lleva a formularios que pretenden decidir y a médicos que solo los rellenan.

4.2 Estado propio frente a estado prestado

Con eso ya se puede decidir qué guarda cada objeto. Un atributo se justifica cuando cumple las dos condiciones:

Definición 4.4 (Atributo)**.** Un *atributo* es un dato que un objeto guarda porque **le pertenece** —nadie más debería cambiarlo— y porque **no se puede deducir** de lo que ya tiene.

La segunda condición es la que más atributos elimina, y la que más cuesta aplicar, porque guardar siempre parece más barato que calcular.

Inténtalo primero · Antes de ver mi lista

Escribe los atributos que le pondrías a un invasor. Todos los que se te ocurran, sin filtrar. Después tacha los que se puedan deducir de los demás.
Casi siempre sobra alguno, y el que sobra suele ser el que parecía más evidente.

4.3 Los atributos del invasor, del cañón y de la bala

Esto es lo que guardé, y por qué.

Código 4.1: Lo que un invasor sabe. Cinco cosas, y ninguna se puede deducir de las otras cuatro.

```
class Invasor(Entidad):
    __slots__ = ("_tipo", "_fila", "_columna", "_resistencia", "_habilidad")
```

Cuadro 4.2: Por qué cada atributo se queda. La columna de la derecha es la que hace el trabajo: si se pudiera deducir, no estaría.

Atributo	Qué guarda	Por qué no se deduce
`_tipo`	Pulpo, cangrejo, calamar o acorazado	Es una decisión de quien lo creó
`_fila`	Su fila en la formación	Su posición en pantalla cambia; su fila, no
`_columna`	Su columna	La necesita la formación para saber quién dispara
`_resistencia`	Impactos que le quedan	Cambia con el juego. Nace del tipo, pero deja de coincidir en cuanto le dan
`_habilidad`	Escudo, o ninguna	Se la otorgan desde fuera (cap. 26)

Y estos son los que **no** están, que es la parte interesante:

Los puntos que vale. Se deducen del tipo. Un calamar vale 30 porque es un calamar, no porque lleve un 30 encima. Guardarlo permitiría que existiera un calamar que vale 10, y eso no es un invasor válido: es un error esperando.

Si está vivo. Lo guarda `Entidad`, su clase base, porque eso lo tienen todos por igual.

Su posición en pantalla. También la guarda `Entidad`, y como una caja, no como dos números. El capítulo 12 explica por qué una caja y no un punto.

Si es el más bajo de su columna. Esto es el ejemplo bueno. Parece un atributo del invasor y no lo es: **depende de quiénes sigan vivos**, y eso cambia cada vez que muere alguien. Es una consulta a la formación, y la formación la responde recorriendo.

Detengámonos a pensar · El atributo que más veces he tenido que quitar

Es siempre el mismo tipo: el que *resume* algo que ya está en otro sitio.

«Cuántos invasores quedan vivos» parece que hay que guardarlo, porque contarlo cincuenta veces por segundo suena caro. Y en cuanto lo guardas tienes dos verdades: la lista y el contador. Basta con que un camino de muerte se olvide de restar uno para que el juego crea que quedan invasores donde no hay ninguno, y no habrá error, ni aviso, ni nada: solo una oleada que no termina nunca.

La regla que sigo: **primero deducir, y guardar solo cuando la medida diga que hace falta**. Con medida, no con intuición.

4.4 Lo que parece un atributo y es una cuenta

Merece la pena mirar el caso concreto, porque es el que más se repite.

Código 4.2: No es un atributo: es una pregunta que se responde recorriendo.

```
class Formacion:
    @property
    def vivos(self):
        return sum(1 for i in self._invasores if i.vivo)

    @property
    def cadencia(self):
        """Turnos entre paso y paso. Cuantos menos quedan, mas deprisa."""
        return 1 + self.vivos // self._divisor_cadencia
```

Fíjate en `cadencia`: es conocimiento puro. No guarda nada, relaciona una información —cuántos quedan— con una decisión —cada cuántos turnos avanzar—. Si la hubiera guardado como atributo, habría tenido que acordarme de recalcularla cada vez que muere un invasor.

Sin programar · Lo mismo, sin código

Imagina una lista de asistentes a un curso y, en la esquina de la hoja, un número: «total: 24».

En cuanto alguien se da de baja hay dos verdades en la misma hoja, y una de las dos se va a quedar desactualizada. La que se queda desactualizada es siempre el número de la esquina, porque tacharlo y volver a escribirlo es un paso que se olvida.

La alternativa es no escribirlo y contar la lista cuando alguien pregunte. Cuesta unos segundos y no puede mentir.

4.5 Datos desnudos: por qué un entero no dice qué significa

Queda el último problema de este capítulo, y es de los que Python no te va a avisar.

En el juego de una sola pieza, el tipo de un invasor era su fila: 0, 1, 2. Y los puntos salían de una tabla indexada por ese número. Funcionaba. Y tenía tres defectos que no daban la cara:

1. Un `2` no dice si es un tipo, una columna o unas vidas.
2. Nada impide escribir `7` donde solo hay tres tipos.
3. Y si alguien confunde el orden de la tabla, los puntos salen mal sin que el programa deje de ejecutarse.

La solución que elegí usa un tipo que Python sí tiene, y no cuesta memoria:

Código 4.3: Un tipo propio en lugar de un entero desnudo. Sigue siendo un entero por dentro.

```
class TipoInvasor(IntEnum):
    PULPO = 0
    CANGREJO = 1
    CALAMAR = 2
    ACORAZADO = 3

    @property
    def puntos(self):
        return (10, 20, 30, 50)[self]

    @property
    def resistencia(self):
        return 2 if self is TipoInvasor.ACORAZADO else 1
```

Ahora `TipoInvasor.CALAMAR` dice qué es, los puntos viven pegados al tipo que los justifica, y `TipoInvasor(7)` da error en el momento de escribirlo en lugar de dar puntos raros tres oleadas después.

Detengámonos a pensar · Nombrar no cuesta memoria, y conviene saberlo desde ahora

Un `IntEnum` **es** un entero: se puede sumar, comparar y guardar en un `array` de un byte por elemento exactamente igual que un `0`.

Lo digo aquí porque el capítulo 19 va a discutir el consumo de memoria en serio, y quiero dejar cerrado desde ya que ponerle nombre a un dato no es un lujo que se pague en bytes. Se paga en otra cosa —una búsqueda de atributo en tiempo de ejecución— y eso también lo mediré allí.

Lo que sí ahorra es la clase de error que nadie encuentra leyendo.

Si esto fuera un equipo · El dato sin dueño

Un número suelto en una hoja de cálculo compartida —«1400»— es el equivalente exacto del entero desnudo. Nadie sabe si son puntos, unidades u horas, ni de qué mes. Habitualmente, se le olvida incluso a quién lo puso.

Y lo que ocurre es lo mismo que en el código: el error no aparece al escribirlo. Aparece cuando alguien lo usa para decidir, meses después, y para entonces ya no hay a quién preguntarle.

Ponerle nombre, unidad y dueño a un dato cuesta un minuto y es lo que lo convierte en información.

Compruébalo · Los tres escalones, en algo tuyo

Toma una tabla que uses —de trabajo, de estudio, la que sea— y clasifica tres de sus columnas:

¿Cuál es un **dato** que no significa nada sin su cabecera? ¿Cuál es **información** completa? ¿Y dónde está el **conocimiento**, es decir, quién decide algo mirando eso?

Si el conocimiento no está en ninguna parte de la tabla, es que está en la cabeza de alguien. Eso no está mal; conviene saberlo.

Con esto, cada objeto ya sabe lo justo: ni un dato prestado ni un resumen que pueda quedarse sin actualizar.

Falta lo otro, y es la mitad que de verdad distingue un objeto de una ficha: qué sabe *hacer*. Porque hasta aquí un invasor no es más que un registro con nombre, y un registro no juega.

Capítulo 5

Métodos: lo que un objeto sabe hacer

Un invasor con cinco atributos y nada más es una ficha. Guarda datos, no juega.

Lo que convierte una ficha en un objeto es que sepa hacer algo, y sobre todo que *lo haga él*. Este capítulo trata de esa diferencia, que parece de matiz y decide el diseño entero.

5.1 Pedir, no manipular

Hay dos maneras de mover un invasor a la derecha. Las dos funcionan.

Dos maneras de mover un invasor

Manipular desde fuera

```
invasor.x = invasor.x + 1
```

Quien lo escribe tiene que saber que la posición se guarda en `x`, que se mide en celdas y que sumar uno es moverse a la derecha.

Y como cualquiera puede hacerlo, la regla de que un invasor no puede salirse del tablero hay que repetirla en todos los sitios donde alguien lo mueva.

Pedírselo a él

```
invasor.desplazar(dx=1)
```

Quien lo escribe solo tiene que saber que quiere moverlo a la derecha.

Dónde guarda su posición, en qué unidad y qué comprueba antes es asunto del invasor. Y si mañana la guarda de otra manera, esta línea no cambia.

Definición 5.1 (Método)**.** Un *método* es algo que un objeto sabe hacer consigo mismo. Se le pide; no se le hace.

La diferencia no está en la sintaxis. Está en **quién tiene que saber cómo funciona por dentro**.

Detengámonos a pensar · La prueba del cambio interno

Cómo saber, sin discutir, si un método está bien puesto:

cambia la representación interna del objeto y mira cuántas líneas de fuera se rompen.

Cuando pasé de guardar la posición como dos números a guardarla como una caja —y lo hice a mitad del libro, porque las colisiones lo pedían—, no se rompió ni una línea fuera de la clase. Ni una. Eso no fue suerte: fue que nadie de fuera tocaba `x` directamente.

Si al hacer ese experimento se te rompen veinte sitios, no tienes métodos: tienes funciones que modifican datos ajenos.

5.2 Preguntar y ordenar son cosas distintas

Segunda distinción, y esta ordena la lista de métodos de cualquier clase.

Un método o bien **responde** algo sin cambiar nada, o bien **hace** algo y cambia el estado. Mezclar las dos cosas en un mismo método es la forma más rápida de que nadie se fíe de tu código.

Cuadro 5.1: Los dos tipos de método, y qué se espera de cada uno.

	Consulta	**Orden**
Qué hace	Responde	Cambia el estado
Cambia algo	No, nunca	Sí, es su oficio
Se puede repetir	Sí, da lo mismo	No: dos veces son dos efectos
Ejemplo	`formacion.vivos`	`invasor.desplazar(dx=1)`

Inténtalo primero · Una que rompe la regla

Mira este método, que llegué a escribir:

> `def quedan_vivos(self):` — cuenta los invasores vivos, y de paso borra de la lista los que ya murieron para que la próxima cuenta sea más rápida.

Suena razonable. Di al menos dos cosas que se rompen por culpa de ese «y de paso», antes de seguir leyendo.

Las que yo encontré fueron tres. Que llamar dos veces a una pregunta dé resultados distintos. Que dibujar la pantalla —que solo pregunta— acabe modificando la partida. Y que sea imposible escribir una prueba que compruebe el recuento sin alterar aquello que está comprobando.

Ninguna de las tres da error. Le resta calidad y mantenibilidad al código.

Sin programar · La regla, sin código

Si alguien te pregunta la hora, no mueves las manecillas del reloj.

Una pregunta que cambia lo preguntado deja de ser una pregunta. Y lo peor no es que esté mal: es que quien la usa no tiene forma de enterarse.

5.3 Los métodos del invasor, uno a uno

Con las dos distinciones puestas, esta es la lista completa de lo que un invasor sabe hacer.

Código 5.1: Todo lo que un invasor sabe hacer. Lo que no está en esta lista, no se le puede pedir.

```
1     def avanzar(self, mundo):
```

```
        """El invasor no decide cuando moverse: lo decide su formacion."""
        self._habilidad.al_avanzar(self, mundo)

    def desplazar(self, dx=0, dy=0):
        self._caja = self._caja.desplazada(dx, dy)

    def recibir_impacto(self):
        """Devuelve True si el impacto lo mata. El escudo puede comerselo."""
        if self._habilidad.absorbe_impacto(self):
            return False
        self._resistencia -= 1
        if self._resistencia <= 0:
            self.morir()
            return True
        return False

    def posicion_de_disparo(self):
        c = self.caja()
        return c.x, c.abajo + 1
```

Tres cosas de esa lista merecen comentario, y la tercera responde una de las preguntas que dejé abiertas en el capítulo 1.

No hay `morir()` público. Lo hay heredado, pero nadie de fuera lo llama. A un invasor no se le dice que se muera: se le dice que ha recibido un impacto, y él decide. Es la diferencia entre ordenar el resultado y ordenar el suceso, y aquí importa mucho porque puede llevar un escudo puesto.

`avanzar()` no mueve. Parece un error de nombre y es deliberado. Un invasor no decide cuándo desplazarse: eso lo decide la formación, que es quien conoce la cadencia. Lo que hace `avanzar()` es dar a su habilidad la oportunidad de actuar. Si un invasor pudiera moverse por su cuenta, la formación dejaría de ser una formación.

Y `recibir_impacto()` no detecta nada. Esta es la respuesta a la pregunta del capítulo 1: *¿quién decide que un invasor ha muerto?*

Detengámonos a pensar · Quién decide un impacto, contestado

Ni la bala ni el invasor, y la razón es la misma para los dos: **ninguno de ellos puede verlo**.

La bala conoce su caja y hacia dónde va. No conoce a los invasores; si los conociera, tendría que conocer también los búnkeres, el OVNI y el cañón, y una bala que conoce el juego entero no es una bala.

El invasor conoce su caja. No conoce las balas, y por el mismo motivo.

Lo ve quien tiene a los dos delante, y en este juego es la partida. Ella comprueba el solape, y a cada uno le dice lo suyo: a la bala, que se ha resuelto; al invasor, que ha recibido un impacto. Cada uno decide sobre lo suyo.

Esto que acabo de describir tiene nombre —asignación de responsabilidades— y tiene un

capítulo entero, el 13. Lo que hay que retener ahora es la forma de la respuesta: cuando dos objetos no pueden saber algo por separado, lo sabe quien los ve a los dos. Nunca uno de los dos «un poco».

Si esto fuera un equipo · Dictar el resultado o confiar el trabajo

Es la diferencia entre decirle a alguien «márcalo como cerrado» y decirle «este expediente ha llegado; míralo».

En el primer caso decides tú y esa persona ejecuta; si había un motivo para no cerrarlo, se pierde. En el segundo decide quien tiene la información delante.

En el juego pasa literalmente eso: si alguien de fuera pudiera matar al invasor, el escudo no serviría de nada, porque el escudo lo sabe el invasor.

5.4 Métodos que devuelven un objeto nuevo en vez de cambiar el actual

Queda un tipo de método que no cambia nada y tampoco es una consulta: el que devuelve otro objeto.

Código 5.2: La caja no se mueve: produce otra caja. Es un objeto de valor.

```
class Caja(NamedTuple):
    x: int
    y: int
    ancho: int = 1
    alto: int = 1

    def desplazada(self, dx=0, dy=0):
        """Otra caja, no esta movida: los objetos de valor no cambian."""
        return Caja(self.x + dx, self.y + dy, self.ancho, self.alto)
```

Reflexioné bastante antes de escribirlo así, porque la alternativa —`caja.mover(dx)`— es más corta y más obvia. Lo que me convenció fue darme cuenta de que una caja no es una cosa del juego: es una *medida*.

Detengámonos a pensar · Cuándo conviene devolver uno nuevo

Regla que uso, y funciona sin excepciones en este libro:

Si el objeto representa algo que existe en el juego —un invasor, un búnker—, se le pide que cambie. Hay un invasor y sigue siendo el mismo después de moverse.

Si el objeto representa un valor —una posición, una caja, un tipo—, se produce otro. Igual que sumar 1 a un 3 no cambia el 3: da un 4.

La prueba es una pregunta: *¿tiene sentido decir «el mismo, pero distinto»?* De un invasor sí. De una caja no: una caja en otro sitio ya es otra caja.

Esto tiene una consecuencia práctica que se cobra sola más adelante: dos cajas con los mismos números son intercambiables, se pueden comparar sin sorpresas y nadie puede modificar una desde el otro extremo del programa sin que su dueño se entere. Cuando en el capítulo 31 tenga que repetir partidas exactamente, esa propiedad será la que lo haga posible.

Prompt · Métodos

```
Tengo esta clase con sus atributos: [pégala].

Antes de escribir nada, cuestióname: ¿hay algún atributo que sobre, o alguno
que se pueda deducir de los demás? Dímelo y espera mi respuesta.

Después propón la lista de métodos, y para cada uno indica:
- si es CONSULTA (responde y no cambia nada) u ORDEN (cambia el estado);
- Quien lo llame, qué tendría que saber de los detalles de la clase.

Regla que quiero comprobar: nadie de fuera debería necesitar saber cómo se
guarda nada por dentro. Señálame los métodos que incumplan eso.

Escribe solo lo que te he pedido. Si crees que falta un método, propónmelo en
una lista aparte en vez de añadirlo.
```

Compruébalo · La prueba del cambio interno, tú mismo

Piensa en una clase tuya y cambia cómo guarda un dato por dentro: dos números por una tupla, una lista por un diccionario, lo que sea. No cambies lo que hace.
Cuenta cuántos sitios de fuera hay que tocar.
Cero es la respuesta buena. Cualquier otra te dice exactamente dónde se está interviniendo por fuera lo que debería pedirse.

Con atributos y métodos, un invasor ya es un objeto: sabe cosas y sabe hacer cosas, y nadie tiene que poderlo modificar desde fuera.

Capítulo 6

Constructores: nacer ya válido

El capítulo anterior deja un agujero abierto: nada impide crear un invasor en la fila -3, ni una formación vacía, ni un cañón con cero vidas.

Vamos a cubrir este agujero a lo largo de este capítulo.

6.1 Objetos que no deberían poder existir

Empiezo por el inventario del desastre. Estos cuatro objetos se pueden crear hoy, ninguno da error, y los cuatro rompen el juego más tarde:

Cuadro 6.1: Objetos imposibles que el programa acepta sin alertas ni errores de compilación. La columna de la derecha es la que hace daño: el fallo no aparece donde se creó.

Se puede crear	**Y revienta...**
Un invasor en la fila -3	Al dibujar, fuera del lienzo. O nunca, y queda uno invisible que impide despejar la oleada
Una formación vacía	Al preguntarle su cadencia: divide entre cero invasores vivos
Un cañón con cero vidas	En el primer impacto, con vidas $= -1$, que nunca llega a la condición «≤ 0» si está mal escrita
Una bala fuera del tablero	Nunca. Vuela hacia arriba desde ninguna parte y no impacta con nada

Fíjate en la última columna, porque es el patrón: **ninguno falla donde se creó**. Fallan lejos, más tarde, y con un error que no menciona la causa. El error dirá «división por cero en cadencia», y el problema estaba en quien creó la formación, tres archivos y doscientos turnos antes.

Detengámonos a pensar · La distancia entre la causa y el síntoma

Es la medida real de lo que cuesta un fallo, y casi nadie la mira.

Un fallo que salta en el mismo sitio donde está la causa es barato: lo ves y lo arreglas.

Un fallo que salta a doscientos turnos de distancia cuesta una tarde, y buena parte de esa tarde se va en descartar sitios inocentes.

Todo lo que este capítulo propone tiene un solo objetivo: **acercar el síntoma a la causa**. No evitar errores —eso no lo promete nadie—, sino hacer que salten donde se cometen.

6.2 Invariantes de clase: lo que se garantiza en el nacimiento

Definición 6.1 (Invariante de clase)**.** Un *invariante* de una clase es algo que es cierto de todos sus objetos desde que nacen hasta que dejan de existir. No es una comprobación que se hace de vez en cuando: es una promesa que el objeto no puede incumplir.

Los de nuestro juego:

Cuadro 6.2: Los invariantes del juego. Cada uno se garantiza en un solo sitio, y ese sitio es el constructor de quien hace la promesa.

Clase	Lo que promete, siempre
`Invasor`	Nace en un sitio real de la formación: fila y columna no negativas
`Formacion`	Tiene al menos un invasor. Una formación vacía no es una formación: es el final de la oleada
`Canon`	Nace con al menos una vida
`Caja`	Nunca cambia. Dos cajas iguales son intercambiables para siempre

Código 6.1: El invariante, escrito donde se puede garantizar: en el nacimiento.

```
class Invasor(Entidad):
    def __init__(self, tipo, fila, columna, caja, habilidad=SIN_HABILIDAD):
        if fila < 0 or columna < 0:
            raise ValueError("un invasor nace en un sitio de la formacion")
        ...

class Formacion:
    def __init__(self, invasores, divisor_cadencia=6):
        if not invasores:
            raise ValueError("una formacion sin invasores no es una formacion")
        ...

class Canon(Entidad):
    def __init__(self, caja, vidas=3, arma=None, habilidad=SIN_HABILIDAD):
        if vidas <= 0:
            raise ValueError("un canon nace con al menos una vida")
        ...
```

Tres líneas de comprobación. Lo que compran es que a partir de ese punto **ningún otro método tiene que preguntar**. `cadencia` no comprueba si hay invasores; `recibir_impacto` no comprueba si las vidas eran positivas.

Dos formas de tratar una regla

Comprobar en cada uso

Cada método que use la formación comprueba primero si está vacía.
La comprobación se repite en ocho sitios, hay que acordarse de ponerla en el noveno, y cuando falta, el fallo aparece justo ahí.
Y peor: leyendo un método no sabes si esa comprobación está porque puede pasar o porque alguien fue precavido.

Garantizar al nacer

La formación no puede estar vacía. Punto.
La comprobación está en un solo sitio y los ocho métodos se escriben sin ella.
Y hay un efecto que no esperaba: **el código se vuelve más corto y a la vez más explícito**, porque lo que queda escrito son las reglas del juego y no las defensas contra estados imposibles.

Inténtalo primero · Antes de que siga

Nuestro juego tiene una regla que aún no he convertido en invariante: *solo puede haber una bala del cañón en el aire a la vez.*
¿Dónde la pondrías? Y una pregunta más difícil: ¿es un invariante de la bala, del cañón o de la partida?
Piénsalo. La respuesta dice bastante sobre cómo repartir responsabilidades.

No es de la bala: una bala no sabe cuántas hay. No es exactamente del cañón, aunque el arma diga cuántas caben. Es de la partida, porque es quien tiene la lista de proyectiles —y por eso el cañón, antes de fabricar una bala, le pregunta:

Código 6.2: El cañón sabe cuántas caben; la partida sabe cuántas hay. Ninguno de los dos sabe lo del otro.

```
    def disparar(self, mundo):
        if mundo.balas_en_vuelo() >= self._arma.balas_maximas:
            return None
        return Bala(self._caja.x, self._caja.y - 1, self._arma.velocidad)
```

6.3 El constructor como contrato de entrada

Con esto, un constructor deja de ser «el sitio donde se rellenan los atributos» y pasa a ser otra cosa:

Definición 6.2 (Constructor)**.** Un *constructor* es el contrato de entrada de una clase: dice qué hace falta para que uno de sus objetos exista, y rechaza lo que no cumpla. Es el único sitio donde un objeto puede estar a medio hacer.

Y de ahí sale un criterio para los parámetros, que es lo que más me costó ajustar:

Lo que un objeto no puede deducir ni suponer, entra por el constructor. Un invasor no puede adivinar su columna. Un cañón sí puede suponer que tiene tres vidas, y por eso ese parámetro tiene valor por omisión.

Detengámonos a pensar · Lo que entra por el constructor no es una lista de datos

Mira otra vez el constructor de la partida:

```
def __init__(self, semilla=0, fabrica=None, estrategia=None,
             sucesos=None, arma=None, ...):
```

Ahí no entran solo números: entran *piezas*. Quién fabrica las oleadas, cómo dispara la invasión, quién escucha lo que pasa. La partida no fabrica ninguna de esas cosas; las recibe.

Eso tiene nombre —inyección de dependencias—, resuelve un problema muy concreto y tiene su capítulo, el 26. Lo dejo asomando aquí porque el constructor es donde ocurre, y porque quiero que cuando llegues allí reconozcas la forma en lugar de aprenderla.

Y el motivo por el que la semilla entra por ahí en lugar de crearse dentro es el que sostiene medio libro: sin eso, ninguna partida se puede repetir, y sin repetirla no se puede comprobar nada.

6.4 Lo que no va en un constructor

Termino por el lado contrario, porque un constructor que hace demasiado es tan malo como uno que no comprueba nada. Listamos entonces lo que no debe ir en un constructor.

No va trabajo pesado. Crear un objeto debe ser barato. Si construir una partida tardara medio segundo, las 500 partidas del capítulo 31 tardarían cuatro minutos más sin haber jugado ni un turno.

No van efectos hacia fuera. Un constructor que abre un archivo, o que imprime, o que se registra en una lista global, convierte «crear» en «hacer», y a partir de ahí no se puede crear uno para mirarlo.

No va lógica del juego. La fábrica de oleadas decide qué invasores lleva la oleada 7; el constructor de `Invasor` no decide nada, solo comprueba. Cuando un constructor empieza a decidir, es que alguien debería ser una fábrica.

Sin programar · El formulario que no deja seguir

Un formulario bien hecho no te deja pasar de página si falta un dato obligatorio. Uno mal hecho te deja rellenarlo, enviarlo, y te avisa tres semanas después cuando alguien intenta tramitarlo.

Los dos «comprueban». La diferencia es *cuándo*, y esa diferencia son tres semanas.

Un constructor con invariantes es el primero. Y de la misma manera que un formulario no debería ponerse a llamar por teléfono mientras lo rellenas, tampoco un constructor hace trabajo: solo comprueba y deja al objeto listo.

Si esto fuera un equipo · Lo que se pide al empezar

Un encargo que empieza sin fecha, sin destinatario y sin criterio de aceptación es el equivalente exacto del invasor en la fila −3: se acepta sin problema, y el problema aparece al final, cuando ya no hay margen.

Y el paralelo se sostiene también por el otro lado: la reunión de arranque no es el sitio para hacer el trabajo. Es el sitio para comprobar que están las condiciones. Ilustración, no demostración.

Compruébalo · Los estados imposibles de algo tuyo

Piensa en una clase tuya y escribe tres objetos que **no deberían poder existir**: valores contradictorios, listas vacías donde hace falta al menos uno, fechas al revés.

Ahora intenta crearlos de verdad. Si el programa los acepta, tienes tres invariantes sin escribir.

Y para cada uno hazte la pregunta que importa: si alguien lo creara hoy, *¿dónde reventaría?* La distancia entre esos dos sitios es lo que te va a costar.

Con esto se cierra la Parte II. Un objeto ya sabe lo justo, sabe hacer lo suyo y no puede nacer roto: es una pieza en la que se puede confiar.

Pero solo se puede confiar en ella mientras nadie lo modifique desde fuera. Todo lo del capítulo anterior —pedir en vez de manipular— y todo lo de este —nacer válido— se viene abajo en cuanto alguien escribe `invasor._resistencia = 99` desde fuera. Hoy nada se lo impide.

La sección siguiente se ocupa de las dos decisiones que lo impiden: qué se publica de un objeto, y qué se esconde.

III

Lo que no se muestra no se vende

Capítulo 7

Encapsulamiento

Todo lo de la parte anterior descansa en una suposición que todavía no he justificado: que nadie va a intervenir desde fuera un objeto.

Hoy nada lo impide. `invasor._resistencia = 99` funciona, no da error, y deja al invasor en un estado que su constructor jamás habría permitido. Este capítulo aborda el porqué conviene asegurar esa puerta y cómo cerrarla.

7.1 La frontera: qué se ve desde fuera

Definición 7.1 (Encapsulamiento)**.** El *encapsulamiento* es la decisión de qué parte de un objeto es asunto de los demás y qué parte es solo suya. Lo de dentro puede cambiar sin avisar; lo de fuera es una promesa.

Fíjate en que la definición no dice «esconder datos». Dice *decidir*. Un objeto encapsulado no es uno que oculta cosas por pudor: es uno que ha trazado una frontera y responde de lo que hay a un lado.

Cuadro 7.1: La frontera de un invasor. Lo de la izquierda es una promesa que no puedo romper sin avisar; lo de la derecha lo cambio cuando quiera.

Fuera: lo que prometo	**Dentro: lo que es asunto mío**
`caja()`, `vivo`, `tipo`, `puntos`, `columna`, `fila`	Que la posición se guarde como una caja y no como dos números
`desplazar()`, `recibir_impacto()`, `avanzar()`	Que la resistencia se descuente antes o después de consultar la habilidad
`posicion_de_disparo()`	Que los puntos se determinen según el tipo en lugar de guardarse

Qué significa la frontera

En el juego

Le pides al invasor que encaje un impacto. No sabes —ni debes saber— si le queda resistencia, si lleva escudo o si el escudo se gasta.
Y como depende solo del invasor, cuando añadí los escudos no tuve que tocar ni una línea de quien dispara.

En la vida real

Sacas dinero de un cajero. No editas tu saldo: pides un retiro, y el banco decide si puede.
Y por eso cuando el banco añadió un límite diario no tuvo que cambiar tu tarjeta. La operación que tú pides es la misma; lo que cambió está del otro lado de la frontera.

7.2 Quién puede romper una regla del juego, y por qué no debería poder

Aquí es donde el encapsulamiento deja de ser estilo y pasa a ser una defensa concreta.

En el capítulo 6 escribí tres invariantes y dije que a partir de ahí ningún método tendría que comprobar nada. Eso solo es verdad si los invariantes no se pueden romper después.

Inténtalo primero · Rompe el juego en una línea

Con lo que sabes de nuestras clases, escribe **una sola línea** que deje la partida en un estado imposible, sin usar ningún constructor.
Se puede hacer de al menos cuatro maneras. Encuentra dos antes de seguir.

Las mías:

Cuadro 7.2: Cuatro líneas que rompen invariantes por la puerta de atrás. Ninguna da error al ejecutarse.

La línea	Qué invariante rompe
`invasor._resistencia = 99`	El acorazado deja de morir. La oleada no termina nunca
`canon._vidas = 0`	El cañón está vivo con cero vidas: nunca dispara el final de partida
`formacion._invasores.clear()`	Formación vacía. `cadencia` divide entre cero
`invasor._caja = Caja(-5, 40)`	Un invasor fuera del tablero, invisible y vivo

Las cuatro tienen la misma forma: **saltarse al objeto y escribir en su memoria**. Y las cuatro rompen el juego en otro sitio y mucho más tarde, que es justo donde identificarlos y resolverlos genera costos mayores.

Detengámonos a pensar · El invariante y la frontera son la misma decisión

Un invariante sin frontera no es un invariante: es una recomendación.
El constructor puede comprobar lo que quiera. Si después cualquiera puede escribir directamente en los atributos, lo que el constructor garantizaba dura exactamente hasta la primera línea que lo ignore.
Por eso van juntos y por eso los puse en capítulos seguidos. Encapsular no es una preferencia estética: es lo que convierte una comprobación en una garantía.

7.3 Cambiar por dentro sin que nadie se entere

La segunda ganancia es la que se cobra a lo largo del tiempo.

Empecé guardando la posición de cada entidad como dos números, `x` e `y`. Cuando llegué a las colisiones me di cuenta de que un búnker ocupa cuatro celdas y un punto no basta, así que cambié los dos números por una caja con ancho y alto.

Es un cambio de fondo: cambia cómo se guarda la posición de todo lo que existe en el juego.

Código 7.1: Antes y después del cambio. Lo de fuera no se enteró.

```
# Antes
class Entidad:
    __slots__ = ("_x", "_y", "_vivo")

# Despues
class Entidad(ABC):
    __slots__ = ("_caja", "_vivo")

    def caja(self):
        return self._caja
```

Detengámonos a pensar · Cuántas líneas de fuera hubo que tocar

Ninguna.
Y no fue suerte. Fue que nadie de fuera leía `x` ni `y`: todos pedían la caja o pedían un desplazamiento. La frontera estaba trazada, y el cambio se quedó dentro.
Compáralo con lo mismo en el juego de una sola pieza. Allí la posición de un invasor son `inv_x[i]` e `inv_y[i]`, leídas desde el bucle de movimiento, el de colisiones, el de disparo del rival y el dibujado. El mismo cambio habría tocado los cuatro, y el capítulo 2 ya midió lo que cuesta un cambio de cuatro sitios.

7.4 Lo que el encapsulamiento *no* es: poner guiones bajos

Y ahora la parte incómoda, que en Python hay que mirar de frente.

Python no impide nada. El guion bajo de `_resistencia` no es una cerradura: es un cartel. Cualquiera puede escribirlo desde fuera y el intérprete no dirá una palabra. Las cuatro líneas de la tabla 7.2 funcionan hoy en nuestro juego.

Podría haberlo disimulado. Prefiero decirlo y sacar de ahí la lección, porque es más útil que la cerradura:

Detengámonos a pensar · En Python el encapsulamiento es un contrato, no un muro

Y un contrato tiene una propiedad que un muro no tiene: **se puede leer**.

El guion bajo dice «esto es asunto mío; si lo tocas, el día que lo cambie se te romperá y será tu problema». Eso no es menos que una cerradura: es otra cosa. Reparte la responsabilidad en lugar de impedir el acto.

Lo que sí exige es disciplina, y una comprobación que se puede automatizar: **contar cuántas veces se toca un atributo privado de otra clase**. Se mide recorriendo el árbol sintáctico del juego entero, y en el capítulo 32 sale otra vez junto al resto del acoplamiento.

Y hay dos cosas más que tampoco son encapsulamiento, y son las que más veces he visto confundir:

No es poner un `get` y un `set` a cada atributo. Eso es la misma puerta con dos pasos más. Si `set_resistencia(99)` existe, la resistencia sigue siendo pública y encima el objeto ya no puede defenderse. Un invasor no tiene `set_resistencia`: tiene `recibir_impacto`, que es una *operación del juego* y no un acceso a memoria.

Y no es esconder por esconder. `tipo`, `puntos` y `columna` son públicos a propósito: la formación necesita la columna para saber quién dispara, y la partida necesita los puntos para sumarlos. Lo que está fuera está porque alguien lo necesita.

Sin programar · La diferencia entre los dos carteles

Un cartel de «privado» en una puerta de oficina no cierra nada. Y sin embargo funciona casi siempre, porque quien entra sabe que asume lo que pase.

Ahora imagina otro cartel: «recepción». Ese no esconde: *invita*, y dice por dónde se pasa. Un objeto bien encapsulado tiene los dos. Y la mayoría de los objetos mal diseñados que he visto no fallaban por tener pocos carteles de «privado»: fallaban por no tener ninguno de «recepción», y entonces cada cual entraba por donde podía.

Si esto fuera un equipo · Cada quien tiene acceso a lo que debe y solo a lo que debe

Cuando alguien necesita algo de otro equipo y, en vez de pedirlo, entra en su carpeta compartida y lo toma, el trabajo sale. Esa vez.

Lo que se rompe es la posibilidad de que el otro equipo cambie nada. En cuanto reorganicen sus carpetas, algo que ellos no sabían que existía dejará de funcionar, y no se enterarán hasta que alguien se queje.

Es exactamente `invasor._resistencia = 99`: funciona, y convierte un detalle interno en

una dependencia que nadie declaró. Ilustración, no demostración: las personas negocian y los objetos no.

Prompt · Encontrar lo que se muestra y lo que se desea cubrir

```
Te paso estas clases: [pégalas].

No refactorices nada todavía. Haz solo esto:

1. Lista cada sitio donde código de FUERA de una clase lee o escribe un
   atributo que empieza por guion bajo. Dime archivo y línea.

2. Para cada uno, dime qué invariante de esa clase podría romperse.

3. Propón, para cada uno, qué método público debería existir en su lugar.
   Solo el nombre y qué haría; no lo implementes.

Si no encuentras ninguno, dímelo y dime cuántos sitios has revisado. No quiero
una felicitación: quiero saber qué has mirado.
```

Compruébalo · Cuenta tus puertas

Sobre un programa tuyo, busca los accesos a atributos privados desde fuera de su clase. En Python basta con buscar el patrón `._` precedido de un nombre de variable.
El número que salga es la medida de cuánto puedes cambiar por dentro sin romperle nada a nadie. Cuanto más alto, menos.
Y no lo estimes: cuéntalo. Es la diferencia entre creer que tu código está bien encapsulado y saberlo.

Con la frontera trazada, cada objeto es una pieza fiable: promete poco, cumple lo que promete y puede cambiar por dentro.

Queda la otra mitad de la decisión, y es la que da nombre a todo lo que este libro persigue. Porque decidir qué se esconde presupone que alguien ya decidió *qué merecía existir*: por qué hay una clase `Entidad` y no seis clases sin nada en común, y por qué el bucle del juego puede actualizar cincuenta cosas distintas sin saber qué son.

Capítulo 8

Abstracción

Este es el capítulo que da nombre a lo que el título del libro promete.

Invadir el espacio del computador es llevar tus necesidades ahí dentro. Y eso no se hace copiando la realidad: se hace quedándose con la parte de la realidad que decide, y descartando el resto. A eso lo llamamos abstraer, y es lo que separa un programa que resuelve tu problema de uno que no atiende correctamente una necesidad.

8.1 Llevar un problema real al programa

Definición 8.1 (Abstracción)**.** Una *abstracción* es una descripción que conserva lo que decide y descarta lo demás. Es útil cuando permite razonar y actuar sobre muchos casos con una sola idea.

Fíjate en las dos mitades, porque las dos hacen falta. Conservar lo que decide: si descartas algo que importaba, tienes un programa que se equivoca. Descartar lo demás: si no ignoras nada, tienes una copia de la realidad, y una copia de la realidad es tan difícil de manejar como la realidad misma.

Qué se conserva y qué se tira

En el juego

Un invasor de verdad, si existiera, tendría color, sonido, una animación de dos fotogramas y una nave. Nada de eso decide. Lo que decide es: dónde está, si sigue vivo, cuánto vale y cuánto aguanta. Cinco datos. El resto es decorado, y el decorado no resuelve el problema: se atiende en otra etapa del software, en el diseño de la interfaz visual.

En la vida real

Un ticket aéreo no necesita conocer tu color favorito, dónde estudiaste ni el nombre de tu mejor amigo. Lleva origen, destino, fecha, hora, asiento y código del vuelo. Nadie se queja de que el ticket no te describa. Todos entendemos que un ticket no es una persona: es *lo que hace falta saber* para que seas un pasajero válido en un avión.

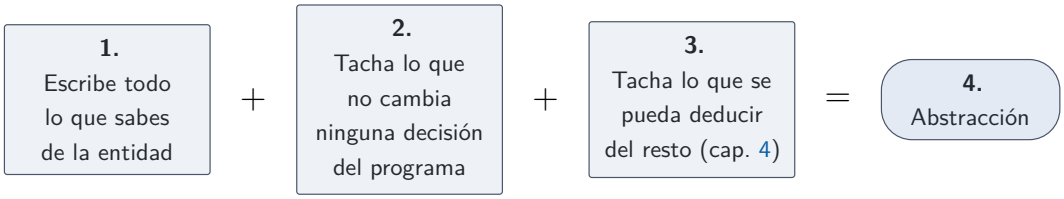
Podemos asegurar que programar bien es emitir buenos tickets.

Detengámonos a pensar · La dirección importa, y este libro va en una sola

Tú invades el espacio del computador, no al revés. Cada vez que en este libro aparece una clase, apareció porque el juego la pedía. Ninguna apareció porque tocaba.

8.2 Quedarse con lo que decide y descartar lo demás

La operación tiene un procedimiento, y es más mecánico de lo que parece. Tres pasos que se suman, y lo que queda al final es la abstracción.



Sobre un búnker, que es el caso más pequeño del juego:

Cuadro 8.1: Abstraer un búnker. De once cosas que se pueden decir de él, sobreviven tres y más adelante usaremos una cuarta.

Lo que se puede decir	¿Decide?	Por qué
Dónde está y cuánto ocupa	**Sí**	Sin eso no hay colisión posible
Cuántos impactos aguanta	**Sí**	Decide cuándo desaparece
Si tiene una habilidad puesta	**Sí**	Puede regenerarse (cap. 26)
De qué color es	No	Es de la vista
Su forma	No	Es de la vista
El ruido al recibir un impacto	No	Es de quien escucha el suceso
Cuánto aguantaba al empezar	**Sí, uno**	Hace falta para repararse, y solo para eso
Cuántos impactos ha recibido	No	Se deduce del inicial menos el actual
Si está entero, tocado o casi roto	No	Se deduce de la resistencia
Quien lo construyó	No	Para el juego, solo importa que existe
Su número de búnker	No	No importa para el juego

Once cosas, cuatro atributos. Y fíjate en las dos filas que más enseñan: «si está entero o casi roto» *parece* información del búnker y es una cuenta; «su número» parece imprescindible y no lo pregunta nadie.

Inténtalo primero · Hazlo con el OVNI

El OVNI cruza por arriba, vale cien puntos y desaparece al salir por el otro lado.
Escribe todo lo que se puede decir de él. Después tacha.
Te aviso de una trampa: hay una cosa que parece decorado y sí decide. Si te salen menos de tres atributos, la has descartado.

8.3 Entidad: lo que el bucle necesita saber, y nada más

Y ahora la abstracción que sostiene el juego entero, que es la que menos se ve porque no representa ninguna cosa concreta.

El bucle de la partida tiene que hacer avanzar el tiempo. ¿Sobre qué? Sobre invasores, balas, bombas, búnkeres, el OVNI y el cañón. Seis clases que no se parecen en nada: una baja, otra sube, otra no se mueve, otra responde a lo que le pides.

La pregunta que me hice fue: *¿qué necesita saber el bucle de cada una de ellas?*

Código 8.1: Todo lo que el juego necesita saber de cualquier cosa que exista en el tablero.

```
class Entidad(ABC):
    """Algo que ocupa un sitio y avanza con el tiempo."""

    __slots__ = ("_caja", "_vivo")

    def caja(self):
        return self._caja

    @property
    def vivo(self):
        return self._vivo

    def morir(self):
        self._vivo = False

    @abstractmethod
    def avanzar(self, mundo):
        """Un paso de tiempo. Lo que eso significa lo decide cada entidad."""

    def simbolo(self):
        return "?"
```

Ocupar un sitio, seguir existiendo, avanzar un paso y saber cómo se dibuja. Con eso, el bucle puede gobernar a sesenta objetos distintos sin preguntarle a ninguno qué es.

Detengámonos a pensar · Lo que desaparece cuando la abstracción es buena

Es la misma señal que aprendí en el libro anterior, y aquí vuelve con otra cara: **la buena noticia no es lo que se añade, es lo que deja de hacer falta**.

Sin `Entidad`, el bucle tendría que preguntar de qué tipo es cada cosa para saber qué hacer con ella: una cadena de condicionales que crece con cada clase nueva. Con ella, esa cadena no existe. No se ha optimizado: ha dejado de ser necesaria.

Y hay una segunda que se nota al dibujar. La vista recorre las entidades, les pide su caja y su símbolo, y **no sabe qué está dibujando**. En el momento en que añadí el acorazado no toqué la vista. Ese es el rendimiento real de una abstracción, y se mide contando lo que no hubo que tocar.

8.4 La prueba que separa una abstracción de una metáfora

Necesito un criterio, porque «quedarse con lo esencial» suena bien y no distingue nada. El libro anterior dejó uno y aquí lo uso tal cual:

Detengámonos a pensar · La prueba, en una pregunta

¿Puedo deducir algo trabajando sobre la abstracción, y que siga siendo verdad al volver al problema?
Si sí, es una abstracción.
Si solo sirve para *contar* el problema de otra manera, es una metáfora. Las metáforas son magníficas para explicar y no resuelven nada.

Sobre `Entidad`: puedo razonar «recorro las entidades vivas, les pido que avancen, y las que salieron del tablero se marcan muertas», deducir que el bucle no necesita conocer ninguna clase concreta, escribirlo así, y funciona. Deducción hecha sobre la abstracción, válida en el problema. Es una abstracción.

8.5 Abstraer de menos y abstraer de más

Los dos fallos existen y no son simétricos.

Abstraer de menos es lo que hacía el juego de una sola pieza: no había ninguna abstracción, y cada cosa se manejaba con sus datos crudos. Se reconoce porque el mismo tipo de trabajo se repite con distintos nombres, y porque añadir algo obliga a tocar muchos sitios.

Abstraer de más es lo contrario y hace menos ruido, que es lo que lo hace peor. Se reconoce por tres señales:

1. La abstracción tiene **una sola implementación**, y no se ve cuál sería la segunda.
2. Para entender qué hace algo hay que abrir tres archivos.
3. Cuando llega un cambio, hay que tocar la abstracción *y* la implementación. Es decir: no aisló nada.

Sin programar · Cuándo un formulario general estorba

Una empresa con tres tipos de cliente puede hacer tres formularios, o uno general con casillas que unos rellenan y otros no.
El general parece mejor: uno en lugar de tres. Y funciona mientras las diferencias sean pocas. En cuanto la mitad de las casillas dice «solo para el caso B», el formulario general ya no simplifica: obliga a todos a leer las instrucciones de todos.
La pregunta es siempre la misma: *¿esto me deja tratar los casos igual, o solo los pone en la misma hoja?* Lo primero es abstraer. Lo segundo es apilar.

Si esto fuera un equipo · El procedimiento que sirve para todo

Un procedimiento general que cubre las cuatro situaciones habituales hace que cualquiera pueda resolverlas sin preguntar. Eso es abstraer bien.
Un procedimiento que cubre las cuatro habituales y once excepciones no ahorra nada: para usarlo hay que saber en cuál de los quince casos estás, que es exactamente lo que no sabías.

Prompt · Abstraer desde el problema, no desde el lenguaje

```
Voy a describirte una necesidad real y quiero que me ayudes a decidir qué hace
falta saber para atenderla. No quiero clases todavía.

La necesidad es: [descríbela en términos de qué tiene que ocurrir].

Paso 1. Cuestióname: ¿hay algo en mi descripción que sea una decisión de
implementación disfrazada de requisito? Señálamelo y espera.

Paso 2. Lista todo lo que se puede decir del asunto. Todo, sin filtrar.

Paso 3. Para cada cosa, dime si CAMBIA alguna decisión del programa. Si no la
cambia, propón tacharla y di de quién es en realidad.

Paso 4. De lo que quede, dime qué se puede deducir de lo demás.

No escribas código. No propongas nombres de clases. Solo la lista que
sobreviva.
```

Compruébalo · La prueba de la deducción

Escoge una abstracción tuya —una clase base, una interfaz, un concepto que hayas introducido— y responde por escrito:
¿Qué puedo deducir razonando solo sobre ella, sin mirar ninguna implementación concreta?
Si la respuesta es «nada, hay que mirar las subclases», no es una abstracción: es un cajón con etiqueta.

Con esto se cierra la Parte III y, con ella, el aparato básico: objetos que saben lo justo, que solo se dejan pedir cosas y que se describen por lo que decide.

Lo que viene ahora son las cuatro palabras que más se confunden entre sí, y las que más diseños han estropeado por usarse en el orden equivocado. Empiezo por la más tentadora de todas: la que promete ahorrarte trabajo diciendo que dos cosas se parecen.

IV

Parecerse es diferente de SER

Capítulo 9

Herencia

La herencia es la herramienta más tentadora de la orientación a objetos, y la que más diseños he visto estropear. Las dos cosas por el mismo motivo: promete ahorrarte trabajo en cuanto dos cosas se parecen.

9.1 Tres tipos de invasor que se mueven igual

En el juego hay tres tipos de invasor —pulpo, cangrejo y calamar— que se mueven igual, mueren igual, disparan igual y solo se diferencian en cuántos puntos valen y en cómo se dibujan.

Inténtalo primero · ¿Tres clases o una?

Antes de seguir: ¿harías `Pulpo`, `Cangrejo` y `Calamar` heredando de `Invasor`?
Y la pregunta que de verdad importa: *¿qué escribirías dentro de cada una de las tres?*

Mi respuesta fue que no, y la razón es la de la pregunta: **dentro de las tres subclases no habría nada**. Solo un número distinto y una letra distinta.

Detengámonos a pensar · Herencia sin comportamiento propio es una constante con ceremonia

Tres clases vacías que solo cambian un valor no son una jerarquía: son una tabla escrita en el sitio más caro posible.
Lo que hice fue lo contrario: un solo `Invasor`, y la diferencia entre los tres en un `IntEnum` que ya conoces del capítulo 4.

> Regla que uso desde entonces: **si una subclase no redefine ningún comportamiento, no es una subclase. Es un valor.**

Y sin embargo el juego sí tiene herencia, en dos sitios, y conviene mirar por qué esos dos sí:

Cuadro 9.1: Dónde hay herencia en el juego, y qué aporta cada caso.

Jerarquía	Qué aporta
`Entidad` → todo lo que ocupa un sitio	Comportamiento común de verdad: vivir, morir, tener caja. Cada subclase redefine `avanzar()`, y son cinco maneras distintas de avanzar
`Proyectil` → `Bala`, `Bomba`	Los dos vuelan y salen del tablero igual. Y difieren en algo real: la bala resuelve impactos mientras sube; la bomba, al terminar de bajar

9.2 Qué se hereda de verdad: comportamiento, no parecido

Definición 9.1 (Herencia)**.** La *herencia* es la declaración de que una clase es un caso particular de otra: hace todo lo que hace la otra, y algo lo hace a su manera. No es una declaración de parecido: es una declaración de **sustituibilidad**.

⚖ Cuándo la herencia dice la verdad

En el juego

`Bala` hereda de `Proyectil` porque en cualquier sitio donde el juego espere un proyectil, una bala sirve: tiene caja, vuela, sale del tablero y muere.
El bucle no necesita saber cuál de los dos es. Esa es la prueba.

En la vida real

Un contrato temporal hereda del convenio general. Tiene las mismas obligaciones y algunas condiciones propias.
Define horario de trabajo y salario. Sin embargo, no necesita volver a mencionar condiciones generales como código de vestimenta ni normas de conducta.

9.3 El OVNI no es un invasor, aunque lo parezca

Este es el caso que más me hizo dudar.

El OVNI aparece arriba, se mueve solo, da puntos al dispararle y desaparece. Los invasores aparecen arriba, se mueven solos, dan puntos al dispararles y desaparecen. Escrito así, hacer `Ovni(Invasor)` parece obvio.

Estuve a punto. Lo que me detuvo fue hacer la lista de lo que tendría que desactivar:

Cuadro 9.2: Lo que el OVNI tendría que ignorar si heredara de `Invasor`. Cinco promesas incumplidas.

Un invasor...	**y el OVNI...**
Pertenece a una formación	no pertenece a ninguna
Tiene fila y columna	no tiene, y el constructor las exige
Baja cuando la formación toca el borde	cruza en horizontal y nunca baja
Puede disparar bombas	no dispara nunca
Cuenta para despejar la oleada	no cuenta: puedes ganar sin verlo

Cinco de las cosas que definen a un invasor, y el OVNI no cumple ninguna.

Detengámonos a pensar · La prueba que uso para decidirlo, y cabe en una frase

No es «¿se parecen?». Es:

¿puedo poner el hijo donde se espera el padre y que nadie lo note?

Si la formación recibiera un OVNI, le pediría su columna y reventaría. Si el juego contara invasores vivos para saber si la oleada está despejada, contaría el OVNI y la oleada no terminaría nunca.

Así que no: el OVNI hereda de `Entidad`, como todo lo demás, y ahí se acaba. Comparte con el invasor exactamente lo que comparte con una bala —ocupa un sitio y avanza—, y eso ya estaba dicho un nivel más arriba.

Sin programar · El error de clasificar por parecido

Una ballena y un tiburón se parecen mucho más entre sí que una ballena y un murciélago. Y sin embargo la ballena y el murciélago están en la misma familia. Algo similar sería querer asegurar que un mosquito que ha perdido sus alas es una araña.

Clasificar por lo que se ve produce categorías cómodas y falsas. Clasificar por lo que se hereda de verdad produce categorías que sirven para predecir.

En programación pasa igual, con una diferencia: la categoría falsa no te avisa. Funciona durante meses y falla el día que alguien trata al hijo como trataría al padre.

9.4 La tentación: una subclase por cada habilidad

Y ahora el caso que hace daño de verdad, porque no se ve venir.

En el capítulo 26 el juego va a tener habilidades: un invasor con escudo, otro más rápido, un búnker que se regenera. La forma natural de añadir la primera es una subclase:

Código 9.1: El primer paso por el camino equivocado. Este es razonable. El problema es el segundo.

```
class InvasorConEscudo(Invasor):
    def recibir_impacto(self):
```

```
        if self._escudo > 0:
            self._escudo -= 1
            return False
        return super().recibir_impacto()
```

Funciona. Es corto. Y ahora llega la segunda habilidad.

Inténtalo primero · Cuenta antes de que lo cuente yo

Tienes 4 tipos de invasor y quieres poder darles 3 habilidades, en cualquier combinación, incluida ninguna.

¿Cuántas subclases hacen falta?

Da el número antes de seguir. Y después la pregunta buena: ¿cuántas harían falta con una cuarta habilidad?

La cuenta no tiene truco: cada habilidad puede estar o no estar, así que hay 2^h combinaciones, y cada una multiplica por los t tipos.

Cuadro 9.3: Clases necesarias para cubrir todas las combinaciones. Calculado con $t \cdot 2^h$ para la herencia y $t + h$ para la composición.

Tipos	Habilidades	Con herencia	Con composición
4	1	8	5
4	2	16	6
4	3	32	7
4	4	64	8
4	6	256	10

Treinta y dos clases para cuatro tipos y tres habilidades. Doscientas cincuenta y seis si algún día hay seis.

Detengámonos a pensar · Lo que crece no es el trabajo: es lo que puede fallar

Sesenta y cuatro clases no son sesenta y cuatro veces más trabajo de escribir —la mayoría se generarían casi solas— y esa es justamente la trampa.

Lo que crece es otra cosa: **el número de sitios donde una corrección hay que aplicar**. Arregla un fallo en cómo se encaja un impacto y tendrás que comprobar sesenta y cuatro clases para saber cuáles lo heredan bien.

Y crece una segunda cosa, peor: la explosión te obliga a decidir por adelantado. `InvasorRapidoConEscudo` tiene que existir *antes* de que nadie pida esa combinación. Estás pagando hoy por combinaciones que quizá nadie use nunca.

La salida no es esta. La salida es dejar de preguntarse qué *es* un invasor con escudo y preguntarse qué *tiene*, y de eso trata el capítulo 11.

9.5 Por qué las jerarquías profundas salen caras

Termino con el otro fallo de la misma familia, que es el de encadenar niveles.

Imagina esta jerarquía, que es la que aparece sola si te dejas llevar:

```
Entidad -> Movil -> Enemigo -> Invasor -> InvasorAcorazado
```

Cinco niveles, y cada uno se justifica de la misma manera: añade algo. Lo que no se ve al añadirlos es que cada nivel cobra tres peajes:

Para entender una clase hay que leer cinco. Y no en orden: hay que ir hacia arriba buscando quién define qué, y hacia abajo comprobando quién lo redefine.

Cada nivel puede romper la promesa del anterior. Con cinco niveles hay cuatro sitios donde alguien pudo cambiar el trato sin que se note desde fuera.

Y lo peor: un cambio en la raíz llega a todos, quieras o no. Añadir un parámetro a `Entidad` obliga a revisar sus veinte descendientes. La herencia no es reutilización: es **acoplamiento permanente** hacia arriba.

Detengámonos a pensar · Nuestra jerarquía, medida

El juego terminado tiene 59 clases y su jerarquía más profunda es de cuatro niveles: `Entidad` → `Proyectil` → `Bala` → `BalaPerforante`. El cuarto escalón lo añade la Parte VII, y llega con su justificación: una bala que atraviesa *es* una bala y redefine qué le pasa al impactar (capítulo 29).

No es una casualidad: es la consecuencia de haber respondido «no» las tres veces que la herencia parecía la salida fácil. Los tipos de invasor son un valor, el OVNI es un hermano y las habilidades se componen.

Lo mido aquí y lo vuelvo a medir en el capítulo 32, porque la profundidad de la jerarquía es de las pocas cosas que sí predicen bien lo que va a costar entender un programa.

Si esto fuera un equipo · El organigrama de cinco niveles

Cada nivel de una jerarquía de mando añade alguien que puede matizar lo que dijo el de arriba.

Con dos niveles, quien recibe una instrucción sabe de dónde viene y qué significa. Con cinco, la instrucción llega abajo con cuatro matices superpuestos, y para entender por qué hay que hacer algo hay que reconstruir la cadena entera.

Compruébalo · Mide tu jerarquía más profunda

En un programa tuyo, busca la clase con más antepasados y cuéntalos.

Después haz el ejercicio honesto: toma esa clase y trata de explicar qué hace **sin abrir ninguna de sus antepasadas**.

Si no puedes, el número que has contado es el número de archivos que alguien tendrá que leer para modificarla.

La herencia queda entonces en su sitio. Medido sobre el juego terminado: hay siete jerarquías y **cinco de las siete tienen dos niveles**, que es una base abstracta y sus implementaciones. Solo dos pasan de ahí: `Entidad`, con cuatro, y `Arma`, con tres. Y una regla: no hereda quien se parece, sino quien puede sustituir.

Lo que todavía no he explicado es qué se gana exactamente con eso, porque hasta aquí la herencia solo ha servido para no repetir código, y eso es lo de menos. Lo que de verdad compra es que una sola línea del bucle pueda gobernar a sesenta objetos distintos sin saber qué son. De eso trata el capítulo siguiente.

Capítulo 10

Polimorfismo

Este es el capítulo donde la orientación a objetos empieza a pagar.

Hasta aquí lo que hemos hecho es ordenar: cada objeto sabe lo suyo, nadie interviene donde no debe. Está bien, y no justifica el esfuerzo por sí solo. Lo que lo justifica es lo que viene ahora, y se ve mejor comparándolo con el juego de una sola pieza.

10.1 Una sola orden para entidades distintas

Definición 10.1 (Polimorfismo)**.** El *polimorfismo* es la propiedad por la que una misma orden produce comportamientos distintos según a quién la dirija, sin que quien la da tenga que saber a quién se la está dando.

La segunda mitad es la que importa. Que un invasor y una bala avancen distinto no tiene mérito. El mérito es que **quien les dice que avancen no sepa cuál es cuál**.

Una orden, muchas ejecuciones

En el juego

El bucle recorre los proyectiles y les dice `avanzar()`.

La bala sube tres celdas y comprueba impactos por el camino. La bomba baja una y comprueba al llegar.

El bucle no sabe cuál tiene delante, y no le hace falta.

En la vida real

El director dice «firmen aquí» en una reunión.

Cada persona firma con su letra, su rúbrica y su ritmo. Nadie le explica al director cómo firma cada cual.

La orden es una. Las ejecuciones son tantas como personas. Y si mañana entra alguien nuevo, la orden sigue valiendo sin cambiarla.

10.2 Varias naves agresoras, un solo bucle

Lo concreto. Estos son los seis tipos de cosa que existen a la vez en el tablero, y lo que significa «avanzar» para cada uno:

Cuadro 10.1: Seis maneras de avanzar. Ninguna aparece en el bucle de la partida.

Entidad	Qué hace al avanzar
`Invasor`	No se mueve: da a su habilidad la oportunidad de actuar
`Formacion`	Cuenta los turnos, y al llegar a su cadencia mueve el bloque o lo baja
`Bala`	Sube tres celdas de una en una, resolviendo impactos en cada paso
`Bomba`	Baja una celda y resuelve impactos al final
`Bunker`	No se mueve: se repara si tiene regeneración
`Ovni`	Cruza en su sentido y muere al salir del tablero

Seis comportamientos. Y este es el código que los gobierna:

Código 10.1: El bucle de la partida. Busca en él los nombres de las seis clases anteriores.

```
    def avanzar(self):
        if not self.en_juego:
            return
        self.turno += 1

        for p in list(self.proyectiles):
            if p.vivo:
                p.avanzar(self)
        self.proyectiles = [p for p in self.proyectiles if p.vivo]

        if self.formacion.despejada:
            self._siguiente_oleada()
            return

        self.formacion.avanzar(self)
        for b in self.bunkeres:
            b.avanzar(self)
```

No están. No aparece `Bala`, ni `Bomba`, ni `Invasor`. Aparece `avanzar()`, tres veces, y cada una se obedece a su manera.

Detengámonos a pensar · Compáralo con lo que había

En el juego de una sola pieza, esas mismas seis cosas ocupaban seis tramos separados del bucle, cada uno con su recorrido y sus índices, y estaban rotulados con comentarios porque si no me perdía.

El capítulo 2 midió lo que costaba añadir algo allí: entre cuatro y seis sitios distintos por cambio.

Aquí, añadir un séptimo tipo de cosa que se mueva —una mina, un meteorito— **no toca el bucle**. Se escribe la clase, se hereda de `Entidad`, se implementa `avanzar()` y ya está

dentro.
Esa es la ganancia, y no es de tamaño: es de naturaleza. Antes había que *modificar* para extender; ahora se extiende sin modificar.

10.3 El bucle que no sabe con quién habla

Hay un segundo sitio donde esto se ve todavía mejor, y es el que más me sorprendió al escribirlo: la vista.

Código 10.2: Todo el dibujado del juego. No sabe qué está dibujando.

```
        for e in partida.entidades():
            caja = e.caja()
            simbolo = e.simbolo()
            for dx in range(caja.ancho):
                x, y = caja.x + dx, caja.y
                if 0 <= x < ancho and 0 <= y < alto:
                    lienzo[y][x] = simbolo
```

Siete líneas dibujan el juego entero: los 55 invasores, los cuatro búnkeres, las balas, las bombas, el OVNI y el cañón. La vista pide dos cosas —dónde estás y con qué letra te pintas— y no pregunta nada más.

Detengámonos a pensar · La prueba de que la separación es real

Cuando añadí el invasor acorazado, con su símbolo propio y su resistencia doble, **no toqué la vista**.
Y cuando escribí `VistaMuda` —la que no dibuja nada, para poder medir 500 partidas sin llenar la pantalla— tampoco toqué la partida.
Las dos cosas son la misma propiedad mirada desde sus dos extremos, y no las conseguí con esfuerzo: son la consecuencia de que ninguna de las dos partes sepa de qué clase es la otra.

10.4 Condicionales que desaparecen

Aquí está la señal que aprendí a buscar en el libro anterior, y que aquí reaparece con otra cara.

La alternativa al polimorfismo no es escribir más clases: es escribir condicionales. Y los condicionales tienen una propiedad que las clases no tienen: **crecen en todos los sitios a la vez**.

Las dos formas de tratar con seis clases

Preguntando qué eres

```
if tipo == "bala":  subir()
elif tipo == "bomba": bajar()
elif tipo == "ovni": cruzar()
```

Añadir un tipo obliga a buscar todos los sitios donde se hace esa pregunta. En nuestro juego serían tres: avanzar, dibujar y resolver impactos.
Y si te olvidas de uno, el juego funciona. Simplemente esa cosa no se dibuja.

Pidiendo que lo hagas

```
e.avanzar(mundo)
```

Añadir un tipo obliga a escribir una clase. Ni un sitio más.
Y si te olvidas de implementar `avanzar()`, Python se niega a crear el objeto, porque `Entidad` lo declara abstracto. El fallo aparece al instante y permite corregirlo cuando es barato. Eso es asegurar la calidad desde el momento de escritura del código.

Detengámonos a pensar · El método abstracto es lo que convierte el olvido en error

Esto es lo que hace que el lado derecho sea mejor y no solo más bonito.
`Entidad.avanzar` está marcado con `@abstractmethod`. Eso significa que una clase que herede de `Entidad` y no lo implemente **no se puede instanciar**: Python levanta un error en el momento de crearla.
Es la misma idea del capítulo 6 —acercar el síntoma a la causa— aplicada a un olvido de diseño en lugar de a un dato malo.

Inténtalo primero · Encuentra el condicional que queda

En el juego terminado hay un sitio donde *sí* se pregunta de qué tipo es algo, y no lo he quitado.
Está en la resolución de impactos: la partida distingue si el proyectil es del jugador o de la invasión, porque contra el cañón solo pueden ir bombas y contra los invasores solo balas.
¿Se podría quitar con polimorfismo? Piénsalo antes de leer la respuesta, y sobre todo piensa *qué costaría* quitarlo.

Sí se podría: bastaría con que cada proyectil supiera contra qué puede chocar. Y no lo hice, por una razón que conviene analizar.

Detengámonos a pensar · Cuándo dejo un condicional a propósito

Para quitarlo, cada proyectil tendría que conocer las clases con las que puede chocar. Es decir: la bala tendría que saber que existen los invasores y el OVNI, y la bomba tendría que saber que existe el cañón.
Cambiaría un condicional en un sitio por conocimiento repartido en dos clases que hoy no saben nada del juego. El capítulo 2 ya dejó dicho cuál de las dos cosas se paga más

cara.

La regla, entonces, no es «ningún condicional». Es esta: un condicional sobre el tipo se quita cuando *crece* al añadir clases. Este no crece: son dos bandos y siempre serán dos. Un condicional que no crece es una decisión, no una deuda.

Si esto fuera un equipo · Encargar el qué y no el cómo

«Elaborar cada uno su informe de gestión» es un encargo polimórfico: el mismo para todos, y cada equipo sabe qué significa en su caso.

La alternativa es un manual con una sección por departamento. Funciona, y hay que abrirlo y ampliarlo cada vez que se crea uno nuevo, y el día que se olvida una sección ese departamento sencillamente no entrega, sin que nadie lo note hasta el cierre.

La diferencia, otra vez, no es de esfuerzo: es de dónde hay que acordarse.

Sin programar · Cómo reconocerlo sin leer código

Busca en cualquier programa las preguntas del tipo «¿de qué clase eres?». En Python se escriben con `isinstance`, con `type(x) ==` o con un atributo que guarda el nombre del tipo.

Cada una de esas preguntas es un sitio que habrá que tocar cuando aparezca un tipo nuevo.

Cuéntalas. Ese número es lo que te va a costar la próxima extensión, y se puede contar sin entender el programa.

10.5 Lo que un sustituto tiene que poder hacer

El bucle escribe `e.avanzar(mundo)` y no pregunta qué tiene delante. Eso solo funciona mientras *cualquiera* de los que pueden estar ahí sirva igual de bien. La bala sube, la bomba baja, el búnker no se mueve: hacen cosas distintas, y ninguna sorprende al bucle. Ninguna pide algo que las demás no pidan, ninguna deja a medias lo que el bucle esperaba, ninguna se niega.

Detengámonos a pensar · La pregunta que hay que hacerle a cada elemento nuevo

Antes de meter una clase en el sitio por donde pasa el bucle, contesta esta pregunta:

> **¿Puedo poner esto donde el bucle espera una entidad, y que quien escribió el bucle no tenga que enterarse?**

Si la respuesta lleva un «sí, pero...», el polimorfismo deja de trabajar a tu favor y empieza a trabajar en tu contra: el bucle sigue sin preguntar, y ahora se equivoca sin decirlo.

Y fíjate en que esta pregunta no la contesta el lenguaje. Python comprueba que el método exista; que el objeto *se comporte* como el bucle espera no lo comprueba nadie. Por eso una sola orden para sesenta objetos es una ganancia grande y una responsabilidad exactamente

del mismo tamaño.

Con esto, el juego ya se extiende sin abrirse: clases nuevas entran sin que el bucle ni la vista se enteren.

Capítulo 11

Composición

Dejé el capítulo 9 con una cuenta incómoda: treinta y dos clases para cuatro tipos de invasor y tres habilidades. Doscientas cincuenta y seis si algún día son seis habilidades.

La salida no es escribir esas clases mejor. Es dejar de hacer la pregunta que las produce.

11.1 Tener algo no define un nuevo ser

La pregunta que produce la explosión es *¿qué es un invasor con escudo?* Y tiene una sola respuesta posible: es una clase de invasor, luego una subclase, luego una por combinación.

La pregunta buena es otra:

Detengámonos a pensar · Cambia el verbo y desaparece el problema

*¿Qué **tiene** un invasor con escudo?*

Tiene un escudo. Es un invasor —uno cualquiera, de cualquier tipo— que lleva algo puesto.

Y si lleva algo puesto, ese algo puede ser una pieza aparte, se le puede dar a cualquiera, se le puede cambiar, y no hace falta una clase por cada combinación: hacen falta *las piezas*.

Definición 11.1 (Composición)**.** La *composición* es construir un objeto a partir de otros que él tiene y usa. Las capacidades no se declaran en la jerarquía: se montan al crear el objeto y pueden cambiar mientras existe.

Ser o tener

En el juego

Un invasor con escudo no es un tipo nuevo de invasor. Es un invasor con un escudo dentro.

Por eso la misma pieza «escudo» sirve para un pulpo, para un acorazado, para el cañón y para un búnker. Cuatro portadores, una habilidad, cero subclases.

En la vida real

Un empleado con firma autorizada no es un tipo nuevo de empleado. Es un empleado con una autorización.

Y por eso la autorización se puede dar, quitar y trasladar sin cambiar el puesto de nadie. Nadie crea el puesto de «administrativo con firma»: se firma una delegación.

11.2 El cañón, su arma y sus vidas

El caso más simple del juego, y el que enseña la mecánica.

Un cañón podría tener un atributo `balas_maximas` y otro `velocidad_bala`. Los saqué a una pieza aparte:

Código 11.1: El arma es una pieza, no dos atributos sueltos del cañón.

```
class Arma:
    """Cuantas balas caben en el aire y a que velocidad suben."""

    __slots__ = ("balas_maximas", "velocidad")

    def __init__(self, balas_maximas=1, velocidad=3):
        self.balas_maximas = balas_maximas
        self.velocidad = velocidad


class Canon(Entidad):
    def __init__(self, caja, vidas=3, arma=None, habilidad=SIN_HABILIDAD):
        if vidas <= 0:
            raise ValueError("un canon nace con al menos una vida")
        super().__init__(caja)
        self._vidas = vidas
        self._arma = arma or Arma()
        self._habilidad = habilidad
```

Inténtalo primero · ¿Merecía la pena?

Son dos números metidos en una clase de seis líneas. Parece ceremonia.

Antes de que te dé mis motivos: di uno a favor y uno en contra.

El motivo en contra es cierto: para un juego que nunca cambiara, `Arma` sobra. Dos atributos habrían bastado.

Los motivos a favor aparecieron después, cuando pensé en otorgarle habilidades extras al cañón:

Cuando quise comparar dos partidas con armas distintas, pude crear las dos armas y pasárselas a dos partidas. Sin duplicar esfuerzos.

11.3 La oleada como objeto, no como lista

Este es el ejemplo que mejor justifica el capítulo, y ya lo anuncié en el primero.

Código 11.2: Lo que la formación sabe y sus miembros no pueden saber.

```
class Formacion:
    @property
    def cadencia(self):
        """Turnos entre paso y paso. Cuantos menos quedan, mas deprisa."""
        return 1 + self.vivos // self._divisor_cadencia
```

```
6
7       def _toca_el_borde(self, mundo):
8           for i in self.vivos_iter():
9               siguiente = i.caja().desplazada(dx=self._sentido)
10              if not siguiente.dentro_de(mundo.limites):
11                  return True
12          return False
```

Detengámonos a pensar · Por qué esto no puede ser una lista

Una lista de invasores es una lista. Sabe cuántos elementos tiene y nada más.

La formación sabe tres cosas que ningún invasor puede saber, y que una lista tampoco: **Que si alguno toca el borde, bajan todos.** Ningún invasor conoce a los demás. Es la respuesta a la tercera pregunta del capítulo 1, y la razón por la que la formación existe. **A qué ritmo avanzar.** Depende de cuántos quedan vivos, que es una propiedad del conjunto.

Quién puede disparar. Solo el más bajo de cada columna. Un invasor no sabe si es el más bajo de la suya.

Sin la formación, resolverlos sería responsabilidad del bucle principal y eso nos resta control.

11.4 Una habilidad se tiene, no se es

Y ahora la salida a la explosión combinatoria.

Código 11.3: Tres habilidades. Sirven a cualquier portador: agresor, defensor u obstáculo.

```
1   class Habilidad(ABC):
2       """Se otorga desde fuera. Nadie se la fabrica a si mismo."""
3
4       @abstractmethod
5       def absorbe_impacto(self, portador) -> bool:
6           """¿Se come este impacto? True = el portador no lo sufre."""
7
8       def al_avanzar(self, portador, mundo) -> None:
9           """Oportunidad de hacer algo cada turno. Por omision, nada."""
10
11
12  class Escudo(Habilidad):
13      __slots__ = ("cargas",)
14
15      def __init__(self, cargas=1):
16          self.cargas = cargas
17
18      def absorbe_impacto(self, portador):
19          if self.cargas > 0:
20              self.cargas -= 1
```

```
            return True
        return False
```

Y así es como el portador la usa, sin saber cuál lleva:

Código 11.4: El invasor no sabe qué habilidad tiene. Solo le da la oportunidad de intervenir.

```
    def recibir_impacto(self):
        if self._habilidad.absorbe_impacto(self):
            return False
        self._resistencia -= 1
        if self._resistencia <= 0:
            self.morir()
            return True
        return False
```

Detengámonos a pensar · Lo que la composición nos dice

Con composición, una habilidad puede **cambiar mientras el objeto existe**. Un búnker puede recibir regeneración a mitad de partida; un escudo se gasta y deja al invasor sin él. Con herencia eso es imposible: la clase de un objeto no cambia. Un `InvasorConEscudo` al que se le gasta el escudo sigue siendo un `InvasorConEscudo`, y hay que apañárselo con una bandera dentro —que es composición mal escrita—.

Esta es la diferencia de fondo: **la herencia se decide al escribir el programa; la composición, al ejecutarlo**. Todo lo que dependa del juego, y no del programador, tiene que ir por el segundo camino.

Si esto fuera un equipo · Ascender a alguien o darle una capacidad

Cuando alguien necesita poder aprobar gastos, hay dos maneras.

Crear el puesto de «coordinador con firma» y ascenderlo. A partir de ahí, esa persona es otra cosa; si mañana deja de necesitar la firma, es un proceso complicado de revertir.

O delegar la firma. Se da, se quita, y la persona sigue siendo la misma.

La primera es herencia; la segunda, composición. Y el motivo por el que la segunda funciona mejor es el mismo que en el código: las necesidades cambian más deprisa que las categorías.

Prompt · Antes de crear una subclase

```
Estoy a punto de crear la clase [nombre] heredando de [clase base] para añadir
[comportamiento].

No escribas código. Antes contéstame:
```

```
1. ¿Cuántas subclases harían falta si en el futuro hubiera 2, 3 y 4
   comportamientos como este, combinables entre sí? Dame los números.

2. ¿Este comportamiento puede aparecer o desaparecer mientras el objeto existe?

3. ¿Podría este mismo comportamiento hacer falta en otra clase que no herede de
   la misma base?

Si la respuesta a 2 o a 3 es que sí, propónme cómo sería con composición: qué
pieza haría falta y qué contrato tendría. Solo el diseño, sin implementarlo.
```

Con la composición, las piezas se montan en lugar de heredarse, y el juego puede tener treinta y dos combinaciones sin tener treinta y dos clases.

Queda una pregunta sin responder, y es de la que depende que todo esto se sostenga: ¿qué tiene que prometer una clase para que quien la usa pueda confiar en ella sin conocerla? El escudo y la regeneración no se parecen en nada y el invasor las trata igual. Alguien tuvo que escribir en algún sitio qué significa «ser una clase».

Capítulo 12

Interfaces y clases abstractas

Terminé el capítulo anterior con una pregunta: ¿qué tiene que prometer una clase para que quien la usa pueda confiar en ella sin conocerla?

La respuesta es un contrato. Y en la orientación a objetos hay dos maneras de escribirlo, que se parecen lo bastante como para confundirse y sirven para cosas distintas.

12.1 Un contrato es un encargo bien dado

Definición 12.1 (Interfaz)**.** Una *interfaz* es la lista de lo que hay que saber hacer para ocupar un sitio. Dice **qué** se promete y no dice nada de **cómo**. No aporta código.

Definición 12.2 (Clase abstracta)**.** Una *clase abstracta* es un contrato que además **trae trabajo hecho**: implementa la parte común y deja abierta la que cada caso resuelve a su manera. No se puede instanciar por sí sola.

Las dos formas de un contrato

En el juego

`Colisionable` pide una sola cosa: *dame tu caja*. No trae código, no impone constructor y no obliga a heredar de nada. Cualquier cosa que sepa decir qué sitio ocupa lo cumple.

`Entidad` es completamente opuesto: ya trae escrito qué significa vivir, morir y tener caja, y solo deja abierto `avanzar()`.

En la vida real

Tu licencia de conducir dice qué puedes hacer, no cómo. La licencia de conducir no indica el cómo conduces: certifica que sabes.

Un manual de procedimiento es completamente opuesto: trae los ocho pasos comunes escritos y deja un hueco para el paso que depende de tu delegación.

El primero *comprueba*. El segundo *ayuda*.

12.2 Colisionable: dame tu caja y no me cuentes más

El contrato más pequeño del juego, y a propósito:

Código 12.1: Un método. Ni uno más.

```
@runtime_checkable
class Colisionable(Protocol):
    """Contrato de una sola cosa: saber decir que sitio ocupas."""

    def caja(self) -> Caja: ...
```

Inténtalo primero · ¿Por qué no está esto en `Entidad`?

`Entidad` ya tiene `caja()`. Todo lo que colisiona en el juego es una entidad. Entonces `Colisionable` parece que sobra.

Antes de que responda: piensa qué cosa podría querer colisionar *sin* ser una entidad. Y piensa también en la dirección contraria.

Hacia un lado: puede haber cosas que ocupen un sitio y no sean entidades del juego. Una zona prohibida, el borde del tablero, la sombra de un búnker. Nada de eso vive ni avanza, y obligarlas a heredar de `Entidad` sería pedirles que implementen `avanzar()` sin tener nada que avanzar.

Y hacia el otro: hay entidades que **no** colisionan con nada. El marcador, si algún día se dibuja como entidad. Un efecto visual. Si «colisionar» estuviera dentro de `Entidad`, todas prometerían algo que la mitad no cumple.

Detengámonos a pensar · Separar los contratos es separar las promesas

Un contrato grande obliga a todos a prometer lo que solo algunos cumplen. `Entidad` promete existir en el tiempo. `Colisionable` promete ocupar un sitio. Casi todas las cosas del juego cumplen las dos, y son dos promesas distintas, así que van escritas por separado.

Que hoy coincidan casi siempre no es un argumento para juntarlas: es la razón por la que juntarlas parece inofensivo.

12.3 Cuándo un contrato y cuándo una base con trabajo hecho

Aquí está el criterio que buscaba al escribir este capítulo, y me costó formularlo porque en Python las dos cosas se escriben casi igual.

Cuadro 12.1: Cómo elijo entre las dos. La columna de la derecha es la que decide casi siempre.

	Interfaz (`Protocol`)	**Clase abstracta**
Qué aporta	Solo la promesa	Promesa más código común
Obliga a heredar	No	Sí
Cuántas puede cumplir un objeto	Todas las que quiera	Una jerarquía
Cuándo la uso	Cuando lo común es **lo que se promete**	Cuando lo común es **lo que se hace**
Ejemplo	`Colisionable`	`Entidad`, `Habilidad`, `EstrategiaDisparo`

La regla en una frase: si al escribir el contrato te sale código común que todos van a repetir, es una clase abstracta. Si no te sale nada, es una interfaz, y forzarla a clase base solo

añade una jerarquía que no aporta.

Detengámonos a pensar · Y por qué `Habilidad` es abstracta y no interfaz

`Habilidad` tiene dos métodos: `absorbe_impacto()`, que cada una resuelve a su manera, y `al_avanzar()`, que **casi ninguna necesita**.
Si fuera una interfaz, cada habilidad tendría que escribir un `al_avanzar()` vacío.
Como clase abstracta, `al_avanzar()` viene implementado sin hacer nada, y solo `Regeneracion` lo redefine. Eso es «traer trabajo hecho», y es exactamente el motivo por el que existen las clases abstractas.

Sin programar · Cuál de los dos documentos necesitas

Antes de escribir cualquier norma en una organización, la misma pregunta:
¿voy a escribir lo que hay que conseguir, o voy a escribir además los pasos comunes?
Si solo escribes lo que hay que conseguir, cada equipo lo resolverá a su manera y podrás cambiar de proveedor sin reescribir nada. Es una interfaz.
Si escribes además los pasos comunes, ahorras a todos repetirlos, y a cambio todos quedan atados a tu manera de hacerlos. Es una clase abstracta.
Las dos son legítimas. Lo que no funciona es escribir la segunda creyendo que escribes la primera.

12.4 Contratos que piden de más y contratos que mienten

El que pide de más

Un contrato con siete métodos obliga a quien lo cumpla a implementar siete, aunque solo use dos. Lo que ocurre entonces es previsible: los otros cinco se escriben vacíos, o peor, lanzando un error.

Código 12.2: Lo que aparece cuando un contrato pide de más. Cada línea de estas es una promesa rota.

```
class MarcadorComoEntidad(Entidad):
    def avanzar(self, mundo):
        pass                        # no avanza

    def caja(self):
        raise NotImplementedError  # no ocupa sitio
```

Ese `raise` es la señal. Cuando una clase tiene que negarse a cumplir parte de su contrato, el contrato estaba mal definido. Y lo grave no es la línea: es que ahora nadie puede fiarse de que una `Entidad` tenga caja.

El que miente

El contrario, y más difícil de ver: la clase implementa todos los métodos, no lanza ningún error, y aun así incumple lo que se esperaba.

Inténtalo primero · Encuentra la mentira

Imagina una `HabilidadInmortal` cuyo `absorbe_impacto()` devuelve siempre `True`.
Cumple el contrato: tiene el método, devuelve un booleano, no falla nunca.
¿Qué se rompe en el juego? Y la pregunta que importa: ¿qué parte del contrato estaba escrita en ningún sitio?

Lo que se rompe es que la oleada no termina jamás, y con ella la partida. Y lo que no estaba escrito en ninguna parte es que un escudo *se agota*.

Detengámonos a pensar · Un contrato es más que su firma

La firma —nombre, parámetros, tipo de vuelta— es la parte que el lenguaje puede comprobar. Y no es el contrato entero. El contrato incluye también lo que se espera del comportamiento: que termine, que no cambie nada al consultar, que dos llamadas seguidas hagan lo que promete.
Nada de eso se declara en Python. Ni en Java. Se escribe en la documentación, se comprueba con pruebas, y se rompe sin que nadie avise.

Si esto fuera un equipo · El encargo que no dice cuándo termina

Un encargo bien dado dice qué hay que conseguir y deja el cómo a quien lo recibe. Eso es una interfaz, y funciona.
Un encargo mal dado tiene la misma forma y le falta lo mismo que a `Habilidad`: no dice cuándo se considera cumplido. Quien lo recibe puede hacer exactamente lo que se le pidió, durante meses, y no haber terminado nunca.
Y como en el código, el problema no se ve en el encargo: se ve en el resultado, tarde.

Compruébalo · Busca los métodos vacíos

En un programa tuyo, busca implementaciones que no hagan nada o que lancen `NotImplementedError`.
Cada una es un sitio donde alguien tuvo que prometer algo que no podía cumplir.
Y para cada una, la pregunta: *¿qué contrato habría que partir en dos para que esa clase no tuviera que mentir?*

Con esto se cierra la Parte IV y, con ella, los diez conceptos del temario. El juego ya está repartido: cada objeto sabe lo suyo, nadie interviene donde no debe, una orden vale para todos y las capacidades se asignan en lugar de heredarse.

Y funciona. Puedes ejecutarlo.

Lo que no tenemos es un criterio para saber si está *bien*. He ido diciendo «esto es mejor» capítulo tras capítulo, apoyándome en cifras cuando las había y en argumentos cuando no.

Antes de aplicar un solo patrón hace falta poder decir qué se persigue exactamente, porque un patrón sin principio detrás es decoración. De eso trata la parte siguiente.

V

Los cinco principios

Capítulo 13

Responsabilidad única

El juego ya está repartido y funciona. Es momento de asegurar la calidad del código y poder decir que está *bien*.

Son cinco principios, se conocen por el acrónimo SOLID, y los reunió Robert C. Martin en el año 2000 —dato que tomo de Cabral et al. (2024)—.

13.1 El enunciado: solo puede haber una única razón para cambiar

Definición 13.1 (Responsabilidad única)**.** Una clase debe tener **una sola razón para cambiar**.

La formulación popular es otra —«una clase debe hacer una sola cosa»— y es la que estropea el principio, porque «una sola cosa» no se puede medir. ¿Un invasor hace una cosa o cuatro? Guarda su sitio, se desplaza, encaja impactos y dice dónde dispara. Con ese criterio habría que partirlo en cuatro, y sería absurdo.

Detengámonos a pensar · El criterio no es qué hace: es quién pide el cambio

Una razón para cambiar no es una función. Es **alguien**.

Prueba a preguntarlo así: *¿qué tendría que pasar en el mundo para que hubiera que tocar esta clase?* Y después: *¿esas cosas pasan por separado?*

Si un invasor tiene que cambiar porque cambian las reglas del juego, y solo por eso, tiene una razón. Si además tuviera que cambiar cuando cambia el aspecto en pantalla, o cuando cambia dónde se guardan los marcadores, tendría tres, y esas tres cosas las piden tres personas distintas en tres momentos distintos.

13.2 La razón para cambiar: quién pide la modificación

Aplicado a nuestras clases:

Cuadro 13.1: Una razón por clase. La última fila es la que no cumple, y sigue sin cumplir en este punto del libro.

Clase	Su razón para cambiar	Quién la pediría
`Invasor`	Cambian las reglas de qué mata a un invasor	Quien diseña el juego
`Formacion`	Cambia cómo se mueve o se acelera el bloque	Quien ajusta la dificultad
`VistaConsola`	Cambia cómo se ve	Quien diseña la interfaz
`EnArchivo`	Cambia dónde se guardan los marcadores	Quien decide la infraestructura
`Caja`	Cambia la geometría del tablero	Casi nadie: no ha cambiado nunca
`Partida`	**Cinco razones distintas**	Cinco personas distintas

13.3 La clase que movía, dibujaba, puntuaba y sonaba

Y aquí tengo que hablar de `Partida`, que lleva incumpliendo este principio desde el capítulo 3, donde ya lo señalé y lo dejé pasar.

Código 13.1: Lo que hace la partida, listado sin piedad.

```
class Partida:
    def avanzar(self):        ...  # 1. hace correr el tiempo
    def resolver_impactos(self, proyectil): ...  # 2. arbitra colisiones
    def _quiza_disparar(self): ...  # 3. decide cuando dispara la invasion
    def _quiza_ovni(self):     ...  # 4. decide cuando aparece el ovni
    def _siguiente_oleada(self): ... # 5. lleva la puntuacion y las oleadas
```

Cinco razones para cambiar, y cinco personas distintas que las pedirían.

Inténtalo primero · Antes de hacer ajustes

`Partida` tiene 251 líneas —la clase más larga del juego, medido— y cinco responsabilidades.

Propón cómo la dividirías. Y después responde a la pregunta que de verdad importa: *¿en cuántas clases, y qué se lleva cada una?*

Aviso: hay al menos una que **no** conviene sacar, y averiguar cuál es el ejercicio.

La que no conviene sacar es la primera. Hacer correr el tiempo —llamar a cada uno para que avance, en el orden correcto— es precisamente lo que hace que esta clase exista. Si le quitas eso, no queda una partida: queda una bolsa de objetos.

Las otras cuatro sí salen, y salen en la parte siguiente, cada una empujada por un dolor concreto:

Cuadro 13.2: Cómo se reparte `Partida` a lo largo del libro.

Responsabilidad	Se va a...	Empujada por
Decidir cuándo dispara la invasión	`EstrategiaDisparo`	Tres formas de disparar y un condicional que crece (cap. 21)
Componer las oleadas	`FabricaDeOleadas`	Montar la oleada 7 a mano (cap. 20)
Avisar de lo que ocurre	`Sucesos`	El invasor tiene que conocer al marcador (cap. 22)
Guardar los marcadores	`RepositorioMarcadores`	El juego atado a un archivo (cap. 24)

Detengámonos a pensar · Por qué la dejé mal hasta aquí, a propósito

Busco enseñarte el proceso mental durante la construcción del juego, y separarla es el proceso que buscaré ilustrar poco a poco conforme avanzamos.

Un patrón que llega antes de que duela se aprende como vocabulario. Los cuatro de la tabla llegan cada uno con su dolor delante, y cuando llegan ya sabes por qué.

Dicho lo cual, hay que reconocer el precio: durante doce capítulos el juego ha tenido una clase con cinco razones para cambiar, y eso es exactamente lo que este capítulo dice que no hay que hacer.

13.4 Partir por motivo de cambio, no por tamaño

El error más común al aplicar este principio es partir por líneas.

Una clase de 300 líneas con una sola razón para cambiar está bien. Una de 30 con tres razones está mal. El tamaño correlaciona con el problema y no es el problema.

Dos maneras de trocear una clase grande

Partir por tamaño

«Esto es muy largo, lo parto en tres.»

Salen `PartidaA`, `PartidaB` y `PartidaC`, que se llaman entre sí todo el rato y comparten estado.

Ahora hay tres archivos y sigue habiendo cinco razones para cambiar, repartidas de forma que hay que abrir los tres para encontrar cualquier cosa.

Partir por motivo

«¿Quién pediría un cambio aquí?»

Salen una estrategia de disparo, una fábrica de oleadas y un repositorio de marcadores: piezas con nombre, que se pueden sustituir enteras.

Y hay una prueba objetiva de que la partición es buena: cada una se puede cambiar sin abrir las otras.

Sin programar · La prueba del organigrama

Piensa en cualquier procedimiento de tu trabajo y pregunta, por cada paso, quién tendría que pedir que ese paso cambiara.

Si todos los pasos los pediría la misma persona, es un procedimiento. Si hay tres personas distintas, son tres procedimientos escritos en la misma hoja, y lo notarás el día que una de las tres pida un cambio y haya que consultar a las otras dos.

13.5 Partir de más

Y el fallo del otro lado, que este principio provoca más que ningún otro.

Código 13.2: Una clase que no hace nada.

```
class GestorDeVidas:
    def __init__(self, canon):
        self._canon = canon

    def quitar_una(self):
        return self._canon.recibir_impacto()

    def cuantas_quedan(self):
        return self._canon.vidas
```

Parece que separa una responsabilidad. No separa nada: cada método reenvía al cañón. Lo único que ha añadido es un archivo más que abrir y un nombre más que recordar.

Detengámonos a pensar · La señal de que has partido de más

Tres, y basta con una:

La clase nueva no tiene estado propio. Solo guarda una referencia a otra.

Sus métodos son de una línea y todas empiezan por `return self._otro.`

Y la definitiva: no se puede sustituir por otra cosa. Una pieza que merece existir es una pieza que podría tener una segunda implementación. Si no imaginas la segunda, probablemente no hacía falta la primera.

Si esto fuera un equipo · Repartir el trabajo y repartir la responsabilidad

Este principio se lee casi igual en un equipo, y con la misma trampa de los dos lados.

De menos: la persona que lleva la facturación, el teléfono, los proveedores y el cierre de mes. Cuatro razones para interrumpirla, y cuando falta no falta una persona, faltan cuatro.

De más: crear un puesto que consiste en trasladar peticiones de un sitio a otro. No reparte trabajo: añade un paso.

Prompt · Buscar las razones para cambiar

```
Te paso una clase: [pégala].

No la refactorices. Haz esto:

1. Enumera cada cosa que hace, sin agrupar.

2. Para cada una, dime QUÉ TENDRÍA QUE PASAR EN EL MUNDO para que hubiera que
   modificarla, y QUIÉN lo pediría (un rol, no una persona).

3. Agrupa las que compartan la misma razón. El número de grupos es el número de
   razones para cambiar.

4. Si salen varios grupos, dime para cada uno si la pieza resultante podría
   llegar a tener DOS implementaciones distintas. Si no, avísame de que
   separarla añadiría un archivo sin separar nada.

No escribas código.
```

Compruébalo · Cuenta las razones, no las líneas

Sobre tu clase más larga, escribe la lista de razones para cambiar y quién las pediría. Después, sobre tu clase más corta, la misma lista.
Es habitual que la corta tenga más razones que la larga. Cuando pasa, tienes delante la diferencia entre partir por tamaño y partir por motivo.

Con el primer principio queda claro por qué `Partida` tiene que dividirse, y quedan señaladas las cuatro piezas que saldrán de ella.

Lo que este principio no dice es qué pasa cuando llega algo *nuevo*. Saber que una clase tiene una sola razón para cambiar no te dice si podrás añadir un cuarto tipo de invasor sin abrirla. Y eso, en un juego que va por oleadas, es la pregunta siguiente.

Capítulo 14

Abierto/cerrado

El principio anterior mira hacia dentro: cuántas razones tiene una clase para cambiar. Este mira hacia fuera y hace la pregunta que de verdad se hace a diario en cualquier proyecto: *¿qué hay que abrir para añadir algo nuevo?*

14.1 El enunciado

Definición 14.1 (Abierto/cerrado)**.** Una pieza debe estar **abierta a la extensión y cerrada a la modificación**: se le puede añadir comportamiento nuevo sin tocar lo que ya funciona.

Suena a contradicción y no lo es. Lo que se abre y lo que se cierra son cosas distintas: se cierra *el código escrito* y se abre *el conjunto de comportamientos posibles*.

Detengámonos a pensar · La prueba, y es objetiva

Este principio tiene una virtud que los otros cuatro no tienen: se comprueba con un diff. **Añade lo nuevo y mira qué archivos aparecen como modificados.** Si solo aparecen archivos nuevos, la pieza estaba cerrada. Si aparece modificado algo que ya funcionaba, no lo estaba.

No hay que opinar. Es la misma medida del capítulo 2 —sitios que hay que tocar— aplicada a una extensión en lugar de a un cambio.

14.2 El cuarto tipo de invasor

El caso concreto, y es el primero de los tres cambios que el capítulo 2 midió sobre el juego de una sola pieza.

Allí, añadir un invasor acorazado costó **16 líneas nuevas, 3 modificadas y seis sitios distintos** que había que encontrar uno por uno: donde se crean, donde impacta la bala, donde se reinicia la oleada, donde se dibuja.

En el juego repartido, lo que yo tenía escrito era esto:

Código 14.1: Mi primera versión del enumerado de tipos. Parece que añadir uno cuesta una línea. No las cuesta.

```
class TipoInvasor(IntEnum):
    PULPO = 0
    CANGREJO = 1
    CALAMAR = 2
    ACORAZADO = 3          # <- lo unico que hay que anadir

```

```
    @property
    def puntos(self):
        return (10, 20, 30, 50)[self]

    @property
    def resistencia(self):
        return 2 if self is TipoInvasor.ACORAZADO else 1
```

Detengámonos a pensar · Lo que la medición encontró

La primera versión de `TipoInvasor` repartía los datos de cada tipo en tres expresiones distintas: el valor en el enumerado, los puntos en una tupla, la resistencia en una condición y el símbolo en una cadena.

Añadir el acorazado tocaba **cuatro sitios en ese archivo**, más cuatro en la fábrica. Ocho en total, frente a los seis del juego de una sola pieza.

Es decir: en este cambio concreto, mi versión repartida era *peor* que el monolito. Y no lo habría sabido nunca leyendo el código, porque cada uno de esos cuatro sitios cabe en una línea y parece inofensivo.

Lo arreglé poniendo todo lo de un tipo en su propia línea:

```
class TipoInvasor(IntEnum):
    PULPO     = (0, 10, 1, "W")
    CANGREJO  = (1, 20, 1, "M")
    CALAMAR   = (2, 30, 1, "X")
    ACORAZADO = (3, 50, 2, "#")

    def __new__(cls, valor, puntos, resistencia, simbolo):
        obj = int.__new__(cls, valor)
        obj._value_ = valor
        obj.puntos = puntos
        obj.resistencia = resistencia
        obj.simbolo = simbolo
        return obj
```

Después de eso, el enumerado pasó de cuatro sitios a **uno**.

Con la corrección hecha, esta es la cifra de verdad:

Cuadro 14.1: El mismo cambio en las dos versiones, medido sobre las dos con el mismo criterio. «Sin la raíz» excluye la raíz de composición, que es donde se exponen las opciones al usuario y no mide diseño.

Añadir un cuarto tipo	Una pieza	Repartido
Sitios distintos que tocar	6	5
Sitios sin contar la raíz de composición	6	5
Archivos que hay que abrir	1	2
Líneas añadidas	16	6

Detengámonos a pensar · Cinco frente a seis no es una victoria, y conviene decirlo

La diferencia es de un sitio. Si el argumento de este libro fuera «repartir abarata los cambios», esta tabla lo dejaría en muy mal lugar.

No es ese el argumento, y aquí se ve por qué. Los cinco sitios del juego repartido están en dos piezas con nombre —el enumerado de tipos y la fábrica de oleadas— y los encuentras porque se llaman como lo que hacen. Los seis del monolito están dentro de una función de 265 líneas, y hay que *buscarlos leyendo*.

Y hay una diferencia que la tabla no recoge y es la que importa de verdad: cuatro de los cinco están en la fábrica, y son la decisión de *cuándo aparece* el acorazado —a partir de qué oleada—. Eso no es coste de añadir un tipo: es la mecánica del juego, que en el monolito estaba enterrada y aquí está en un sitio donde se puede ajustar.

Que es justo de lo que va este libro: no de escribir menos, sino de poder afinar el juego sin tener que cambiarlo por completo.

Detengámonos a pensar · Y aun así el principio no se cumple del todo

`TipoInvasor` se modifica. El principio pide *cero* archivos abiertos, y aquí es uno.

Para llegar a cero habría que convertir cada tipo en una clase con su propio comportamiento, y volveríamos a las tres clases vacías que el capítulo 9 descartó por buenas razones.

Así que aquí incumplo a propósito, y digo por qué: un enumerado que crece una línea no es una deuda. Convertirlo en jerarquía para cumplir un principio sería pagar un coste real por un beneficio teórico.

Un principio es un criterio, no una obligación. La diferencia entre usarlo y obedecerlo es todo el capítulo 34.

14.3 Extensión frente a modificación, con una prueba objetiva

Donde el juego sí cumple el principio de verdad es en las tres extensiones que más se van a pedir. Y aquí no hay matices:

Cuadro 14.2: Tres extensiones y lo que hay que abrir. Contado sobre el juego terminado.

Extensión	Qué se escribe	Qué se abre
Una cuarta forma de disparar la invasión	Una clase que cumpla `EstrategiaDisparo`	Nada
Una habilidad nueva	Una clase que cumpla `Habilidad`	Nada
Una entidad nueva que se mueva	Una clase que herede de `Entidad`	Nada

Tres extensiones, cero archivos abiertos. Y no es una casualidad: las tres pasan por un contrato —una clase abstracta o una interfaz— y ese contrato es exactamente el punto por donde la pieza queda abierta.

Detengámonos a pensar · El principio, dicho de la manera que sí sirve

Cerrar todo es imposible y no es lo que se pide. Lo que se pide es esto:

Decide por dónde va a crecer, y deja ahí un contrato.

En este juego decidí tres sitios: cómo dispara la invasión, qué habilidades hay y qué cosas existen en el tablero. Los tres tienen contrato, y por los tres se crece sin abrir nada. Todo lo demás está cerrado a la extensión, y está bien que lo esté. El tablero mide 44 por 24 y no hay un contrato para cambiarlo, porque nadie ha pedido nunca un tablero distinto.

Abrir por donde se crece

En el juego

Para añadir una forma de disparar escribo una clase y se la paso a la partida al crearla. No abro la partida, no abro las otras tres estrategias, y las partidas que ya funcionaban siguen funcionando igual.

En la vida real

Para dar de alta un servicio nuevo, un reglamento bien hecho tiene un procedimiento de alta. Se rellena y ya está.

Uno mal hecho obliga a reescribir el reglamento, y entonces hay que revisar que el cambio no rompa los servicios que ya existían. Eso es abrir lo que funcionaba.

14.4 Lo que hay que dejar abierto y lo que conviene dejar cerrado

Esta es la parte que no suele contarse, y es la que decide si el principio ayuda o estorba.

Cada punto de extensión cuesta: un contrato más que entender, una indirección más que seguir al leer, y una pieza más que puede estar mal implementada. Abrirlo todo produce un programa donde no se puede seguir nada.

Inténtalo primero · Elige tú los puntos de extensión

Si tuvieras que dejar **solo dos** sitios abiertos en este juego, ¿cuáles serían?
Y la pregunta que importa más: ¿con qué criterio los eliges? Antes de leer el mío, escribe el tuyo.

El criterio que uso, y no es de diseño sino de historia:

Detengámonos a pensar · Se abre por donde ya ha cambiado

No por donde crees que va a cambiar. **Por donde ya cambió.**
La diferencia es enorme. Adivinar el futuro produce puntos de extensión que no usa nadie —y el capítulo 8 ya avisó de qué aspecto tiene una abstracción con una sola implementación—.
En este juego los tres puntos abiertos no los elegí de antemano: aparecieron porque el juego me pidió tres formas de disparar, tres habilidades y seis clases de entidad.
Regla de bolsillo: a la primera se escribe; a la segunda se sospecha; a la tercera se abre.

14.5 Adivinar el futuro sale caro

Y para no dejarlo en teoría, cuento uno mío.

Al empezar el juego previ que algún día haría falta un tablero de otro tamaño, y dejé el ancho y el alto como parámetros de la partida con sus valores por omisión. Parecía barato.

Lo que pasó fue esto:

1. Nadie pidió nunca otro tamaño.
2. Cada método que dibuja o comprueba límites tiene que preguntarle el tamaño a la partida en lugar de usar una constante.

Detengámonos a pensar · Lo dejé, y digo por qué

No lo he quitado, y no por pereza: el parámetro sí se usa en las pruebas, donde un tablero pequeño hace las comprobaciones más rápidas y más legibles.
O sea que el punto de extensión resultó útil *para otra cosa distinta de la que lo motivó.* Pasa a menudo, y no lo convierte en una buena decisión: lo convierte en una decisión que salió bien por coincidencia, no por conveniencia.
Cuento la diferencia porque es fácil confundirlas, y confundirlas es lo que lleva a abrirlo todo «por si acaso».

Si esto fuera un equipo · El procedimiento de alta

Una organización que sabe crecer tiene procedimientos de alta: cómo se incorpora alguien, cómo se da de alta un cliente, cómo se abre una delegación. Se usan y no se discuten.
Una que no los tiene convierte cada incorporación en una reunión, y en cada reunión se revisan cosas que ya estaban decididas. Eso es abrir lo que funcionaba.

Y el exceso también existe: procedimientos de alta para cosas que ocurren una vez cada cinco años, que nadie recuerda y que hay que leerse enteros para usarlos una vez.

Prompt · ¿Por dónde crece esto?

```
Te paso mi código: [pégalo].

No propongas abstracciones todavía. Primero:

1. Dime qué cosas habría que tocar para añadir cada una de estas extensiones:
   [enumera 3 o 4 extensiones realistas]. Para cada una: qué archivos que YA
   FUNCIONAN habría que abrir.

2. De esas extensiones, dime cuáles ya han ocurrido al menos una vez en el
   historial que te enseño y cuáles son hipotéticas mías.

3. Propón un punto de extensión SOLO para las que ya han ocurrido. Para las
   hipotéticas, dime expresamente que no conviene abrirlas todavía.

Si te parece que hay que abrir más sitios, dímelo con su motivo en lugar de
proponerlo.
```

Compruébalo · El diff de una extensión

Busca un programa tuyo con control de versiones y toma el último añadido de funcionalidad —no una corrección, un añadido—.
Mira el diff y separa: archivos nuevos frente a archivos modificados.
La proporción es tu medida de este principio, y la tienes sin escribir una línea. Si casi todo son modificaciones, ya sabes por dónde no estaba abierto.

Con esto, el juego crece por donde tiene que crecer y no se abre por donde no.

Queda un supuesto que he dado por bueno en los dos principios anteriores y que todavía no he justificado: que una pieza que cumple un contrato *se comporta* como el contrato promete. El capítulo 12 ya dejó asomando que una clase puede implementar todos los métodos y aun así mentir. El capítulo siguiente se ocupa de eso.

Capítulo 15

Sustitución de Liskov

Este es el principio que más me ha hecho pensar de los cinco, y el único que señala un fallo que el lenguaje no puede detectar.

Los dos anteriores se comprueban leyendo o con un diff. Este no: una clase puede cumplirlo o incumplirlo con exactamente el mismo código, según lo que prometa. Y por eso es el que de verdad decide cuándo se puede aplicar la herencia.

15.1 El enunciado, en la versión que se entiende

Definición 15.1 (Sustitución de Liskov)**.** Si una clase hereda de otra, debe poder **ocupar su sitio en cualquier parte** sin que quien la usa tenga que enterarse. No basta con tener los mismos métodos: hay que *comportarse* como el padre prometió.

La formulación original es más precisa. Le pertenece a Barbara Liskov y Jeannette Wing, que la formalizaron en 1994.

Detengámonos a pensar · Por qué el lenguaje no puede ayudarte aquí

Python comprueba que el método exista. Java comprueba además los tipos. Ninguno de los dos comprueba lo único que este principio pide.

Un método puede tener el nombre correcto, los parámetros correctos y el tipo de vuelta correcto, y aun así hacer algo que quien lo llama no esperaba. El compilador dice que sí. El juego dice que no.

Esa es la razón de que este principio necesite un capítulo y no una línea: **es el único que hay que sostener a mano**.

15.2 El invasor indestructible que rompe el bucle

Lo concreto, y lo escribí para verlo fallar.

Código 15.1: Cumple el contrato. Y rompe el juego.

```
class HabilidadInmortal(Habilidad):
    """Absorbe cualquier impacto. Nada mas."""

    nombre = "inmortal"

    def absorbe_impacto(self, portador) -> bool:
        return True
```

Tiene el método. Devuelve un booleano. No falla nunca. Y con un solo invasor que la lleve, la oleada no se despeja jamás, la partida llega al límite de turnos y el jugador no puede ganar.

Inténtalo primero · ¿Dónde estaba escrito que no se podía?

Antes de que lo diga: busca en el contrato de `Habilidad`, tal como está en el capítulo 11, la cláusula que `HabilidadInmortal` incumple.

No la vas a encontrar. La pregunta buena es: *¿cómo se escribe esa cláusula?*

15.3 Precondiciones, poscondiciones e invariantes: qué se le puede exigir

Aquí está el aparato, y es más sencillo de lo que suena. Un contrato tiene tres partes, y para sustituir hay que respetar las tres de una manera concreta:

Cuadro 15.1: Las tres reglas de la sustitución. La columna de la derecha es la que la gente invierte.

Parte	Qué es	Qué puede hacer el hijo
Precondición	Lo que hay que cumplir para llamarlo	Exigir **lo mismo o menos**. Nunca más
Poscondición	Lo que promete al terminar	Prometer **lo mismo o más**. Nunca menos
Invariante	Lo que es cierto siempre	Mantenerlo. Nunca relajarlo

Detengámonos a pensar · La regla en una frase, y la trampa

Pide menos, promete más. Nunca al revés.

La trampa es que suena simétrico y no lo es, y por eso se equivoca casi todo el mundo. Un hijo que *exige más* rompe a quien lo llame confiando en el padre. Un hijo que *promete menos* rompe a quien use el resultado.

Y aplicado a nuestro caso: el contrato de `Habilidad` tenía una poscondición que yo nunca escribí —*la sucesión de respuestas termina llegando a* `False`— y `HabilidadInmortal` promete menos que eso. Promete nada.

Escrito el contrato entero, el problema desaparece:

Código 15.2: El contrato con su poscondición dicha. La línea que faltaba es la de la documentación, no una de código.

```
class Habilidad(ABC):
    @abstractmethod
    def absorbe_impacto(self, portador) -> bool:
        """¿Se come este impacto? True = el portador no lo sufre.
```

```
5
6           POSCONDICION: la sucesion de respuestas a llamadas sucesivas debe
7           terminar dando False. Una habilidad que absorba siempre convierte a su
8           portador en indestructible y hace que la oleada no se despeje nunca.
9           """
```

Detengámonos a pensar · Un contrato que no se puede comprobar no sirve de nada

Y aquí está el problema del párrafo anterior: eso es un comentario. Nadie lo ejecuta. Un contrato de comportamiento solo vale si hay algo que lo comprueba, y en este caso ese algo es una prueba:

```python
def test_toda_habilidad_termina_cediendo():
    for clase in Habilidad.__subclasses__():
        h = clase()
        assert any(not h.absorbe_impacto(None) for _ in range(1000))
```

Mil llamadas y al menos una tiene que ceder. Es tosco y encuentra el fallo.
La lección no es la prueba: es que las poscondiciones de comportamiento hay que escribirlas y hay que comprobarlas. Lo que no se comprueba, en cuanto alguien escriba la cuarta habilidad, se romperá sin que nadie se entere.

Y ahí me quedé un rato, porque una prueba en verde es una respuesta muy pobre a la pregunta que de verdad importa.

Detengámonos a pensar · Y una prueba que pasa no demuestra que vigile nada

Esa prueba lleva todo el libro en verde, y eso no dice lo que parece decir. Una guardia que nunca ha cazado nada puede estar vigilando bien o puede estar mirando a otro lado, y desde fuera se ven igual.
Así que la puse a prueba a ella: escribí la habilidad inmortal del apartado «el invasor indestructible» entre las pruebas de contrato, esta vez heredando de `Habilidad`, y ejecuté. **La guardia la cazó sola**, sin que yo la llamara, porque recorre `Habilidad.__subclasses__()` y la clase nueva entró ahí por el hecho de existir.
Es la demostración que buscaba, y trae de regalo un problema: mientras el contraejemplo herede de `Habilidad`, la suite del juego se queda en rojo para siempre. **Un contraejemplo no puede vivir dentro de la jerarquía que sirve para vigilar.**
Por eso, entre las pruebas de contrato, la habilidad inmortal *no* hereda de nada: cumple la misma firma y se le pasa al mismo predicado, que es lo único que hacía falta. Y al lado va su pareja, que le pasa el mismo predicado a un escudo y sí cede. Las dos juntas dicen lo que una sola no puede decir: que la comprobación distingue.

15.4 El caso clásico —y por qué aquí no lo uso—

Todo texto sobre este principio recurre al mismo ejemplo: el cuadrado que hereda de rectángulo y rompe todo lo que suponía que ancho y alto son independientes.

Es un buen ejemplo y no lo voy a usar, por dos motivos.

Porque no le pasa a nadie. Nadie ha escrito nunca esa jerarquía en un programa de verdad. Aprender el principio con un caso que no ocurre deja la sensación de que el principio tampoco ocurre.

Y porque esconde dónde muerde de verdad. El fallo del cuadrado y el rectángulo está en los datos —dos atributos que dejan de ser independientes—. El que te va a pasar está en el comportamiento, como el de `HabilidadInmortal`, y ese no se ve mirando los atributos.

Sustituir de verdad

En el juego

`Bala` y `Bomba` son intercambiables allí donde el juego espera un proyectil: las dos vuelan, las dos salen del tablero, las dos mueren al salir.
El bucle las trata igual y no se entera de cuál tiene delante. Eso es sustituir.

En la vida real

Un sustituto en un puesto puede hacer lo que se esperaba del titular. Puede hacer además otras cosas —promete más— y eso no molesta a nadie.
Lo que rompe la sustitución es que necesite algo que el titular no necesitaba: «yo eso solo lo firmo con la autorización de mi jefe». Eso es exigir más, y quien contaba con la firma, se queda detenido.

15.5 Cuando la jerarquía era el error, no el sustituto

Termino con el diagnóstico que más veces acierta, y que ya apliqué sin nombrarlo en el capítulo 9.

Cuando una subclase no puede sustituir a su padre, casi nunca el problema es la subclase. El problema es que la jerarquía dice algo falso.

Cuadro 15.2: Tres incumplimientos y qué había detrás de cada uno. Ninguno se arregla en el hijo.

Síntoma	Diagnóstico	Arreglo
El hijo lanza `NotImplementedError` en un método	El contrato pedía de más	Partir el contrato (cap. 16)
El hijo exige un parámetro que el padre no exigía	No es un caso particular del padre	Sacarlo de la jerarquía
El hijo hace lo mismo pero «al revés»	Se heredó por parecido	Componer en vez de heredar (cap. 11)

Y el caso del OVNI, que ya conté, es el segundo de esa tabla visto antes de cometerlo: si hubiera heredado de `Invasor`, habría tenido que negarse a dar su columna. La solución no era arreglar el OVNI. Era no meterlo ahí.

Detengámonos a pensar · La pregunta que sustituye al principio entero

Si te quedas con una sola cosa de este capítulo, que sea esta:

¿Puedo darle este objeto a alguien que espera el padre, sin avisarle de nada?

Sin avisarle. Si tienes que decir «sí, pero ten en cuenta que este no...», entonces no. Y ese «ten en cuenta que» es el sonido exacto que hace una jerarquía mal puesta.

Si esto fuera un equipo · El sustituto que necesita permiso

Cubrir una baja funciona cuando quien sustituye puede hacer lo mismo, no necesariamente igual de bien, pero sí sin condiciones nuevas.

Cuando el sustituto necesita algo que el titular no necesitaba —una firma, una autorización, que le expliquen un caso—, quien contaba con ese puesto se queda detenido, y normalmente se entera tarde.

En el código pasa lo mismo y con el mismo retraso: el sustituto encaja al escribirlo y falla al usarlo.

Prompt · Buscar sustituciones rotas

```
Te paso una jerarquía de clases: [pégala].

No la cambies. Para cada subclase, dime:

1. ¿Exige algo para funcionar que la clase base no exigía? (precondición más
   fuerte)
```

```
2. ¿Promete menos que la base en algún método? Mira también lo que la base
   promete en su documentación, no solo su firma.

3. ¿Hay algún método donde el hijo no haga nada, o lance una excepción?

Después, y solo después: para cada incumplimiento, dime si lo arreglarías en el
hijo o si el problema es que esa clase no debería estar en esa jerarquía.
Justifica cuál de las dos, no propongas las dos.
```

Compruébalo · Escribe la poscondición que falta

Elige una clase base tuya con varias implementaciones. Toma un método.
Escribe, en una frase, **qué promete ese método más allá de lo que dice su firma**: que termine, que no devuelva vacío, que dos llamadas seguidas hagan tal cosa.
Ahora comprueba si todas las implementaciones lo cumplen. ¿Todas la cumplen?

Con esto quedan tres principios y una lección de fondo: dos se comprueban con herramientas y el tercero hay que sostenerlo escribiendo lo que un contrato promete de verdad.

El capítulo 12 dejó pendiente el otro fallo de los contratos, el de signo contrario: no el que promete de menos, sino el que *pide* de más y obliga a media docena de clases a implementar métodos que no usan. Lo abordaré en el capítulo siguiente.

Capítulo 16

Segregación de interfaces

El principio anterior se ocupa de quien promete de menos. Este, de quien *pide* de más.

Es el más corto de los cinco y muchas de las veces lo he visto incumplir sin que nadie lo note, porque su síntoma —un método vacío— parece inofensivo.

16.1 El enunciado

Definición 16.1 (Segregación de interfaces)**.** Nadie debe verse obligado a depender de métodos que no usa. Es preferible tener **varios contratos pequeños** que uno grande.

Detengámonos a pensar · Qué significa «depender de un método que no usas»

No es solo tener que escribirlo. Es peor que eso, y tiene tres consecuencias que conviene separar:

Tienes que implementarlo. Vacío, o lanzando un error. Trabajo inútil.

Tienes que mantenerlo. Cuando el contrato cambia, cambias tú, aunque el método que cambió no lo uses jamás.

Y quien te recibe no puede fiarse. Si la mitad de las clases que cumplen un contrato lo cumplen a medias, el contrato deja de significar algo. Esta es la que merece más atención y cuidado, y es la que enlaza con el capítulo anterior: un contrato que se cumple a medias es un contrato que miente.

16.2 La interfaz que obliga al marcador a saber dibujarse

`Entidad` promete tres cosas: ocupar un sitio, avanzar con el tiempo y saber con qué símbolo se dibuja. Casi todo en el juego cumple las tres.

Inténtalo primero · Qué cosas no cumplen las tres

Piensa en tres cosas que este juego podría tener, o tiene, y que **no** deberían prometer las tres.

Pista: dos de ellas existen ya en el código.

Las dos que existen son estas:

Cuadro 16.1: Lo que cada pieza necesita prometer. Ninguna de las dos últimas es una entidad, y forzarlas a serlo habría producido métodos vacíos.

Pieza	Ocupa sitio	Avanza	Se dibuja
`Invasor`, `Bala`, `Bunker`...	sí	sí	sí
`Habilidad`	no	sí, si quiere	no
`Sucesos`	no	no	no

Una habilidad tiene la oportunidad de actuar cada turno —la regeneración la usa— y no ocupa ningún sitio ni se dibuja. Si hubiera heredado de `Entidad`, tendría un `caja()` que devolvería qué, y un símbolo que no usaría nadie.

Por eso `Habilidad` es su propio contrato, con dos métodos, uno de ellos opcional. Y por eso `Colisionable` está separado de `Entidad`: pide una sola cosa, y así puede cumplirlo algo que no sea una entidad.

Un contrato grande y varios pequeños

En el juego

Si `Entidad` pidiera además saber guardarse en disco, los 55 invasores tendrían un método de guardado que nadie llama, y el día que cambiara el formato habría que revisarlos todos.
Con contratos separados, lo que se guarda cumple el contrato de guardarse. Lo demás ni se entera.

En la vida real

Un formulario único que pide atención de varios roles, habitualmente tiene partes en blanco, y hay que leerlo completo para saber qué apartado se llena.
Y cuando cambia el apartado que cubre a uno de los roles, se reimprime el formulario para todos —incluidos los que no debían siquiera enterarse—.
Un formulario por rol se rellena completo y cambia por separado.

16.3 Interfaces pequeñas: cuántas y de qué tamaño

Un criterio sencillo que sale de mirar quién llama a qué:

Detengámonos a pensar · Un contrato por cada grupo de clientes

No por cada método, y no uno para todos.
Mira quién usa el componente y qué le pide cada uno. Si todos los que lo usan le piden lo mismo, un contrato. Si hay dos grupos que piden cosas distintas, dos contratos.
En este juego:

- El bucle de la partida le pide a todo `avanzar()` y `vivo`.
- La resolución de impactos le pide `caja()` y nada más.

- La vista le pide `caja()` y `simbolo()`.

Tres grupos de clientes, y por eso `Colisionable` —que es lo que el segundo necesita— existe aparte. Es el contrato más pequeño del juego, con un método, porque el grupo que lo usa no necesita más.

Los cuatro contratos del juego, con su tamaño:

Cuadro 16.2: Los contratos del juego y cuánto piden. Ninguno tiene un método que alguna implementación deje vacío.

Contrato	Métodos	Quién lo usa
`Colisionable`	1	La resolución de impactos
`Habilidad`	2 (uno opcional)	Invasor, cañón y búnker
`EstrategiaDisparo`	1	La partida, una vez por turno
`Entidad`	4	El bucle y la vista

16.4 Segregar de más: la interfaz de un solo método por objeto

Llevado al extremo, «contratos pequeños» significa un contrato por método: `Avanzable`, `Dibujable`, `Mortal`, `Ubicable`. Cuatro contratos donde había uno, y cada entidad declarando los cuatro.

Detengámonos a pensar · Por qué eso es peor

No ha separado nada. Si las mismas clases cumplen los cuatro, los cuatro cambian juntos y se mantienen juntos. Solo hay más nombres.

Se pierde la idea. `Entidad` dice algo: «esto es una cosa que existe en el tablero». `Avanzable` + `Dibujable` + `Mortal` no dice nada; hay que reconstruir la idea leyendo la lista.

Y la prueba: no hay dos grupos de clientes. El bucle y la vista usan casi lo mismo. Separar contratos donde no hay clientes distintos es partir por partir.

Inténtalo primero · Aplica el criterio a un caso real

`Entidad` tiene cuatro métodos y dos grupos de clientes que se solapan casi del todo: el bucle usa `avanzar()` y `vivo`; la vista usa `caja()` y `simbolo()`.

Se solapan «casi». ¿Los partirías?

No hay respuesta correcta, y por eso lo pregunto. Escribe tu decisión y el motivo, y compárala con el mío.

El mío fue no partirlos, y tengo un motivo y una duda.

El motivo: las cuatro cosas cambian juntas. Si algún día una entidad deja de dibujarse, dejará también de existir en el tablero. No he encontrado un caso donde uno de los dos grupos

cambie sin el otro.

La duda: si el juego tuviera dos vistas —consola y gráfica— con necesidades distintas de dibujado, probablemente sacaría `Dibujable`. Hoy tiene una vista y media —la consola y la muda—, y la muda no dibuja nada, así que no hay tensión.

Sin programar · Cuántos formularios

La pregunta que resuelve esto en cualquier organización no es «¿cuántos formularios?». Es **«¿cuántos tipos de solicitante hay?»**.

Si tres perfiles distintos piden tres cosas distintas, tres formularios. Si son tres perfiles que piden lo mismo, uno.

Y el exceso también se reconoce: un formulario por campo obliga a rellenar seis para hacer un trámite. Nadie llamaría a eso simplificar.

Si esto fuera un equipo · La descripción del puesto que pide de todo

Una oferta que exige diez capacidades de las que el puesto usa tres deja fuera a gente que habría hecho el trabajo, y obliga a quien entra a demostrar siete cosas que no va a usar. Y como en el código, tiene un efecto de segundo orden: cuando media plantilla figura con capacidades que no ejerce, la descripción del puesto deja de significar nada y ya no sirve para buscar a nadie.

Compruébalo · Busca los métodos que nadie llama

Sobre una interfaz o clase base tuya, sigue cada método y busca quién lo llama.

Después, para cada implementación, mira si ese método hace algo de verdad.

Un método que en la mitad de las implementaciones está vacío, o que solo llama un cliente de cinco, es un candidato a salir del contrato. No lo saques todavía: comprueba antes si hay dos grupos de clientes distintos. Si no los hay, el contrato está bien y lo que sobra es el método.

Cuatro principios. Los contratos ya no piden de más ni prometen de menos, las clases tienen una razón para cambiar y el juego crece por donde tiene contrato.

Queda el quinto, que es el que ata todo lo anterior y el que más se confunde con una técnica concreta. Y para verlo hace falta mirar algo que hasta ahora no he dibujado: de quién depende cada pieza de este juego.

Capítulo 17

Inversión de dependencias

El quinto principio es el que ata los otros cuatro, y también el que más se confunde: se le suele llamar por el nombre de una técnica que no es.

Empiezo por dibujar de quién depende cada pieza del juego, porque el principio se ve mucho mejor sobre el mapa que sobre la definición.

17.1 El enunciado

Definición 17.1 (Inversión de dependencias)**.** Lo importante no debe depender de lo accesorio. Los dos deben depender de un **contrato**, y el contrato pertenece a lo importante.

La segunda frase es la que casi siempre se omite, y es la que más importa. No basta con meter una interfaz por medio: hay que preguntarse **de quién es** esa interfaz.

Detengámonos a pensar · Qué se invierte exactamente

Sin el principio, la dependencia va en la dirección natural: el juego necesita guardar marcadores, así que el juego conoce el archivo.

```
Partida  ----->  EnArchivo        (el juego conoce el disco)
```

Con el principio, el juego declara qué necesita —un sitio donde anotar— y es el archivo quien se adapta a eso:

```
Partida  ----->  RepositorioMarcadores  <-----  EnArchivo
```

La flecha de la derecha ha cambiado de sentido. Eso es la inversión, y por eso se llama así.

Y fíjate en de quién es el contrato: `RepositorioMarcadores` lo define el juego, con las palabras del juego —«anotar», «mejores»—. No lo define el disco. Si lo definiera el disco, hablaría de archivos y no habríamos invertido nada: solo habríamos añadido un intermediario.

17.2 De quién depende el juego, dibujado

Esto sale de recorrer las importaciones del juego y contarlas en las dos direcciones.

Cuadro 17.1: Acoplamiento del juego terminado. «Depende de» son módulos del propio paquete que ese módulo importa; «dependen de él», cuántos lo importan. Medido sobre las 23 unidades del paquete.

Pieza	Depende de	Dependen de ella	Qué es
La geometría —`Caja`—	0	9	Lo más estable
Los tipos propios	0	5	Lo más estable
El contrato de habilidad	0	5	Contrato
El publicador de sucesos	0	2	Contrato
El repositorio	0	1	Contrato y sus dos casos
El contrato `Entidad`	1	6	Contrato
La vista de consola	0	1	Lo más accesorio
La partida	12	1	El orquestador
El punto de entrada	11	0	La raíz de composición

Inténtalo primero · Lee la tabla antes de que la lea yo

Hay un patrón en esas cifras, y dice algo sobre el diseño.

Fíjate en las dos columnas de números: ¿qué relación hay entre «depende de muchos» y «dependen de él muchos»? ¿Y qué tipo de módulo está en cada extremo?

El patrón aparece por sí mismo.

Lo que no depende de nada es de lo que depende todo. La geometría y los tipos propios no importan nada y los importan catorce piezas entre los dos. Son las piezas más estables del juego, y por eso pueden sostener al resto.

Y lo que depende de todo no lo usa casi nadie. La partida depende de doce piezas y solo una depende de ella. El punto de entrada depende de once y de él no depende nadie.

Detengámonos a pensar · Los dos extremos y por qué están bien

Un módulo con doce dependencias suena a alarma. Aquí no lo es, y conviene saber distinguirlo.

El punto de entrada es la **raíz de composición**: el único sitio que conoce todas las piezas a la vez, precisamente porque es quien las monta. Alguien tiene que saber que este juego usa la vista de consola y el repositorio en archivo. Concentrar ese conocimiento en un solo sitio —y que ese sitio no lo use nadie— es exactamente lo que permite que el resto no lo sepa.

La partida es el orquestador, y su cifra sí es una señal: es la clase con cinco razones para cambiar del capítulo 13. Sus doce dependencias son la misma deuda contada de otra manera.

La regla que sale de la tabla: está bien depender de mucho si nadie depende de ti.

Lo que no puede ser es depender de mucho *y* que muchos dependan de ti: eso es un cuello de botella.

17.3 Depender de lo que se promete, no de quién lo cumple

El caso concreto, y es el que hace visible el principio.

Código 17.1: La partida no sabe de dónde sale el azar. Y por eso la partida se puede repetir.

```
class Partida:
    def __init__(self, semilla=0, fabrica=None, estrategia=None,
                 sucesos=None, arma=None, ...):
        self._azar = random.Random(semilla)
        self._fabrica = fabrica or FabricaDeOleadas()
        self._estrategia = estrategia or AlAzar()
        self.sucesos = sucesos or Sucesos()
```

Cuatro piezas que la partida usa y no fabrica según su tipo concreto: recibe lo que le den, y solo exige que cumpla el contrato.

⚖ Depender del papel, no del nombre

En el juego

La partida depende de «una estrategia de disparo». Hay tres, y podría haber diez. Ninguna está escrita dentro de la partida. Por eso el capítulo 21 puede cambiar cómo dispara la invasión sin abrir la partida, y por eso las pruebas pueden pasarle una estrategia falsa que dispare siempre a la misma columna.

En la vida real

Una empresa depende del papel «proveedor de material», no de un proveedor con nombre y apellidos.
Cuando depende del nombre, cambiar de proveedor obliga a rehacer procesos internos que nunca deberían haber sabido quién era.
Y hay una prueba: si al cambiar de proveedor hay que reescribir procedimientos, es que dependían del nombre.

17.4 DIP no es inyección de dependencias, y confundirlas es lo normal

Cuadro 17.2: El principio y la técnica. Contestan a preguntas distintas.

	Inversión de dependencias	**Inyección de dependencias**
Qué es	Un principio	Una técnica
Responde a	*¿De qué debo depender?*	*¿Cómo me llega?*
Dice	«De un contrato, y el contrato es mío»	«Te lo pasan; no lo fabricas tú»
Dónde vive	En el diseño	En el constructor
Capítulo	Este	El 26

Detengámonos a pensar · Se puede tener cada una sin la otra, y las dos cosas pasan

Cumplir el principio sin inyectar nada. Una función que recibe cualquier cosa que sepa dar su caja depende de un contrato, y no hay ningún constructor ni ningún contenedor por medio. Principio: sí. Inyección: no.

Inyectar sin invertir nada. Esto es lo frecuente, y es la trampa:

```
def __init__(self, escritor_json_en_disco):   # se lo inyectan...
    self._escritor = escritor_json_en_disco   # ...y sigue siendo un JSON
```

Recibe la dependencia desde fuera —técnica aplicada— y sigue dependiendo de una clase concreta que sabe de JSON y de disco. No se ha invertido nada: la flecha apunta en la misma dirección de siempre, solo que ahora entra por el constructor.

La prueba para distinguirlas: mira el *tipo* de lo que recibes. Si es un contrato con vocabulario de tu problema, has invertido. Si es una clase concreta con vocabulario de la herramienta, solo has movido la construcción de sitio.

17.5 Invertir de más: la abstracción con una sola implementación

Y el exceso, que en este principio es el más caro de todos porque no se ve.

Cada inversión añade un contrato, una indirección y un sitio más donde mirar. Cuando la abstracción tiene una sola implementación y no se ve cuál sería la segunda, se ha pagado el coste sin comprar nada.

Detengámonos a pensar · Las tres del juego, examinadas

Soy consecuente y las examino:

`EstrategiaDisparo`**: tres implementaciones.** Justificada.

`Habilidad`**: cuatro, contando la que no hace nada.** Justificada.

`RepositorioMarcadores`**: dos —archivo y memoria—.** Justificada, y la segunda merece nuestra atención: es la que usan las pruebas, y sin ella no se podría comprobar el guardado sin ensuciar el disco.

Y una que **no** llegué a escribir: pensé en un contrato `Vista` para que consola y vista muda dependieran de él. No lo hice, porque la vista muda no es otra vista: es la ausencia de vista. Con una sola implementación de verdad, el contrato habría sido un archivo con una promesa y ningún cliente que lo eligiera.

Detengámonos a pensar · Regla que uso, y viene del capítulo 14

No inviertas por si acaso. Invierte cuando aparezca la segunda implementación, o cuando la primera te impida probar.

La segunda mitad es importante y se olvida: *necesitar probarlo* es una segunda implementación legítima. El repositorio en memoria existe por eso, y no es trampa: una prueba es un cliente tan real como el juego.

Si esto fuera un equipo · Depender de un papel o de una persona

Un equipo que depende de «quien sabe cerrar el mes» tiene una dependencia con nombre propio. Funciona perfectamente hasta que esa persona se va de vacaciones.

Uno que depende del papel «responsable de cierre» puede tener a otra persona haciéndolo, con su método, sin reescribir nada.

Y el exceso también existe, y produce organigramas con papeles que ocupa siempre la misma persona: la indirección está escrita y no invierte nada.

Prompt · ¿De qué depende esto de verdad?

```
Te paso un módulo: [pégalo].

No propongas interfaces todavía. Primero:

1. Lista de qué depende: clases concretas, bibliotecas, formatos, rutas.

2. Para cada dependencia, dime si el vocabulario es DEL PROBLEMA ("anotar un
   marcador") o DE LA HERRAMIENTA ("escribir un JSON"). Solo las segundas son
   candidatas a invertir.

3. Para cada candidata, dime si HOY existe una segunda implementación posible,
   o si probarla sin ella es difícil. Si no se cumple ninguna de las dos,
```

```
   dímelo expresamente y no propongas invertirla.

4. Solo para las que pasen el filtro: propón el contrato, con el vocabulario
   del problema y no el de la herramienta.
```

Compruébalo · Dibuja tus flechas

Toma tu programa y haz la lista de importaciones de cada módulo. La mayoría de lenguajes lo dan con una orden.
Para cada módulo cuenta dos números: de cuántos depende y cuántos dependen de él. Después identifica aquel que tenga elevados ambos valores. Ese es tu cuello de botella, y probablemente ya lo sabías sin poder demostrarlo. Ahora puedes.

Con esto se cierra la Parte V. Los cinco principios ya no son un acrónimo: son cinco criterios con los que se puede discutir una decisión de diseño, cada uno con su medida y con su exceso declarado.

Y hay algo que conviene decir antes de seguir. En estos cinco capítulos he incumplido tres de los cinco principios a propósito: el enumerado que se abre, la partida con cinco razones para cambiar, la entidad con dos grupos de clientes que no separé. Un libro que presentara cinco principios y un código que los cumple todos estaría enseñando otra cosa: que existen diseños sin tensiones. No existen.

Lo que viene ahora es la mitad práctica. El juego funciona y está bien repartido, y todavía no hemos hablado de lo que consume ni de si aguantaría una partida de horas. Y ahí, en un juego que nació en una máquina con muy poca memoria, es donde la pregunta cambia.

VI

Que funcione no es suficiente

Capítulo 18

El juego que nació apretado

El juego funciona y está repartido. Es momento de validar su rendimiento, para ello debemos regresar a verlo desde otro punto: **lo que consume**.

Y para eso hay que hablar de 1978, porque el Space Invaders original no nació en una máquina con recursos. Nació en una con características muy modestas.

18.1 Las condiciones de 1978

Vamos a incomodar a más de uno a partir de ahora.

Detengámonos a pensar · Lo que no voy a contarte, y por qué

Todo el mundo sabe la historia: el hardware era tan justo que, al morir invasores, la máquina tenía menos que dibujar y la formación se aceleraba sola. La aceleración —la mecánica más recordada del juego— habría sido un **accidente**.

Es una historia magnífica. La he buscado y **no he encontrado ni una fuente que la sostenga**.

Está en enciclopedias colaborativas, en prensa técnica y en decenas de artículos que se citan entre sí. Busqué la entrevista con Tomohiro Nishikado donde debería estar, la abrí, y **no la contiene**. Busqué literatura académica sobre el hardware del arcade: no hay.

Así que no la cuento como un hecho. La cuento como lo que he podido comprobar: que es una historia repetida en todas partes cuyo origen no he sabido localizar.

Lo que sí puedo afirmar, porque no depende de ninguna fuente sobre 1978, es esto: **en aquella máquina el consumo no era una preocupación de ingeniería elegante. Era la condición para que el juego existiera.** Si no cabía, no había juego.

18.2 Optimizar no era una virtud: era la condición para que el juego existiera

Merece la pena detenerse en la diferencia, porque es lo que hace útil el recuerdo.

Dos maneras de mirar el consumo

Cuando no cabe

La restricción decide el diseño. Si el juego no entra en la memoria que hay, no se publica.
Eso obliga a saber, en todo momento, cuánto ocupa cada cosa. No como comprobación posterior: como parte de escribirlo.
Y produce un efecto secundario que se nota: no hay nada de más, porque no cabe nada de más.

Cuando sobra sitio

La restricción desaparece del diseño. Todo entra, todo corre, y nadie mide.
El consumo deja de ser una decisión y pasa a ser un residuo: lo que salga.
Y produce el efecto contrario: hay mucho de más, y no molesta, porque nada avisa.

18.3 La pregunta incómoda: hoy sobran recursos, ¿para qué preocuparse?

Hoy hay memoria de sobra. Este juego, escrito con toda la desidia del mundo, corre en cualquier ordenador y sobra máquina. Si aun malgastando funciona en segundos, *¿no es el consumo un problema resuelto?*

Inténtalo primero · Contéstala tú primero

Es una pregunta legítima y tiene una respuesta buena. Antes de leer la mía, escribe la tuya.
Y una pista de lo que busco: la respuesta **no** es «porque hay que hacer las cosas bien». Eso no es una razón, es una preferencia. Busca las razones que se puedan medir.

18.4 La respuesta: sí, y por cuatro motivos que convergen

Sí hay que preocuparse. Y no por elegancia: por cuatro motivos que se refuerzan entre sí.

Uno. Ambientes grandes. Lo que sobra en una máquina falta multiplicado por mil. Un desperdicio que en tu portátil no se nota es, en un servicio con muchos usuarios, la diferencia entre una máquina y diez. El consumo energético de la computación es hoy un cuello de botella sistémico, y lo es por coste, por clima y por límites de suministro (Nana Tchakoute & Tadonki, 2026).

Dos. Tareas múltiples. Nadie ejecuta un solo programa. Tu juego no compite contra el vacío: compite contra el navegador, el editor y otras quince cosas. La memoria que reservas de más no se la quitas a la máquina: se la quitas al resto.

Tres. Portabilidad. La máquina de destino no es la de desarrollo. Un programa escrito sin mirar el consumo funciona donde se escribió y hay que rehacerlo para todo lo demás. Y hoy «todo lo demás» incluye aparatos con menos memoria que el portátil donde se escribió.

Cuatro. Instalación. Lo que ocupa, lo que hay que descargar y lo que tarda en arrancar. Es lo primero que ve quien lo usa, y suele ser lo último que mira quien lo escribe.

Detengámonos a pensar · Lo que la fuente sostiene y lo que no

He citado un trabajo, y quiero ser preciso sobre qué respalda.

Sostiene que el consumo energético de la computación es un problema sistémico y creciente, y que las optimizaciones de software son una de las vías que se estudian para atacarlo.

No sostiene ninguna cifra concreta sobre este juego, ni sobre Python, ni permite atribuir un ahorro determinado a una decisión de diseño. Es una condición general de diseño.

Los cuatro motivos de arriba son, entonces, un argumento mío apoyado en un respaldo general.

18.5 No se trata solo de resolver la tarea, sino de hacerlo bien

En el libro anterior el reto era **garantizar que la máquina no pudiera perder**. Un problema de corrección: se resuelve una vez, se demuestra y se acabó.

Aquí el reto es otro, y conviene enunciarlo:

Detengámonos a pensar · Qué hay que garantizar en este juego

Que **las mecánicas funcionen**. Que la velocidad de los enemigos sea la adecuada. Que resulte atractivo de jugar. Que sea **estable durante horas**. Que tenga el dinamismo que se buscaba.

Ninguna de esas cinco cosas se demuestra. Se *ajusta*: se prueba, no convence, se cambia la cadencia, se vuelve a probar. Y se ajusta cientos de veces.

Y «estable durante horas» no es una manera de hablar. Es necesario medirlo:

Cuadro 18.1: Sesión larga: 21 partidas encadenadas, 61 057 turnos, con el jugador automático. La columna de la derecha es memoria reservada frente al arranque, medida con `tracemalloc`. Reproducible con `medir_memoria.py --largo 60000`.

Turno	Partida	Invasores vivos	Proyectiles	Memoria
7 500	3	2	7	31 405 B
15 000	6	51	2	30 945 B
22 500	7	2	0	31 997 B
30 000	10	37	2	31 217 B
37 500	14	3	7	32 885 B
45 000	17	44	4	31 233 B
52 500	19	1	7	33 309 B
60 000	21	10	7	32 625 B

Detengámonos a pensar · Qué dice esa tabla, y qué no

Dice que después de 21 partidas seguidas la memoria está donde estaba: varía un 3,9 % entre la primera medida y la última, y no crece. No hay nada que se acumule: ni objetos que no se sueltan, ni oyentes que se apilan al suscribirse partida tras partida, ni listas de proyectiles que crezcan sin límite.

Eso es lo que significa «aguanta»: no que sea rápido, sino que la partida número veintiuna se comporte como la primera.

No dice que no haya fugas. Dice que en 61 057 turnos no apareció ninguna. Es una comprobación por muestreo, no una demostración.

Sin programar · Por qué esa medida es la que importa

Un programa que funciona es fácil de comprobar: lo ejecutas y funciona.

Un programa que *aguanta* no se comprueba ejecutándolo una vez. Prácticamente todos los fallos que arruinan un producto —el que se pone lento después de dos horas, el que se cae el viernes por la tarde— son de esta familia, y ninguno aparece en una prueba corta. La forma de verlos es mirar una magnitud que debería quedarse quieta y comprobar que se queda quieta. Aquí es la memoria. En otro sitio será el número de conexiones abiertas o el tamaño de un archivo de registro.

18.6 La tensión que no vamos a esquivar

Termino con una advertencia sobre lo que viene, porque las dos mitades de esta parte tiran en direcciones opuestas y sería deshonesto no decirlo.

Las dos mitades de esta parte se contradicen

Dimensionar el dato

Dimensionar el dato **ahorra**: elegir el contenedor adecuado para 55 invasores es la diferencia entre 1 644 y 14 856 bytes, según medí para el capítulo siguiente.

Menos memoria, menos objetos, menos trabajo para el recolector.

Los ocho patrones

Los ocho patrones que en breve abordaremos **gastan**. Cada uno añade indirección: un objeto estrategia, una lista de oyentes, un repositorio, una fábrica.

Más objetos, más llamadas, más saltos por fotograma.

No es una contradicción aparente. Es real, y se paga.

Si esto fuera un equipo · Lo que sobra en uno falta multiplicado por mil

Un procedimiento que hace perder cinco minutos a cada persona no cuesta cinco minutos: cuesta cinco minutos por esa cantidad de personas y por día.

Y como en el software, el problema es que nadie los ve. Cinco minutos no se notan en la agenda de nadie; solo aparecen cuando alguien los suma.

De ahí la parte útil de la analogía: el consumo no se percibe, se mide. Y lo que no se mide, no existe hasta que es un problema.

Ya está dicho por qué el consumo importa y qué hay que garantizar en un juego que se ajusta en lugar de demostrarse.

El capítulo siguiente baja al detalle más concreto que existe: cuánto ocupa un número. Y lo hace en un lenguaje que, a diferencia del de 1978, no te obliga a saberlo.

Capítulo 19

Dimensionar el dato

Un invasor guarda su resistencia: un número que vale 1 o 2. Nunca 3, nunca -1, nunca cuatro mil millones.

Este capítulo trata de cuánto ocupa ese número, de por qué en Python la pregunta se hace distinto, y de qué resultado obtuve al medirlo.

19.1 Un entero de propósito general para guardar un número del 0 al 5

En un lenguaje con tipos de ancho fijo, esto es una decisión de una línea: un número del 0 al 5 va en un byte, y escribir `int` donde cabe un `byte` es gastar cuatro veces lo necesario.

19.2 En Python el ancho lo decide el contenedor, no la variable

Estuve un rato buscando cómo declarar un entero estrecho en Python, y la respuesta es que no se puede. Un `int` de Python es de precisión arbitraria: crece hasta donde haga falta y no se declara corto.

Lo que hay que cambiar es la pregunta:

> **En Python el ancho lo decide el contenedor, no la variable.**

No eliges cuánto ocupa el número. Eliges **dónde lo guardas**, y ahí sí hay opciones, y son del lenguaje: `bool`, `IntEnum`, `bytes`, `bytearray`, `array`, `struct`, `__slots__`.

Y la reformulación resulta ser más útil que la original, porque se traslada: también en un lenguaje con `byte`, guardar un millón de bytes sueltos en una lista de punteros gasta ocho veces más que guardarlos en un vector. Dimensionar nunca fue solo elegir el tipo escalar. Ahí se nota menos.

19.3 Los tipos que Python sí te da, y cuál corresponde a cada dato

Esta es la caja de herramientas: son tipos del lenguaje.

Cuadro 19.1: Qué usar para cada cosa. La columna de la derecha es la que guía la decisión.

Tipo	Para qué	Cuándo lo elijo
`bool`	Dos valores	Vivo o muerto. Nunca un `0` y un `1` sueltos
`IntEnum`	Pocos valores *con nombre*	Tipo de invasor, bando, desenlace. Es un entero y además dice qué es
`bytes`	Muchos valores de 0 a 255, que no cambian	Una tabla fija
`bytearray`	`bytes`, mutable	Estados de 55 invasores
`array`	Muchos números del mismo ancho	Coordenadas, con su código de tipo
`struct`	Cuando importa el orden de los bytes	Guardar o transmitir
`__slots__`	Objetos de los que hay muchos	Toda entidad del juego

Detengámonos a pensar · Lo que *no* hago, y por qué

No uso `ctypes`, ni estructuras empaquetadas para simular un `byte` en memoria, ni trucos para que un atributo ocupe menos.

Eso sería escribir C dentro de Python, y contradiría la razón por la que elegí Python en el capítulo 1: la legibilidad. Un juego que hay que descifrar para leerlo ya no sirve al lector que no programa, y ese lector es uno de los tres en los que pensé al decidir escribir esta serie.

La regla que sigo: **elegir bien entre lo que el lenguaje ofrece, y no pelearme con el lenguaje.**

19.4 Lo que cuesta un objeto antes de guardar nada

Antes de medir los datos, hay que medir el envase, porque en Python el envase no es gratis.

Cada objeto lleva, por omisión, un diccionario propio donde guarda sus atributos. Es lo que permite añadirle atributos nuevos sobre la marcha, y en un juego con 55 invasores es una comodidad que nadie usa y todos pagan.

Código 19.1: Una línea que quita el diccionario. Todas las entidades del juego la llevan.

```
class Invasor(Entidad):
    __slots__ = ("_tipo", "_fila", "_columna", "_resistencia", "_habilidad")
```

19.5 Medido: una oleada de invasores en cuatro representaciones

Cuadro 19.2: Los mismos 55 invasores, guardados de cuatro maneras. Medido con `tracemalloc`, que cuenta lo que Python reserva de verdad y no solo el tamaño declarado del objeto de cabecera.

Representación	Bytes	Veces
Objetos con diccionario	14 856	9,0×
Objetos con `__slots__` —lo que usa el juego—	11 208	6,8×
Listas paralelas —las del capítulo 2—	3 128	1,9×
Campos paralelos en `array`	1 644	1,0×

De la tabla, podemos obtener tres conclusiones.

Uno: `__slots__` ahorra un 25 %. De 14 856 a 11 208 bytes, por escribir una línea. Como suelo decir, es dinero encontrado en el suelo.

Dos: los vectores de un byte son 6,8 veces más pequeños que eso. Un orden de magnitud entero separa «objetos bien hechos» de «datos empaquetados».

Y tres, la incómoda: la tercera fila es el juego del capítulo 2, con sus cuatro listas `inv_x`, `inv_y`, `inv_fila` e `inv_vivo`.

Una aclaración importante, vectores y `array` no son lo mismo, una lista de Python guarda punteros a objetos, no bytes. Medido de verdad, el monolito ocupa **3 128 bytes**.

Detengámonos a pensar · La versión monolítica gasta tres veces y media menos memoria

Dieciséis capítulos justificando por qué el invasor número 7 debe existir como objeto, y resulta que la representación que descartamos ocupa **3,6 veces menos** que la que tenemos ahora.

Los dos casos, y la frontera está en un número

Cuándo gana el objeto

Hay pocos, cambian mucho y cada uno tiene comportamiento propio.
55 invasores que deciden si un impacto los mata, que llevan habilidades y de los que se pregunta la columna.

Cuándo gana el vector

Hay muchísimos, son homogéneos y se recorren haciendo lo mismo.
Un millón de partículas que solo tienen posición y velocidad, y sobre las que únicamente se hace «avanza todas».

Detengámonos a pensar · La frontera, dicha con un número

No es «depende». Es esto:

Cuenta cuántos hay y multiplica por lo que cuesta cada uno. Si el total es despreciable frente a la memoria disponible, elige por diseño. Si no lo es, elige por consumo.

En este juego: 55 invasores × 204 bytes = 11,2 KB, y lo que me ahorraría volver atrás

son 8,1 KB. En la actualidad se considera despreciable. Decido entonces por diseño.

19.6 Cuándo dimensionar es prematuro, y cómo se sabe

Hay una frase muy repetida sobre la optimización prematura, y como casi todas las frases muy repetidas, no son un criterio real.

Cuadro 19.3: Cuándo sí y cuándo no. La columna del medio es la que hay que contestar con un número, no con una intuición.

Situación	La pregunta	Qué hacer
No lo has medido	—	Mídelo. Nada más
Medido y es despreciable	¿Cuánto es frente al total?	Elegir por diseño
Medido y pesa	¿Crece con algo?	Dimensionar la parte que crece, y solo esa
Pesa y no crece	¿Cuánto trabajo cuesta arreglarlo?	Casi siempre, dejarlo

Detengámonos a pensar · Y el error del otro lado, que también existe

Dimensionar de más produce código que nadie entiende para ahorrar lo que nadie echaba de menos.
11 208 bytes son despreciables y no crecen. Cambiar un juego legible por un ahorro de diez kilobytes no es un buen negocio.

Si esto fuera un equipo · Medir antes de recortar

Un recorte decidido sin medir cae siempre donde es más visible, no donde más sobra. Es la versión organizativa de optimizar el bucle equivocado.
Y el exceso también se reconoce: procesos de control que cuestan más que lo que vigilan. Una revisión que consume media hora para autorizar un gasto de veinte dólares no está controlando: está gastando.
En los dos casos la salida es la misma y es aburrida: contar antes de decidir.

Prompt · Dimensionar con medida, no con intuición

```
Tengo esta estructura de datos: [pégala]. Hay [N] de ellas en memoria a la vez.

No la optimices. Antes:

1. Cuestióname: ¿el número N que te he dado crece con algo, o es fijo?
```

```
   Si crece, ¿con qué?

2. Escríbeme un guion CORTO que mida cuánto ocupan realmente N de estas, con
   tracemalloc y no con getsizeof. Explícame en una frase por qué getsizeof no
   sirve aquí.

3. Cuando yo te dé el resultado de ejecutarlo, y solo entonces, dime si merece
   la pena cambiar algo, comparándolo con la memoria disponible.

No propongas optimizaciones antes del paso 3. Si crees que hay una obvia,
anótala aparte y espera al número.
```

Con el consumo medido y la decisión tomada con un número delante, empieza la segunda mitad de esta parte: los ocho patrones.

Capítulo 20

Factory: crear sin saber qué se crea

Empiezan los ocho patrones. Los ocho llegan igual: primero el dolor, después la salida, y al final lo que cuesta la salida.

20.1 El dolor: montar cada oleada a mano

Al llegar a la segunda oleada, esto era lo que tenía dentro de la partida:

Código 20.1: Componer una oleada, dentro de la clase que además hace correr el tiempo.

```
    def _siguiente_oleada(self):
        self.oleada += 1
        descenso = min(self.oleada - 1, 4)
        invasores = []
        for fila in range(5):
            tipo = (CALAMAR, CANGREJO, CANGREJO, PULPO, PULPO)[fila]
            if fila == 0 and self.oleada >= 3:
                tipo = ACORAZADO
            for col in range(11):
                habilidad = SIN_HABILIDAD
                if fila == 0 and self.oleada >= 5:
                    habilidad = Escudo(1)
                invasores.append(Invasor(tipo, fila, col, ...))
        self.formacion = Formacion(invasores)
```

Funciona. Y tiene tres cosas mal, y solo la tercera me decidió.

La partida sabe demasiado. Sabe que hay cinco filas y once columnas, qué tipo va en cada fila, a partir de qué oleada aparecen los acorazados y a partir de cuál llevan escudo. Nada de eso es asunto de quien hace correr el tiempo.

Es una razón más para cambiar. La cuarta de las cinco que el capítulo 13 le contó a `Partida`. Ajustar la dificultad —que es lo que más veces se hace en un juego— obligaba a abrir la clase central.

Y no podía probar una oleada sin jugar. Para ver cómo queda la séptima había que jugar hasta la séptima. Con el jugador automático eso son unos dos mil turnos y varios segundos.

Detengámonos a pensar · El dolor real casi nunca es el que se cuenta en los libros

Los dos primeros motivos son los que aparecen en cualquier explicación de este patrón, y los dos son ciertos.

El que me hizo levantarme a cambiarlo fue el tercero, y es de otra clase: no es que el diseño fuera feo, es que **el bucle de afinar era insoportable**. Cambiar un número, esperar, mirar, repetir.
Aquí el reto es que las mecánicas funcionen, y una mecánica se afina cientos de veces. Cualquier cosa que ponga varios segundos entre el cambio y el resultado no es una molestia: es lo que hace que dejes de afinar.

20.2 Pedir la oleada 7

Definición 20.1 (Factory)**.** Una *fábrica* es un objeto cuyo único oficio es construir otros. Quien la usa pide lo que quiere y no sabe cómo se compone.

Código 20.2: Todo lo que la partida sabe ahora sobre componer una oleada.

```
    def _siguiente_oleada(self):
        self.oleada += 1
        self.puntuacion += BONO_OLEADA
        self.formacion = self._fabrica.crear(self.oleada)
```

Y la fábrica, que sabe eso y nada más:

Código 20.3: La fábrica. Cinco parámetros, y los cinco son perillas de dificultad.

```
class FabricaDeOleadas:
    """Sabe componer una oleada. Es lo unico que sabe."""

    def __init__(self, columnas=11, filas=5, descenso_maximo=4,
                 oleada_acorazados=3, oleada_escudos=5):
        ...

    def crear(self, numero) -> Formacion:
        ...
```

Pedir montado

En el juego

`fabrica.crear(7)` devuelve la séptima oleada, con sus acorazados y sus escudos. Quien la pide no sabe cuántas filas tiene ni desde qué oleada hay blindados. Y ahora puedo mirar la oleada 7 sin jugar seis: la creo y la miro.

En la vida real

Pides a compras «un puesto de trabajo montado» y llega con su mesa, su equipo y sus accesos.
No eliges el proveedor de la silla, y no quieres elegirlo. Si mañana cambian de proveedor, tú pides exactamente lo mismo.

Detengámonos a pensar · Lo que de verdad compró la fábrica

No fue el diseño. Fue esto:

```
>>> FabricaDeOleadas(oleada_acorazados=2).crear(7).vivos
55
```

Puedo inspeccionar cualquier oleada, con cualquier ajuste de dificultad, sin jugar ni un turno. El bucle de afinar pasó de varios segundos a instantáneo.
Y hay un segundo efecto que no busqué: los cinco parámetros del constructor **son** las perillas de dificultad del juego, y ahora están todas juntas en un sitio con nombre. Antes estaban repartidas por el cuerpo de un método de la clase central. Nadie las había reunido; la fábrica las reunió sola.

20.3 Qué se gana y qué se paga

Cuadro 20.1: El balance de este patrón, sobre este juego. Las cifras de la última fila están medidas sobre el juego terminado.

Se gana	Se paga
Ajustar la dificultad sin abrir la partida	Un módulo más y una clase más
Inspeccionar cualquier oleada sin jugar	Una indirección: para saber qué invasores hay, hay que abrir otro archivo
Las perillas de dificultad, juntas y con nombre	Un objeto más vivo durante la partida
Una razón menos para cambiar en `Partida`	La fábrica depende de cinco piezas

Detengámonos a pensar · Cuándo *no* vale la pena

Una fábrica que solo sabe llamar a un constructor no aporta nada. Si `crear()` es `return Invasor(...)` y ya está, has añadido un archivo.
La prueba es la misma del capítulo 13: **¿podría haber una segunda fábrica?** Aquí sí —una de oleadas simétricas, una que lea la composición de un archivo, una que genere oleadas al azar para probar—, y por eso existe.
Si no imaginas la segunda, lo que necesitas es una función, no una clase.

Si esto fuera un equipo · Pedir un resultado, no una lista de piezas

Un encargo bien hecho dice qué hace falta conseguir. Uno mal hecho enumera los pasos, y entonces quien lo recibe no puede mejorarlos y quien lo da tiene que conocerlos todos. Y el efecto secundario es el mismo que en el juego: cuando el encargo enumera pasos, cambiar un paso obliga a reescribir todos los encargos.

Compruébalo · Tu bucle de afinar

Elige algo de tu programa que ajustes a menudo —un umbral, un formato, una regla— y cronometra el ciclo completo: cambiar el valor, ejecutar, ver el resultado.
Si pasa de unos segundos, ese es tu problema, y casi siempre se arregla pudiendo construir el caso directamente en lugar de llegar a él.

Con la fábrica fuera, `Partida` tiene cuatro razones para cambiar en lugar de cinco.

La siguiente que sale es la que peor estaba, porque no era una responsabilidad mal colocada: era un condicional que crecía cada vez que se me ocurría una forma nueva de que la invasión disparase.

Capítulo 21

Strategy: cambiar el comportamiento sin tocar el código

El segundo patrón sale de la partida por la puerta que peor estaba. No era una responsabilidad mal colocada —esa fue la fábrica—: era una decisión que crecía cada vez que se me ocurría una idea nueva, y que además dejaba su estado tirado en medio.

21.1 El dolor: tres formas de disparar y un condicional que crece

La invasión dispara. La primera versión elegía tirador al azar, como el arcade, y estaba bien hasta que quise probar si el juego era más divertido de otra manera.

Probé tres cosas: disparar al azar; disparar al invasor más cercano a la columna del cañón, que es cruel; y disparar en ráfaga, concentrando varios turnos en la misma columna. Y esto es lo que salió:

Código 21.1: El condicional después de la tercera idea. La cuarta iba a ir debajo.

```
    def _quiza_disparar(self):
        tiradores = self.formacion.tiradores()
        if self._modo == "azar":
            tirador = tiradores[self._azar.randrange(len(tiradores))]
        elif self._modo == "columna":
            tirador = min(tiradores, key=lambda i: abs(i.caja().x - self.canon.x))
        elif self._modo == "rafaga":
            if self._restantes <= 0 or self._elegido not in tiradores:
                self._elegido = tiradores[self._azar.randrange(len(tiradores))]
                self._restantes = 3
            self._restantes -= 1
            tirador = self._elegido
        ...
```

Inténtalo primero · Antes de que lo cuente

Ese condicional tiene tres ramas y funciona. Di al menos **tres** cosas que estén mal, sin usar la palabra «feo».

Una de ellas no está en el condicional: está dos líneas más arriba.

Las tres que encontré:

Crece con cada idea. Cada forma nueva de disparar es una rama más dentro de la clase que hace correr el tiempo. Es exactamente lo contrario del principio del capítulo 14:

extender obliga a modificar.

El estado de una rama contamina a todas. `_restantes` y `_elegido` solo los usa la ráfaga, y sin embargo son atributos de `Partida`. Dos de las 52 variables del capítulo 2 volviendo por la puerta de atrás.

Y la que está dos líneas más arriba: `self._modo` es una cadena de texto. La cadena `rafaga` sin tilde y la cadena `ráfaga` con ella son dos valores distintos, y basta con teclear la que no es para que el juego se quede sin disparar sin decir nada. Es el entero del capítulo 4 con otro disfraz.

21.2 La estrategia de disparo, sustituible en caliente

Definición 21.1 (Strategy)**.** Una *estrategia* es un objeto que encapsula una manera de hacer algo, de forma que se pueda sustituir por otra sin tocar a quien la usa.

Código 21.2: El contrato: elegir quién dispara. Un método.

```
class EstrategiaDisparo(ABC):
    """Elige que invasor dispara este turno, o ninguno."""

    @abstractmethod
    def elegir(self, tiradores, canon_x, azar) -> Optional[Invasor]:
        ...
```

Y las tres ramas, cada una en su clase. La de la ráfaga se lleva su estado consigo, que era la mitad del problema:

Código 21.3: La ráfaga, con su estado dentro. Ya no son atributos de la partida.

```
class EnRafaga(EstrategiaDisparo):
    """Concentra varios turnos seguidos en la misma columna."""

    __slots__ = ("_restantes", "_elegido", "_largo")
    nombre = "rafaga"

    def elegir(self, tiradores, canon_x, azar):
        if not tiradores:
            return None
        if self._restantes <= 0 or self._elegido not in tiradores:
            self._elegido = tiradores[azar.randrange(len(tiradores))]
            self._restantes = self._largo
        self._restantes -= 1
        return self._elegido
```

Y en la partida queda esto:

```
tirador = self._estrategia.elegir(
```

```
        self.formacion.tiradores(), self.canon.x, self._azar)
```

Sustituir la manera sin cambiar el sitio

En el juego

La partida pide un tirador. No sabe si lo eligió el azar, la cercanía al cañón o una ráfaga en curso.
Una cuarta forma de disparar es una clase nueva. La partida no se abre.

En la vida real

Una tienda cambia su política de precios —rebajas, precio fijo, dinámico— sin cambiar la caja, ni el almacén, ni la forma de cobrar.
La caja pregunta «cuánto vale esto». Quién responde y con qué criterio es otro asunto, y por eso se puede cambiar un lunes.

21.3 El principio abierto/cerrado, hecho código

Este patrón es el capítulo 14 escrito en Python, y merece la pena verlo con la prueba objetiva que aquel capítulo propuso: añadir algo y mirar el diff.

Cuadro 21.1: Añadir una cuarta forma de disparar, antes y después. Contado sobre el código real.

	Con el condicional	Con la estrategia
Archivos nuevos	0	0 (una clase en el mismo)
Archivos que ya funcionaban y hay que abrir	1: la partida	0
Sitios que tocar dentro	2: la rama y sus atributos	1: la clase nueva
Riesgo de romper las tres anteriores	Sí: comparten método	No: son clases distintas

Detengámonos a pensar · Y el efecto que genera: probar sin azar

El contrato recibe el generador de azar en lugar de fabricarlo. Eso permite escribir esto en una prueba:

```
class SiempreElPrimero(EstrategiaDisparo):
    def elegir(self, tiradores, canon_x, azar):
        return tiradores[0] if tiradores else None
```

Una estrategia falsa, de tres líneas, que hace la partida completamente predecible en la parte que más ruido metía.

21.4 Lo que cuesta

Tres estrategias son tres clases, un contrato y una llamada indirecta por turno. Y hay un coste menos visible: para saber qué hace el juego al disparar ya no basta con leer la partida.

Detengámonos a pensar · Cuándo un condicional es mejor que una estrategia

El consejo «sustituye condicionales por polimorfismo» se aplica demasiado.

Con dos ramas que no van a crecer, el condicional gana. Es más corto, se lee de una vez y no obliga a abrir nada.

La frontera que uso: **tres ramas, o dos que ya han cambiado una vez**. Aquí eran tres y con estado propio, y el estado propio es lo que apalanca la decisión: en cuanto una rama necesita recordar algo entre turnos, ese algo va a acabar en la clase de al lado.

Si esto fuera un equipo · La política y la operación

Quien atiende el mostrador no debería tener que conocer la política comercial: debería poder preguntarla.

Cuando las dos cosas viven en la misma cabeza, cambiar la política obliga a reentrenar a todo el mundo, y durante unos días conviven dos versiones.

Con la estrategia fuera, la partida ya no decide cómo dispara la invasión y ha perdido dos de sus atributos.

Lo siguiente que hay que sacar no es una responsabilidad suya, sino algo peor: un conocimiento que el invasor no debería tener y que arrastra desde el capítulo 3.

Capítulo 22

Observer: enterarse sin preguntar

Lo que sale ahora no es una responsabilidad de la partida: es un conocimiento que el invasor nunca debió tener. Y se arrastra desde el capítulo 3, donde lo dejé pasar a propósito.

22.1 El dolor: el invasor tiene que conocer al marcador

Cuando un invasor muere pasan cuatro cosas: sube la puntuación, suena algo, la formación se acelera y, si era el último, empieza otra oleada.

La pregunta es quién avisa a quién. Y la respuesta ingenua se escribe sola:

Código 22.1: Lo que sale si el invasor avisa a quien le interesa. Cada línea es una dependencia nueva.

```
    def recibir_impacto(self):
        ...
        if self._resistencia <= 0:
            self.morir()
            self._marcador.sumar(self.puntos)      # conoce el marcador
            self._sonido.reproducir("explosion")   # conoce el sonido
            self._vista.parpadear(self.caja())     # conoce la vista
            return True
```

Inténtalo primero · Cuenta el daño

Ese invasor conoce tres cosas más de las que conocía. Antes de seguir:
¿Cuántos parámetros necesita ahora su constructor? ¿Y quién tiene que proporcionárselos, sabiendo que se crean 55 de golpe en una fábrica?
Para escribir una prueba de que un impacto lo mata, *¿qué hay que montar antes?*

Tres parámetros más, que la fábrica tiene que recibir y pasar a los 55. **La fábrica pasa a conocer el marcador, el sonido y la vista**, sin usarlos para nada. Y para probar que un impacto mata a un invasor **hay que construir un marcador, un reproductor de sonido y una vista**.

Detengámonos a pensar · La dirección de la dependencia está al revés

El invasor es la pieza más central del juego y acaba dependiendo de la más accesoria, que es la que dibuja.
Es exactamente lo que el capítulo 17 decía que no puede ser: lo importante dependiendo de lo accesorio. Y el síntoma clásico: **cuando probar algo pequeño obliga a montar medio programa, la dependencia va en el sentido equivocado.**

22.2 Quien lo publica no sabe quién lo escucha

Definición 22.1 (Observer)**.** Un *observador* es alguien que se apunta para enterarse de que ha pasado algo. Quien lo publica no sabe quién escucha, ni cuántos, ni si hay alguno.

Código 22.2: El publicador. No sabe quién escucha, y ese es todo el punto.

```
class Sucesos:
    """Publicador. No sabe quien escucha, y ese es todo el punto."""

    __slots__ = ("_oyentes",)

    def suscribir(self, suceso, oyente):
        self._oyentes.setdefault(suceso, []).append(oyente)

    def publicar(self, suceso, **datos):
        for oyente in self._oyentes.get(suceso, ()):
            oyente(**datos)
```

Y quien publica lo hace en una línea, sin conocer a nadie:

```
self.sucesos.publicar(Suceso.INVASOR_ABATIDO,
                      puntos=self.puntuacion, tipo=invasor.tipo)
```

Publicar sin conocer al público

En el juego

El invasor muere. La partida publica que ha muerto. El marcador, el sonido y lo que haga falta se enteran si estaban apuntados.
Añadir un cuarto interesado —por ejemplo, guardar la partida— no toca ni al invasor ni a la partida: se apunta y ya está.

En la vida real

Un letrero de «se vende». Quien publica anuncia una vez y no sabe cuántos leen, ni quiénes.
Y por eso alguien puede decidir ignorar el letrero sin que quien lo publicó se entere ni tenga que cambiar nada.

Detengámonos a pensar · Quién publica aquí, y por qué no es el invasor

Decisión: **el que publica es la partida, no el invasor**.
Podría publicar el invasor: sabe que ha muerto. Pero entonces necesitaría el publicador entre sus atributos, y volveríamos a un invasor con un parámetro más que la fábrica tiene que repartir entre 55.
La partida ya tiene el publicador y ya ve el impacto entero —lo decidimos en el capítulo 5—. Publicar desde ahí no le añade ninguna dependencia: le añade una línea.
Regla: publica quien ya estaba mirando, no quien protagoniza.

22.3 El precio: un flujo que ya no se lee de arriba abajo

Y ahora lo que cuesta, que en este patrón es más de lo que suele admitirse.

Cuadro 22.1: El balance del observador. La segunda fila de la derecha es la que duele.

Se gana	Se paga
El invasor no conoce a nadie	Para saber qué pasa al morir un invasor hay que buscar quién se suscribió
Un interesado nuevo no toca nada	El orden en que se atiende a los oyentes es el orden en que se apuntaron: implícito
Se puede probar un impacto sin montar nada	Si un oyente falla, falla en medio de la partida y el rastro no dice de dónde venía
La raíz de composición decide quién escucha	Un oyente que se apunta y no se da de baja es una fuga

Detengámonos a pensar · La fuga que este patrón provoca, y que aquí no ocurre

La cuarta fila es el fallo típico del observador: se suscribe un oyente por partida, nadie lo da de baja, y tras cien partidas hay cien oyentes recibiendo sucesos de una partida que ya no existe.

Es la clase de fallo que no se ve en una prueba corta y arruina una sesión larga, que es justo lo que el capítulo 18 decía que hay que garantizar.

En este juego **no ocurre**, y no lo digo por confianza: está medido. La tabla de estabilidad de aquel capítulo son 21 partidas encadenadas con la memoria plana. Si los oyentes se acumularan, se vería ahí.

La razón es de diseño: cada partida crea su propio `Sucesos` y se lo lleva al morir. Nadie se suscribe a un publicador que sobreviva a la partida. **Es la misma decisión que el capítulo 25 va a discutir, tomada al revés.**

Si esto fuera un equipo · Enterarse por lista o por llamada

Un jefe que avisa uno por uno de cada decisión funciona con cuatro personas. Con cuarenta se convierte en su trabajo, y siempre se olvida de alguien.

Publicar en un sitio donde se apunta quien quiera escala, y tiene el mismo precio que en el código: nadie sabe con certeza quién se ha enterado.

Y también tiene la misma fuga: listas de distribución con gente que ya no está en el proyecto y sigue recibiéndolo todo.

Compruébalo · Cuenta tus suscripciones

Si usas eventos, señales o callbacks en algún programa, busca dónde se suscribe cada oyente y dónde se da de baja.

Cuenta las dos cosas. Si hay más suscripciones que bajas y el objeto que publica vive mucho, ahí tienes una fuga esperando.

Y si no puedes contarlo porque las suscripciones están repartidas, ese es un hallazgo por sí solo.

Con el observador, el invasor vuelve a no conocer a nadie y la partida pierde la responsabilidad de avisar.

Lo siguiente que aparece no viene de dentro del juego, sino de fuera: alguien tiene que jugar, y no todos los que juegan hablan el mismo idioma.

Capítulo 23

Adapter: hablar con lo que no se diseñó para ti

Los cuatro patrones anteriores ordenaban lo que pasa *dentro* del juego. Este es el primero que se ocupa de la frontera: algo de fuera tiene que poder hablarle, y lo de fuera no habla su idioma.

23.1 El dolor: cada mando habla su idioma

El juego se maneja con el teclado. Y en cuanto pensé en probarlo en otro sitio, apareció el problema: un mando de consola no da letras, da códigos de botón; una pantalla táctil no da ninguna de las dos cosas, da coordenadas.

Si el juego los conoce a los tres, cada aparato nuevo lo abre:

Código 23.1: El control sabiendo de teclas, de botones y de toques. Tres cosas que no son suyas.

```
    def _aplicar(self, entrada):
        if isinstance(entrada, str):            # teclado
            if entrada in ("a", "h"):
                self._partida.mover_canon(-1)
            ...
        elif isinstance(entrada, int):          # mando
            if entrada == 14:
                self._partida.mover_canon(-1)
            ...
        elif isinstance(entrada, tuple):        # tactil
            ...
```

Un aparato nuevo abre el control. Otra vez lo contrario del capítulo 14.

Vuelven los `isinstance`. El capítulo 10 midió que en el juego no había ninguno. Aquí entrarían tres de una vez, y además preguntando por tipos de Python, no por cosas del juego.

Y el juego pasa a saber que existe un teclado. Que es la definición exacta de que lo importante dependa de lo accesorio.

23.2 Una sola puerta de entrada

Definición 23.1 (Adapter)**.** Un *adaptador* traduce lo que habla una pieza ajena al idioma que entiende la tuya. Es la única pieza que conoce los dos lados.

Primero se declara el idioma del juego, que son cinco palabras:

Código 23.2: Lo único que el juego entiende. No hay teclas aquí.

```
class Orden(Enum):
    """El unico idioma que el juego entiende."""

    IZQUIERDA = auto()
    DERECHA = auto()
    DISPARAR = auto()
    SALIR = auto()
    NADA = auto()
```

Y después, un traductor por aparato. Cada uno sabe de lo suyo y de nada más:

Código 23.3: Tres adaptadores. Ninguno sabe qué es un invasor.

```
class AdaptadorTeclado:
    MAPA = {"a": Orden.IZQUIERDA, "d": Orden.DERECHA,
            " ": Orden.DISPARAR, "q": Orden.SALIR}

    def traducir(self, tecla):
        return self.MAPA.get(tecla.lower(), Orden.NADA)


class AdaptadorMando:
    MAPA = {14: Orden.IZQUIERDA, 15: Orden.DERECHA,
            0: Orden.DISPARAR, 9: Orden.SALIR}

    def traducir(self, boton):
        return self.MAPA.get(boton, Orden.NADA)
```

Un traductor en la puerta

En el juego

El control recibe órdenes. No sabe si vinieron de una tecla, de un botón o de un dedo.
Un aparato nuevo es un adaptador nuevo. El juego no se abre, y los tres adaptadores anteriores tampoco.

En la vida real

Un adaptador de enchufe permite usar un aparato en un país con otra toma. No cambia el aparato ni cambia la instalación: **se pone en medio**.

Detengámonos a pensar · Y aquí hay un cuarto adaptador que no parece uno

El jugador automático —el que juega 500 partidas para que se puedan medir— devuelve órdenes del mismo tipo que el teclado.
Es un adaptador: traduce «lo que decidiría un jugador» al idioma del juego, igual que el teclado traduce «lo que pulsó una persona».
Y por eso el control no distingue entre una partida jugada por alguien y una jugada por el autómata, que es lo que hace posible medir.

23.3 Dónde acaba el adaptador y empieza el remiendo

La frontera, porque este patrón se usa para tapar cosas.

Detengámonos a pensar · Un adaptador traduce. No decide, no arregla, no recuerda

Tres señales de que lo que has escrito ya no es un adaptador:

Decide. Si el traductor del teclado empieza a mirar si el cañón puede moverse antes de devolver la orden, ha dejado de traducir: está jugando.

Recuerda. Si guarda estado entre llamadas —la última tecla, un contador—, tiene lógica propia y hay que preguntarse de quién es.

Arregla. Si compensa un fallo de la pieza ajena —«este mando manda el botón dos veces, ignoro la segunda»—, eso es un remiendo. Puede ser necesario y hay que llamarlo por su nombre, con un comentario que diga qué fallo compensa; si no, dentro de un año nadie sabrá por qué está ahí.

El de este juego cumple las tres: un diccionario y un `get`. Cuatro líneas.

Si esto fuera un equipo · Traducir sin decidir

Quien hace de puente entre dos equipos que no hablan igual añade valor mientras traduce. En cuanto empieza a decidir qué mensajes pasa y cuáles no, deja de ser un puente y se convierte en un filtro. Y entonces los dos equipos dependen de su criterio sin saberlo, que es lo mismo que un adaptador con lógica propia escondida.

Compruébalo · Busca lo ajeno en tu código

Busca en tu programa los sitios que conocen algo externo: un formato de archivo, una biblioteca, un protocolo, una tecla.

Cuenta cuántos son. Si el mismo conocimiento externo aparece en más de un sitio, el adaptador que falta ya está escrito, solo que repartido.

Con la entrada adaptada, el juego no sabe con qué se juega.

Lo que sigue es lo contrario: no lo que entra, sino lo que hay que sacar y guardar en algún sitio cuando la partida termina.

Capítulo 24

Repository: guardar y recuperar

Y ahora la frontera del otro lado. El capítulo anterior resolvió lo que entra; este, lo que hay que sacar y dejar en algún sitio cuando la partida termina.

24.1 El dolor: los marcadores atan el juego a un archivo

Al terminar una partida hay que guardar la puntuación si está entre las diez mejores. Es la funcionalidad más pequeña del libro.

Código 24.1: El juego sabiendo de archivos, de JSON y de rutas. Tres cosas que no son del juego.

```
    def _terminar(self, desenlace):
        self.desenlace = desenlace
        tabla = []
        if os.path.exists("marcadores.json"):
            with open("marcadores.json", encoding="utf-8") as f:
                tabla = json.load(f)
        tabla.append({"puntos": self.puntuacion, "oleada": self.oleada})
        tabla.sort(key=lambda m: m["puntos"], reverse=True)
        with open("marcadores.json", "w", encoding="utf-8") as f:
            json.dump(tabla[:10], f)
```

Inténtalo primero · El problema no es el que parece

Ese código funciona. Y hay una cosa que se rompe y que no tiene nada que ver con el diseño.

Pista: piensa en las 500 partidas que el libro ejecuta para medir.

Lo que se rompe es esto: **cada una de las 500 partidas de medición escribe en el disco**. Quinientas lecturas, quinientas escrituras y un archivo de marcadores que aparece en el directorio cada vez que se mide algo.

Y hay más, en orden de gravedad creciente:

Las pruebas ensucian. Una prueba que compruebe el final de partida deja un archivo. Dos pruebas seguidas se pisan.

No se puede probar el guardado sin disco. Para comprobar que la tabla se ordena bien hay que escribir y volver a leer un archivo de verdad.

Y el juego depende de que exista un sistema de archivos escribible, que es una suposición enorme para un programa tan sencillo.

24.2 Pedir sin saber dónde está guardado

Definición 24.1 (Repository)**.** Un *repositorio* es un contrato para guardar y recuperar, escrito con el vocabulario del problema y no con el del almacén.

Código 24.2: Dos verbos, y ni una palabra sobre dónde se guarda.

```python
class RepositorioMarcadores(ABC):
    """El contrato. Dos verbos, y ni una palabra sobre donde se guarda."""

    @abstractmethod
    def mejores(self) -> List[Dict]: ...

    @abstractmethod
    def anotar(self, puntos: int, oleada: int) -> List[Dict]: ...
```

Fíjate en el vocabulario, porque es lo que distingue este patrón de meter una interfaz por medio: dice `anotar` y `mejores`, no `escribir` ni `leer_json`. Es el capítulo 17 en su forma más literal: **el contrato lo define el juego, con sus palabras**.

Y hay dos implementaciones:

Cuadro 24.1: Las dos implementaciones y para qué sirve cada una.

Clase	Dónde guarda	Quién la usa
`EnArchivo`	Un JSON al lado del juego	Quien juega, con `-marcadores`
`EnMemoria`	Una lista, y se pierde	Las pruebas y las 500 partidas de medición

Detengámonos a pensar · La segunda implementación es la que justifica el contrato

El capítulo 17 dejó una regla: no inviertas hasta que aparezca la segunda implementación, o hasta que la primera te impida probar.

Aquí se cumplieron las dos a la vez. `EnMemoria` existe porque sin ella no se puede medir sin ensuciar, y **una necesidad de prueba es una segunda implementación legítima**, no una excusa para añadir una interfaz.

Pedir sin saber dónde está

En el juego

La partida dice «anota esto». No sabe si va a un archivo, a una base de datos, a la nube o a ninguna parte.
Y por eso las 500 partidas de medición no tocan el disco: se les pasa el repositorio de memoria y ya está.

En la vida real

Pides un expediente al archivo de la oficina. No te importa en qué armario está, ni si lo han digitalizado.
Y por eso pueden cambiar los armarios sin avisarte. El día que la oficina pasa a digital, tú sigues pidiendo el expediente igual.

24.3 Lo que este juego apenas necesita, y por qué se cuenta igual

Este juego guarda diez números. No es un problema de persistencia: es una lista corta en un archivo pequeño. Un repositorio con su contrato abstracto y sus dos implementaciones para eso puede parecer desproporcionado, y en cierto modo lo es.

Detengámonos a pensar · Por qué está en el libro de todas formas

Porque el problema de verdad no era guardar, era probar. Sin `EnMemoria` no hay 500 partidas limpias. Eso no es desproporción: es la razón.
Y porque el coste está medido y es pequeño. Sacarlo del juego costó, medido, **un archivo nuevo y un solo sitio** fuera de la raíz de composición. De los tres cambios comparados, es el que mejor sale.
Dicho lo cual: si tu programa guarda diez números y no necesitas probarlo, un repositorio sobra.

Si esto fuera un equipo · El archivo y el almacén

Un procedimiento que dice «toma la carpeta azul del segundo estante» deja de funcionar el día que reorganizan el archivo.
Uno que dice «pide el expediente» sobrevive a la reorganización, y además sobrevive a que el archivo pase a ser digital.
La diferencia es la misma que entre `leer_json` y `mejores`: uno habla del almacén, el otro de lo que se quiere.

Compruébalo · Busca el vocabulario del almacén

En tu programa, busca nombres de métodos que mencionen la tecnología: JSON, SQL, fichero, endpoint, caché.
Cada uno es un sitio donde el vocabulario del almacén se ha colado en el de tu problema. No todos hay que cambiarlos —dentro de la implementación están en su casa—. Los que importan son los que aparecen en la parte del programa que resuelve tu problema.

Con el repositorio, el juego no sabe que existe un disco.

Lo que viene ahora es el patrón que más veces se enseña.

Capítulo 25

Singleton: uno solo, y lo que cuesta

Enseñar solo lo que uno recomienda produce lectores que reconocen las buenas soluciones y no reconocen las malas. Y este patrón lo incluyo para desaconsejarlo.

25.1 El dolor: dos relojes en la misma partida

El juego necesita un reloj. Para medir cuánto tarda un turno, para acelerar la formación con el tiempo real en lugar de con turnos, para lo que sea.

Y un reloj parece la cosa más obvia del mundo que debe ser única. ¿Dos relojes en un juego? Sería absurdo: darían horas distintas.

Detengámonos a pensar · El razonamiento que lleva al patrón

Solo debe haber un reloj. Si cualquiera puede crear uno, alguien creará dos. Y si hay dos, el juego medirá el tiempo de dos maneras. Luego hay que impedir que se pueda crear más de uno.

Las tres primeras son ciertas. La cuarta **no se sigue**, y ahí está toda la trampa.

De «debe haber uno» no se deduce «la clase debe impedir que haya dos». Se deduce que **alguien** tiene que crear uno solo y repartirlo. Y ese alguien ya existe: es la raíz de composición del capítulo 17.

25.2 Cómo se hace

Definición 25.1 (Singleton)**.** Un *singleton* es una clase que impide que exista más de un objeto suyo: en lugar de crear uno, se pide el que hay. La garantía la impone la clase, no quien monta el programa.

Código 25.1: El patrón, tal cual. Se conserva en el juego porque este capítulo lo comenta.

```
class RelojSingleton:
    """El patron, tal cual. Se conserva porque el capitulo lo comenta."""

    _instancia = None

    def __new__(cls):
        if cls._instancia is None:
            cls._instancia = super().__new__(cls)
            cls._instancia._inicio = time.monotonic()
        return cls._instancia

```

```
    def transcurrido(self):
        return time.monotonic() - self._inicio
```

`RelojSingleton()` devuelve siempre el mismo objeto. Nadie puede crear dos aunque quiera. Funciona.

25.3 Por qué es el patrón más criticado, y con razón

Cuatro problemas, en orden de gravedad creciente.

Uno: es una variable global con corbata. Cualquier parte del programa puede llegar a ella escribiendo su nombre. Todo lo que el capítulo 7 defendió sobre trazar fronteras se salta con una línea, y sin que aparezca en ningún constructor: la dependencia existe y no está declarada en ninguna parte.

Dos: no se puede probar. Para comprobar qué hace el juego después de dos horas hay que esperar dos horas. Con un reloj recibido desde fuera se le pasa uno falso y se adelanta la hora a mano. Con el singleton, no hay por dónde.

Tres: el estado sobrevive entre pruebas. La instancia se guarda en la clase, así que la primera prueba que la toque deja el reloj arrancado para la segunda. Aparecen fallos que dependen del orden en que se ejecutan las pruebas, que son de los peores que hay.

Y cuatro, el que me llevó a decidir removerlo:

Detengámonos a pensar · No pude comparar dos partidas

El capítulo 32 necesita ejecutar dos versiones del juego y comparar lo que tardan. Y el capítulo 18 encadena 33 partidas seguidas.

Con un reloj único, las dos partidas comparten reloj. La segunda arranca con el tiempo de la primera ya corriendo, y las medidas no significan nada.

No es un problema teórico de diseño. Es que **el patrón me impidió medir**.

Dos maneras de garantizar que hay un solo reloj

Impedir que haya dos

La clase se niega a crear el segundo. Nadie puede equivocarse. Y nadie puede tener dos aunque los necesite: ni las pruebas, ni una comparación, ni dos partidas a la vez.

La garantía la da la clase, y se la impone a todo el mundo.

Crear uno y repartirlo

La raíz de composición crea uno y se lo pasa a quien lo necesite.

Sigue habiendo uno solo, porque solo se crea uno. Y quien necesite otro —una prueba, una comparación— puede.

La garantía la da quien monta el programa, que es quien tiene la información para decidirlo.

25.4 Qué usar en su lugar casi siempre

Lo que hay en el juego:

Código 25.2: El reloj que sí uso. Y su doble de prueba, que es la mitad del motivo.

```
class Reloj:
    """El que uso. Uno por partida, recibido por el constructor."""

    __slots__ = ("_inicio", "_fuente")

    def __init__(self, fuente=time.monotonic):
        self._fuente = fuente
        self._inicio = fuente()

    def transcurrido(self):
        return self._fuente() - self._inicio


class RelojFalso:
    """Doble de prueba: el tiempo avanza cuando yo lo digo."""

    __slots__ = ("ahora",)

    def __call__(self):
        return self.ahora

    def avanzar(self, segundos):
        self.ahora += segundos
```

Sigue habiendo un reloj por partida. Lo que ha desaparecido es la *prohibición*, y con ella los cuatro problemas.

Detengámonos a pensar · Y el hallazgo que este patrón dejó en el propio libro

El capítulo 7 mide cuántas veces alguien toca por fuera un atributo privado de otra clase. Sobre las 1 737 líneas del juego, la respuesta es **dos**.

La primera está aquí. Es esta línea:

```
cls._instancia._inicio = time.monotonic()
```

La clase interviniendo en su propia instancia por una vía que no es `self`, porque `__new__` no tiene `self` todavía.

De las dos líneas que el detector señala en todo el juego, **una está en el único patrón que el libro desaconseja**.

No la he arreglado. La dejo, medida y señalada, porque un patrón que obliga a escribir esa clase de línea se está explicando solo.

25.5 Cuándo sí

Para no dejarlo en «nunca», que tampoco es un criterio.

Detengámonos a pensar · Las dos condiciones

Que sea de verdad único por naturaleza y no por conveniencia. No «solo necesito uno»: que dos sean imposibles. Un registro que el sistema operativo impone, por ejemplo.

Y que no tenga estado que las pruebas necesiten controlar. Un objeto sin estado —una tabla de constantes, un conversor— no arrastra nada entre pruebas, y entonces la mayor parte del daño desaparece.

Un reloj falla las dos: dos relojes son perfectamente posibles, y su estado es justo lo que hay que controlar para probar. Por eso es el mal ejemplo perfecto.

Si esto fuera un equipo · La llave única de la sala

Una sola llave garantiza que nadie entra sin permiso. Y garantiza también que cuando más personas la necesitan a la vez, una espera.

Lo que suele pasar entonces es lo que pasa en el código: alguien hace una copia por su cuenta, y ahora hay dos llaves y nadie sabe cuántas.

La alternativa no es repartir llaves a todos: es que alguien lleve el control de a quién se le da. Que es exactamente la raíz de composición.

Compruébalo · Busca tus singletons

En un programa tuyo, busca clases con un atributo de clase que guarde una instancia, o funciones tipo `obtener_instancia()`, o variables de módulo que guarden un objeto vivo. Para cada una, hazte una sola pregunta: *¿podría escribir una prueba que la reemplace por una falsa?*

Si la respuesta es no, ese objeto está fijando el comportamiento de todo lo que lo use, y lo estará fijando también en tus pruebas.

Queda un patrón antes de cerrar la parte, y es el que hace posible todo lo que este capítulo echaba de menos: pasar las piezas en lugar de fabricarlas.

Capítulo 26

Inyección de dependencias: recibir en vez de fabricar

El capítulo anterior terminó echando de menos algo: poder pasarle al juego un reloj falso, una estrategia predecible, un repositorio que no toque el disco.

26.1 El dolor: cada habilidad y cada azar, fabricados dentro

Dos casos, y parecen distintos hasta que se miran juntos.

El azar. La primera versión creaba su generador dentro:

Código 26.1: El azar fabricado dentro. La partida es irrepetible.

```
class Partida:
    def __init__(self):
        self._azar = random.Random()      # sin semilla: cada vez otra cosa
```

Con eso, dos ejecuciones del mismo juego dan partidas distintas. No se puede repetir una para verla otra vez, no se puede comparar dos versiones sobre la misma partida, y no se puede escribir una prueba que dependa de lo que hace la invasión.

Sin esto, la mitad de las cifras de este libro no existirían.

Las habilidades. El mismo problema con otra cara. Si el invasor se fabrica su escudo:

```
class Invasor:
    def __init__(self, tipo, fila, columna, caja):
        self._habilidad = Escudo(1) if fila == 0 else SIN_HABILIDAD
```

...entonces el invasor sabe que la fila de arriba lleva escudo. Que es una regla de dificultad, no un asunto del invasor, y que además cambia con la oleada.

Inténtalo primero · Une los dos casos

El azar y el escudo parecen problemas distintos: uno es de reproducibilidad y el otro de responsabilidades.

Antes de seguir: ¿qué tienen en común, exactamente? Escríbelo en una frase.

Lo que tienen en común: **el objeto decide algo que no le corresponde, y lo decide en el peor sitio posible, que es dentro de sí mismo, donde nadie puede cambiarlo.**

26.2 Otorgar habilidades desde fuera: al agresor, al defensor y al búnker

Definición 26.1 (Inyección de dependencias)**.** Un objeto *recibe* las piezas que necesita en lugar de fabricarlas. Quien lo construye decide con qué se monta.

Y aquí se ve la ganancia que no esperaba: la misma habilidad sirve a los tres bandos, porque ninguno se la fabrica.

Código 26.2: Tres portadores, una habilidad, cero subclases. La explosión del capítulo 9, resuelta.

```
Invasor(TipoInvasor.CALAMAR, 0, 3, Caja(11, 1), habilidad=Escudo(1))
Canon(Caja(22, 22), vidas=3, habilidad=Escudo(2))
Bunker(Caja(5, 19, 4, 1), 3, habilidad=Regeneracion(cada=120))
```

Un agresor con escudo, un defensor con escudo y un obstáculo que se repara. Ninguna clase nueva. Y quien decide quién lleva qué es la fábrica, que es quien sabe de dificultad.

⚖ Recibir en vez de fabricar

En el juego

El invasor no sabe si lleva escudo hasta que se lo dan. Y por eso el mismo invasor puede llevarlo en la oleada 5 y no en la 4, sin que su clase cambie.
La decisión está donde vive el conocimiento: en la fábrica.

En la vida real

Un puesto no se compra su propio material: se lo suministran.
Y por eso se puede cambiar de proveedor, dar un equipo mejor a quien lo necesita, o montar un puesto de prueba con material prestado.
Si cada persona comprara lo suyo, no habría manera de cambiar nada sin hablar con todas.

26.3 La semilla que entra desde fuera

El otro caso:

```
class Partida:
    def __init__(self, semilla=0, ...):
        self._azar = random.Random(semilla)
```

Una línea. Y con ella:

Cuadro 26.1: Lo que la semilla recibida desde fuera hizo posible en este libro. Ninguna de las cuatro se podría haber hecho sin ella.

Qué	Dónde
Repetir una partida exacta	`-semilla 7` muestra siempre lo mismo
Comparar dos versiones sobre las mismas partidas	Capítulos 2 y 32
500 partidas reproducibles	Las tandas de todo el libro
21 partidas encadenadas para medir estabilidad	Capítulo 18

Detengámonos a pensar · Determinismo no es lo contrario de azar

El juego sigue siendo aleatorio: la invasión dispara donde no esperas y el OVNI aparece cuando le parece. Lo que cambia es que **esa aleatoriedad es repetible**.

Es la diferencia entre no saber qué va a pasar y no poder volver a verlo. La primera es una propiedad del juego y hace falta. La segunda es una propiedad del programa y no hace ninguna falta.

26.4 Dobles de prueba: probar el comportamiento, no la suerte

Y de ahí sale lo que hace que las pruebas de este juego existan.

Código 26.3: Tres dobles. Son implementaciones legítimas de contratos que ya existían.

```
EnMemoria()                       # repositorio que no toca el disco
RelojFalso()                      # el tiempo avanza cuando yo lo digo
class SiempreElPrimero(EstrategiaDisparo):   # sin azar
    def elegir(self, tiradores, canon_x, azar):
        return tiradores[0] if tiradores else None
```

Detengámonos a pensar · Un doble no es una trampa, y la prueba es esta

Se oye que probar con dobles es probar otra cosa. A veces es verdad, y hay una forma de distinguirlo.

Un doble legítimo cumple el mismo contrato. `EnMemoria` y `EnArchivo` implementan `RepositorioMarcadores`: si una prueba pasa con uno y falla con el otro, el fallo está en la implementación, no en el doble.

Un doble tramposo cumple el contrato *y algo más*. Si el falso devolviera los marcadores ya ordenados sin ordenarlos, la prueba pasaría midiendo la mentira.

De ahí la regla del capítulo 15, otra vez: un doble es un sustituto, y vale lo mismo que valga su sustituibilidad.

26.5 La técnica y el principio: qué la separa del capítulo 17

El capítulo 17 dice de qué debes depender. Este dice cómo te llega.

Se puede cumplir el principio sin inyectar nada, y se puede inyectar sin invertir nada —recibir por el constructor una clase concreta que sabe de JSON—. La prueba para distinguirlos está allí.

26.6 Lo que esto le regala al capítulo 31

Cierro con lo que viene, porque este patrón es su condición previa.

El capítulo 31 tiene que comprobar un juego cuyo espacio de estados no cabe. No puede recorrerlo completo, así que hace tres cosas distintas: fijar el azar para repetir, comprobar propiedades en lugar de casos, y exigir invariantes.

Las tres dependen de haber recibido las piezas desde fuera. Sin la semilla inyectada no hay repetición; sin estrategias sustituibles no hay casos controlados; sin repositorio en memoria no hay tandas limpias.

Si esto fuera un equipo · Dar los medios o pedir que se los busquen

Un encargo que llega con los medios asegurados —acceso, presupuesto, información— se puede evaluar por su resultado.

Uno donde cada cual se busca la vida produce resultados que dependen de lo que cada persona pudo conseguir, y entonces no se está midiendo el trabajo: se está midiendo el acceso.

Es lo mismo que un objeto que se fabrica sus dependencias: funciona, y no se puede comparar con nada.

Compruébalo · Busca lo que tu código fabrica dentro

Busca en tu programa las creaciones de objetos dentro de otros: llamadas a constructores, aperturas de archivo, conexiones, generadores de azar, lecturas del reloj.

Cada una es algo que quien usa esa clase no puede cambiar.

Y para cada una, la pregunta es: *¿podría escribir una prueba de esta clase sin que ocurra eso?* Si no, ya sabes cuál es el siguiente parámetro de tu constructor.

Queda un patrón, y es el que pone en su sitio la última cosa que la partida todavía no ha soltado: la frontera entre lo que el juego es y cómo se ve.

Capítulo 27

MVC: lo que el juego es y cómo se ve

Último patrón de diseño, y el que pone en su sitio lo que el capítulo 1 anunció como la diferencia grande con el libro anterior: aquí hay algo que dibujar.

27.1 El dolor: el juego sabe dibujarse

La versión de una sola pieza dibujaba dentro del bucle. Ochenta líneas de lienzo, caracteres y códigos de terminal, entre el movimiento de la formación y el disparo del rival.

Y eso produce cuatro cosas:

Para medir hay que dibujar. Las 500 partidas de cualquier tanda llenaban la pantalla de basura, y buena parte del tiempo se iba en imprimir.

El juego no funciona sin terminal. Los códigos de escape suponen uno.

Cambiar cómo se ve obliga a abrir las reglas. Y al revés: al tocar una regla se pasa por el dibujado.

Y no se puede tener dos vistas. Ni la muda para medir, ni una gráfica, ni ninguna.

27.2 Modelo, vista y control, elemento por elemento

Definición 27.1 (Modelo, vista y control)**.** El *modelo* es lo que el programa es: su estado y sus reglas. La *vista* lo muestra y no decide nada. El *control* traduce lo que llega de fuera en órdenes al modelo. Ninguno de los tres hace el trabajo de otro.

En este juego, repartido:

Cuadro 27.1: Los tres papeles y qué cae en cada uno. Las líneas están medidas, no estimadas.

Papel	Quién	Qué sabe
Modelo	Las entidades y las estrategias	Las reglas. Ni un `print`
Vista	`VistaConsola`, 39 líneas	Dibujar. No decide nada
Control	`Control`, 41 líneas	Traducir órdenes y hacer girar el turno

Y el control entero cabe en la página:

Código 27.1: Todo el control. No dibuja y no decide reglas: coordina.

```
1 class Control:
```

```
    def ejecutar(self):
        p = self._partida
        while p.en_juego and p.turno < self._turnos_maximos:
            for orden in self._jugador.decidir(p):
                self._aplicar(orden)
            p.avanzar()
            self._vista.dibujar(p)
        ...

    def _aplicar(self, orden):
        if orden is Orden.IZQUIERDA:
            self._partida.mover_canon(-1)
        elif orden is Orden.DERECHA:
            self._partida.mover_canon(1)
        elif orden is Orden.DISPARAR:
            self._partida.disparar()
```

Tres oficios que no se pisan

En el juego

El modelo no sabe dibujar. La vista no sabe qué es un invasor: pide una caja y un símbolo. El control no conoce ninguna regla: traduce.
Y por eso existe `VistaMuda`, que no dibuja nada, y el juego no nota la diferencia.

En la vida real

La cocina, el camarero y el comedor de un restaurante.
La cocina no atiende mesas. El camarero no decide la receta. Y el comedor puede reformarse sin cambiar la carta.
Cuando el cocinero sale a atender, funciona —esa noche—. Lo que se pierde es poder cambiar una de las tres cosas sin cerrar.

27.3 La misma partida en consola y en pantalla

La prueba de que la separación es real no es que quede bonita: es que existan dos vistas y el modelo no se entere.

Código 27.2: La vista muda. Existe para poder medir, y es la prueba de que el modelo no depende de ninguna.

```
class VistaMuda:
    """No dibuja. Para medir y para probar."""

    def dibujar(self, partida) -> None:
        pass

    def cerrar(self) -> None:
        pass
```

27.4 Dónde MVC se queda corto

Y ahora los límites, que en este patrón son más de los que se cuentan.

La frontera control-modelo es discutible. ¿Quién decide cuándo termina la partida: el modelo, que sabe las reglas, o el control, que lleva el bucle? Aquí lo decide el modelo y el control solo pregunta `en_juego`. Es defendible y no es la única opción defendible.

En un juego de verdad la vista necesita más. Animaciones, interpolación entre fotogramas, efectos que duran varios turnos. Nada de eso cabe en «pide caja y símbolo», y en cuanto aparezca, la vista necesitará estado propio y este reparto se quedará pequeño.

Detengámonos a pensar · Y aquí topa este libro con su propia pantalla

Es el sitio donde se ve el precio de la decisión del capítulo 1: la interfaz de este juego es deliberadamente la más sencilla que se puede tener, caracteres de texto sobre el terminal, para que nada compita con la pregunta de quién sabe qué.

Esa decisión ha salido barata durante veintisiete capítulos, y aquí se acaba. `VistaConsola` pide una caja y un símbolo porque no tiene nada más que pedir. Una vista que tuviera que animar una explosión durante ocho fotogramas necesitaría recordar en qué fotograma va, y en cuanto una vista recuerda algo hay que volver a hacerse la pregunta de este libro sobre ella: ¿ese recuerdo es del juego o del dibujo? La respuesta no es evidente, y contestarla mal es como se acaba metiendo la lógica del juego dentro de la pantalla.

No la voy a contestar aquí. **La interfaz visual es otro libro de esta serie**, y ahí es donde toca: cómo se construye una, cómo se reparte entre lo que se dibuja y lo que se decide, y qué está bien hecho al construirla. Este libro llega hasta la frontera y la deja trazada, que es exactamente lo que tenía que hacer.

Y el nombre es de tres letras para tres cosas que casi nunca son tres. Con dos vistas, dos controles y un modelo, «MVC» describe la idea y no la estructura.

Detengámonos a pensar · Lo que sí se lleva uno de aquí

Una sola frase, y no menciona las tres letras:

Lo que el programa *es* no debe saber cómo se muestra.

Todo lo demás —cuántas capas, cómo se llaman, quién lleva el bucle— depende del programa.

Si esto fuera un equipo · Quien decide y quien informa

Un equipo que decide y además prepara la presentación de lo que decidió acaba decidiendo en función de lo que se presenta bien.

Separar las dos cosas no es burocracia: es lo que permite que la decisión se tome por sus motivos y se explique después.

Y el límite es el mismo que en el código: si quien informa no entiende nada de lo que decide, informará mal. La separación no es un muro.

Prompt · Separar sin inventar capas

```
Te paso un programa que mezcla lógica y presentación: [pégalo].

No lo refactorices todavía. Antes:

1. Marca cada línea como MODELO (reglas y estado), VISTA (mostrar) o CONTROL
   (traducir lo que llega de fuera). Si alguna no encaja en ninguna, dímelo:
   esas son las interesantes.

2. Cuestióname: ¿hay alguna decisión que hoy tome la vista y que en realidad
   sea una regla? Señálamela y espera mi respuesta.

3. Dime qué haría falta para escribir una segunda vista que no dibuje nada.
   Si la respuesta es "nada", la separación ya está hecha.

No propongas más de tres capas. Si crees que hacen falta más, explícame por qué
antes de proponerlas.
```

Con esto se cierra la Parte VI y con ella los ocho patrones. El juego está repartido, afinado y medido, y una sesión de 21 partidas encadenadas deja la memoria plana. En la próxima parte del libro analizaremos la inclusión del jugador humano, solicitaremos algunos cambios en las mecánicas del juego y validaremos la robustez de la arquitectura que hemos implementado.

Y queda pendiente la factura. Los ocho patrones han añadido objetos, indirecciones y llamadas, y el capítulo 19 había ahorrado memoria. Anuncié en el capítulo 18 que esas dos mitades tiraban en direcciones opuestas y que se saldarían con cifras.

La parte siguiente las pone encima de la mesa. Y empieza por lo más incómodo: comprobar que todo esto funciona, en un juego que no se puede recorrer entero.

VII

El jugador humano entra en el juego

Capítulo 28

El jugador humano

Hasta este capítulo el juego se jugaba solo.

Había un cañón, había una invasión, había veintisiete capítulos de diseño detrás, y las partidas las jugaba un autómata. Funcionaba, se medía, se repetía con la misma semilla. Y faltaba lo único por lo que existe un juego.

28.1 Hasta aquí jugaba una máquina, y eso escondía una pregunta

El autómata del capítulo 27 no es un jugador de mentira. Decide dónde apuntar, esquiva bombas, y con la misma semilla juega dos veces la misma partida. Es lo que permite medir.

Pero tiene una propiedad que un humano no tiene: **está siempre listo**. Le preguntas qué hace y responde en el acto. Nunca se distrae, nunca pulsa dos teclas a la vez, nunca tarda.

Detengámonos a pensar · Lo que un autómata no permite comprobar

Un autómata responde cuando se le pregunta. Una persona pulsa cuando quiere, y a veces dos veces seguidas, y a veces nada durante tres turnos.

Eso obliga a responder éstas preguntas que hasta ahora no habían hecho falta:

¿Qué pasa con lo que pulsaste y no dio tiempo a atender? Se pierde, o se acumula. Las dos respuestas son defendibles y hay que elegir una.

¿Cuánto dura un turno? Para el autómata, lo que tarde la máquina en calcularlo. Para una persona, eso es invisible o insufrible, según el día.

28.2 Un jugador es un contrato, no una persona

Desde el capítulo 27, el bucle no habla con un autómata: habla con un `Jugador`, que es un contrato de un solo método.

Código 28.1: El contrato, escrito veintisiete capítulos antes de que hiciera falta.

```
class Jugador(ABC):
    @abstractmethod
    def decidir(self, partida) -> Tuple[Orden, ...]:
        """Ordenes de este turno, en el orden en que deben aplicarse."""
```

Y el jugador humano:

Código 28.2: Seis líneas de código. Todo lo demás ya estaba.

```python
class Humano(Jugador):
    def __init__(self, fuente, adaptador=None):
        self._fuente = fuente
        self._adaptador = adaptador or AdaptadorTeclado()

    def decidir(self, partida):
        ordenes = (self._adaptador.traducir(t) for t in self._fuente.teclas())
        return tuple(o for o in ordenes if o is not Orden.NADA)
```

Seis líneas de código. Y quiero ser exacto con lo que esa cifra significa, un triunfo de la arquitectura y el diseño que hemos tratado hasta ahora.

Son seis líneas porque tres decisiones tomadas mucho antes se juntan aquí:

1. `Jugador` era un contrato y no una clase concreta (capítulo 12).
2. El bucle pide *varias* órdenes por turno, no una (capítulo 27).
3. Traducir teclas ya estaba resuelto y separado (capítulo 23).

⚖ Incorporar a alguien a un puesto que ya estaba definido

En el juego

El bucle no distingue entre el autómata y una persona. Pide órdenes a «un jugador» y aplica lo que le entreguen. Sustituir uno por otro es una línea en el punto de entrada.

Y no hubo que tocar ni la partida, ni la vista, ni la formación, ni el bucle.

En la vida real

Un puesto bien definido se cubre en un día: hay un encargo, unos límites y una forma de entregar. Quien llega se pone a ello.

Un puesto que solo existía en la cabeza de quien lo ocupaba tarda meses en cubrirse, y la primera semana consiste en averiguar qué hacía.

28.3 Leer teclas y traducir teclas son dos oficios

El capítulo 23 dejó resuelto cómo se traduce una tecla a una orden. Lo que no había —porque no hacía falta— era *de dónde salen las teclas*.

Y son dos cosas distintas, aunque suenen a lo mismo:

Un adaptador	**Una fuente**
Traduce: de una tecla a una `Orden`	Lee: dame las teclas que hay ahora
No lee nada	No traduce nada
Se prueba con una cadena de texto	Se prueba con un terminal de verdad

Separarlas no es purismo. Es lo que permite probar al jugador humano **sin un teclado delante**: se le da una fuente de mentira que devuelve las teclas que uno quiera, igual que el `RelojFalso` del capítulo 25 devuelve la hora que uno quiera.

Código 28.3: El doble de prueba. Con esto se comprueba al jugador humano en microsegundos.

```
class TecladoGuardado(FuenteDeTeclas):
    def __init__(self, *tandas):
        self._tandas = list(tandas)

    def teclas(self):
        return self._tandas.pop(0) if self._tandas else ()
```

Compruébalo · El jugador humano, sin tocar una tecla

Entre las pruebas del juego hay una que juega tres turnos moviéndose a la derecha y comprueba que el cañón acabó tres columnas más allá. No hay teclado, no hay terminal y no hay espera.

Si tu usuario humano no se puede probar así, es que leer y traducir están pegados.

28.4 El ritmo: un turno que dura lo que dura un parpadeo

Un autómata quiere ir lo más rápido que pueda. Una persona necesita que un turno dure lo que dura un parpadeo.

La tentación es poner una espera dentro del bucle. Es una línea y funciona. Y es el error, por dos motivos que no se ven hasta después:

Uno: medir dejaría de ser posible. El capítulo 32 juega treinta mil turnos para cronometrar el juego. Con una espera dentro, esa medición tardaría media hora y mediría la espera.

Dos: una espera fija se desajusta. Si el turno cuesta más de lo previsto —y con cincuenta y cinco invasores y varias balas, a veces cuesta—, el juego se va frenando sin que nadie lo note.

Código 28.4: El ritmo se pone por fuera, y se mide lo que ha costado el turno en lugar de suponerlo.

```
class RitmoReal:
    def esperar(self):
        self._siguiente += self.segundos
        falta = self._siguiente - self._fuente()
        if falta > 0:
            self._dormir(falta)
        else:
            # Vamos tarde: no se acumula deuda, se reengancha al presente.
            self._siguiente = self._fuente()
```

Por omisión el bucle usa `SinEspera`, que no hace nada. Jugar es la excepción, no la norma: **medir sigue costando lo que costaba**.

Sin programar · Por qué «no acumular deuda» no es un detalle

Si el turno tarda más de lo que debía, hay dos formas de reaccionar.
La primera es apuntar el retraso y correr después para recuperarlo. Suena justo, y produce un juego que se queda parado y luego pega un acelerón.
La segunda es olvidar el retraso y volver a marcar el compás desde ahora. Pierdes el turno perdido, y el juego se siente estable.
Es la misma decisión que toma cualquiera que lleve una agenda: cuando una reunión se alarga, o corres el resto del día detrás del reloj, o aceptas que lo de hoy se ha perdido y recolocas. La primera opción produce días en los que todo llega tarde.

28.5 Los dos defectos que solo aparecen con el teclado delante

El jugador humano pasaba todas sus pruebas. Se movía, disparaba, salía cuando se le pedía salir. Y con el teclado de verdad fallaba de dos maneras distintas, las dos invisibles para el doble de prueba.

Detengámonos a pensar · Uno: de tres teclas pulsadas seguidas llegaba una

El código era este, y parece correcto:

```
while select.select([self._flujo], [], [], 0)[0]:
    c = self._flujo.read(1)
    leidas.append(c)
```

Preguntar a `select` si hay algo que leer, leer un carácter, repetir. De «a», «d» y espacio pulsados seguidos, llegaba solo la «a».
La causa es que **hay dos buzones y no uno**. `select` mira el buzón del sistema operativo. Y `read(1)` no lee un carácter del sistema operativo: se trae un bloque entero al buzón de Python y devuelve el primero. Para `select`, ya no queda nada por leer, aunque en Python queden dos teclas esperando.
Se arregla leyendo del descriptor y no del flujo, con `os.read`, que no tiene buzón propio. Y lo que importa: **ninguna prueba con un teclado de mentira podía encontrarlo**, porque un teclado de mentira no tiene dos buzones.

Detengámonos a pensar · Dos: la flecha izquierda movía a la derecha

Una flecha no manda una tecla. Manda tres caracteres: `ESC`, `[` y una letra. La flecha izquierda manda `ESC [ D`.
Y `d` es, en este juego, la tecla de mover a la derecha.
Así que al pulsar la flecha izquierda llegaban tres teclas al adaptador, dos no significaban nada y la tercera significaba justo lo contrario de lo que el jugador había pedido.
La pregunta de siempre resuelve dónde arreglarlo: *¿quién debería saber que una flecha son tres caracteres?* No el adaptador, que traduce teclas: para él una flecha **es** una tecla.

Lo sabe quien lee, porque leer es justamente el oficio de saber en qué forma llegan las cosas.
Se agrupan en la fuente. El adaptador no cambió.

Los dos están ahora vigilados por una prueba que abre un pseudo-terminal de verdad, escribe en él y comprueba lo que llega. Es la única prueba del libro que necesita un aparato.

Lo que cada uno puede decirte, y lo que no

El doble de prueba

Rápido, repetible, sin condiciones. Comprueba la lógica: que una «a» se convierte en mover a la izquierda, que lo que no significa nada no llega al juego.
Y te dirá que todo está bien mientras el teclado real pierde dos de cada tres pulsaciones.

El aparato de verdad

Lento, incómodo, y a veces no está disponible —por eso esta prueba se salta sola donde no hay pseudo-terminales—.
Es el único que sabe que hay dos buzones y que una flecha son tres caracteres.
Ninguno de los dos sobra.

28.6 Lo que costó, contado en líneas

Las cifras, medidas sobre el código y no estimadas:

Pieza	Líneas	Qué es
`Humano`	6	Una clase nueva que cumple el contrato de jugador
`FuenteDeTeclas` y su doble	14	El contrato de quien lee, y su falso
`TecladoTerminal`	65	El aparato de verdad
`RitmoReal` y `SinEspera`	24	Quién marca el compás, y quién no
El ritmo en el bucle	4	Cuatro líneas en el control
La opción `-humano`	22	En la raíz de composición

Y la lectura honesta de esa tabla no es «el jugador humano costó seis líneas». Es esta: **el jugador costó seis líneas y el aparato sesenta y cinco**.

Lo que el diseño abarató fue la parte que dependía del diseño. Hablar con un terminal cuesta lo que cuesta hablar con un terminal, y ningún reparto de responsabilidades lo va a bajar.

Si esto fuera un equipo · Lo que se puede preparar y lo que no

Un puesto bien definido se cubre rápido, y eso es real. Pero si el trabajo consiste en tratar con una administración concreta, con su ventanilla y sus plazos, esa parte cuesta lo que cuesta y no hay organigrama que la abarate.
Confundir las dos —creer que porque el puesto está bien definido el trabajo será fácil—

es la forma más común de subestimar un proyecto.

Inténtalo primero · Antes de seguir

Ahora que hay una persona jugando, el cañón sigue siendo exactamente el mismo de la primera pantalla a la última: una vida menos, un disparo cada vez.
Piensa qué haría falta para que el jugador *mejorase* durante la partida, y en concreto: ¿quién debería saber que esa nave que acabas de derribar llevaba algo dentro?
De eso va el capítulo siguiente, y la respuesta no es «el invasor».

El jugador ya está dentro. Falta que tenga algo que ganar.

Capítulo 29

Recompensas que caen

En el capítulo anterior entró una persona a jugar. Y se encontró con un cañón que es idéntico en la primera pantalla y en la última: se mueve, dispara una bala cada vez y tiene tres vidas.

La invasión, en cambio, lleva veinte capítulos ganando cosas: tipos distintos, escudos, aceleración. **El jugador era la única pieza del juego que no se había diseñado.**

Este capítulo atiende una mecánica que permite mejorar al jugador durante el juego.

29.1 El jugador también se diseña

Lo que quiero es sencillo de decir: que algunas naves suelten algo al morir, que eso caiga, y que si el cañón pasa por debajo se lo quede y mejore.

Tres mejoras, y cada una cambia una cosa distinta del disparo:

	Mejora	Qué cambia
`D`	Tiro doble	Dos balas en paralelo por cada una que hubiera
`R`	Tiro rápido	Más balas en el aire y suben más deprisa
`P`	Perforante	La bala no se para en el primero: atraviesa varios

Dicho así parece una lista de tareas. Es, en realidad, cuatro preguntas de reparto, y de las respuestas depende que esto sea un rediseño o una implementación sencilla.

29.2 ¿Quién sabe que esa nave lleva algo dentro?

Es la pregunta del libro, otra vez, y aquí tiene cuatro respuestas encadenadas.

Quién	Qué sabe, y nada más
La nave	**Qué lleva** encima. No sabe soltarlo
La partida	Que la nave ha muerto y dónde. **Crea** la recompensa
La recompensa	**Caer**. No sabe qué hace lo que transporta
El cañón	**Ponerse** una mejora. No sabe de dónde vino

Detengámonos a pensar · Por qué la nave no suelta su propia recompensa

La respuesta cómoda es que el invasor, al morir, cree la recompensa y la eche al tablero. Es una línea, y está mal por el mismo motivo que el capítulo 6 ya explicó con los impactos.

Un invasor no ve el tablero. No sabe dónde está el suelo, ni si la partida sigue en

juego, ni si esa recompensa cabe. Para soltarla tendría que recibir el tablero entero, y en cuanto lo reciba podrá hacer con él cualquier cosa.
Quien ve al invasor muerto y al tablero a la vez es la partida. Y ya lo hacía: es la misma clase que decide que un impacto mata, porque es la única que ve a la bala y al invasor (capítulo 6).
Así que el invasor gana un atributo —`carga`, lo que lleva encima— y ni un método. Sabe qué transporta y no sabe transportarlo, igual que una carta no sabe echarse al buzón.

Definición 29.1 (Recompensa)**.** Una *recompensa* es una entidad que cae y que transporta una mejora sin usarla. No sabe quién la soltó ni quién la recogerá; solo sabe bajar y, si sale del tablero, dejar de existir.

Y una decisión de juego que es también de diseño: la recompensa baja una fila **cada tres turnos**, más despacio que una bomba. No es un capricho. Si cayera a la velocidad de una bomba no daría tiempo a llegar, y una recompensa inalcanzable es una recompensa que no existe.

⚖ Quien lo lleva no es quien lo entrega

En el juego

El invasor sabe qué carga. La partida ve el tablero. La recompensa sabe caer. El cañón sabe ponérsela.
Cuatro piezas, y si mañana quiero que las recompensas reboten en los búnkeres, lo toco en una.

En la vida real

Quien ejecuta un pedido no es quien lo empaqueta, ni quien lo transporta, ni quien lo envía. Cada uno sabe su tramo y ninguno necesita saber los otros tres.
Cuando una sola persona hace los cuatro tramos funciona, hasta el día que se va de vacaciones.

29.3 Una mejora es un arma que envuelve a otra

El capítulo 11 dejó esto escrito, y entonces era casi una promesa:

> Un cañón con disparo doble no es una subclase de cañón: es un cañón con otra arma puesta.

Ahora hay que cumplirla, y con una condición nueva: el cañón cambia de arma **a mitad de partida**, y puede llevar las tres mejoras a la vez.

La salida fácil sería un arma con tres banderas: `es_doble`, `es_rapida`, `perfora`. Funciona con tres. Con la cuarta hay ocho combinaciones dentro de un método, y es el capítulo 11 contado al revés.

Definición 29.2 (Mejora)**.** Una *mejora* es un arma que envuelve a otra arma y le cambia una cosa, delegando todo lo demás. Como envuelve a un arma y *es* un arma, se puede poner encima de otra mejora, y de ahí que se apilen sin límite y sin conocerse.

El nombre que esto tiene en la literatura es **decorador**, y no le doy capítulo propio a propósito: no es nada que este libro no lleve veintinueve capítulos haciendo. Es componer en lugar de heredar, con la vuelta de tuerca de que lo compuesto es del mismo tipo que la pieza.

Código 29.1: El decorador base. Doce líneas, y no cambia nada: solo se pone en medio.

```
class Mejora(Arma):
    """Un arma que envuelve a otra. Por omision no cambia nada."""

    __slots__ = ("interior",)

    def __init__(self, interior):
        super().__init__(interior.balas_maximas, interior.velocidad)
        self.interior = interior

    def mejoras(self):
        return self.interior.mejoras() + (self.nombre,)

    def bocas(self, x, y):
        return self.interior.bocas(x, y)

    def fabricar_bala(self, x, y, velocidad):
        return self.interior.fabricar_bala(x, y, velocidad)
```

Y ponerse una mejora, en el cañón, es una línea:

```
self._arma = mejora(self._arma)
```

Fíjate en lo que **no** hay en el cañón: ni un `if` por tipo de mejora, ni un atributo `tiene_tiro_doble`, ni una lista de mejoras que alguien tenga que recorrer. El cañón no sabe cuántas mejoras existen. Sabe ponerse la que le den.

29.4 Cada mejora cambia una sola cosa, y por eso se apilan

Un arma tiene exactamente dos costuras por donde se la puede extender, y las dos son públicas a propósito:

Costura	Qué decide
`bocas(x, y)`	De dónde sale cada bala de un disparo
`fabricar_bala(...)`	Qué clase de bala sale

Con eso, las tres mejoras son casi triviales:

Código 29.2: Tres mejoras. Ninguna sabe que las otras existen.

```
class TiroDoble(Mejora):
```

```
    def bocas(self, x, y):
        return tuple((px + d, py)
                     for px, py in self.interior.bocas(x, y)
                     for d in (-1, 1))


class TiroRapido(Mejora):
    def __init__(self, interior):
        super().__init__(interior)
        self._balas_maximas = interior.balas_maximas + 2
        self._velocidad = interior.velocidad + 1


class Perforacion(Mejora):
    def fabricar_bala(self, x, y, velocidad):
        return BalaPerforante(x, y, velocidad, self.atraviesa)
```

29.5 La bala que atraviesa: quién decide si un proyectil sigue vivo

La bala perforante parecía la más difícil de las tres y resultó ser la más ilustrativa. Hasta aquí, quien resolvía un impacto mataba al proyectil:

```
if invasor is not None:
    proyectil.morir()          # <- la partida decide por la bala
```

Con una bala que atraviesa, eso deja de funcionar. Y la salida fácil es preguntar de qué tipo es:

```
if isinstance(proyectil, BalaPerforante):
    ...
```

Un condicional que crece con cada tipo de bala nuevo, en el sitio más transitado del juego.

La pregunta de siempre lo resuelve: *¿quién debería saber si esta bala sigue viva tras dar en algo?* La partida sabe **a quién ha dado**, porque es la única que ve a los dos. Lo que le pasa al proyectil después solo lo sabe el proyectil, porque solo él sabe si atraviesa.

Código 29.3: Un método en la clase base, otro en la que atraviesa, y ni un condicional en la partida.

```
class Proyectil(Entidad):
    def impactar(self):
        """Ha dado en algo. Por omision, ahi se acaba el viaje."""
        self.morir()


class BalaPerforante(Bala):
    def impactar(self):
```

```
        # POSCONDICION del capitulo 15: esto tiene que terminar muriendo.
        self._atraviesa -= 1
        if self._atraviesa <= 0:
            self.morir()
```

La partida pasa a decir `proyectil.impactar()` y no se entera de nada más. **La clase nueva no añadió un solo `if` en ninguna otra parte del juego**, que es la definición operativa del principio abierto/cerrado del capítulo 14.

Con una excepción, y es deliberada: **un búnker para cualquier cosa**, también lo que atraviesa invasores. Un muro no es un enemigo, y ahí sí se llama a `morir()` directamente.

Sin programar · Cómo reconocerlo sin leer código

Cuando veas un condicional que pregunta «¿de qué tipo eres?» para decidir qué le pasa a ese mismo objeto, casi siempre la decisión está en el sitio equivocado.

La prueba: si añadir un tipo nuevo te obliga a volver a ese condicional, entonces ese condicional era el objeto contestando por otro.

29.6 El orden no importa, y eso es una cláusula que se prueba

Aquí hay una promesa fuerte que conviene analizarla a detalle.

`TiroDoble(Perforacion(Arma()))` y `Perforacion(TiroDoble(Arma()))` tienen que disparar lo mismo: dos balas perforantes. Si no fuera así, el orden en que recoges las cajas cambiaría tu arma, y nadie podría razonar sobre ella.

Que se cumpla no es casualidad: sale de que **cada mejora delega hacia dentro lo que no cambia**. La capa que sabe hacer algo lo hace, y las demás se apartan. Ninguna mejora sabe qué hay debajo ni encima.

Compruébalo · La cláusula, escrita como prueba

Entre las pruebas de este capítulo:

```
def test_el_orden_en_que_se_apilan_no_cambia_el_resultado(self):
    una = TiroDoble(Perforacion(Arma()))
    otra = Perforacion(TiroDoble(Arma()))
    for arma in (una, otra):
        balas = arma.disparar(20, 21)
        self.assertEqual(2, len(balas))
        self.assertTrue(all(isinstance(b, BalaPerforante) for b in balas))
    self.assertEqual(una.balas_maximas, otra.balas_maximas)
```

Es una poscondición como las del capítulo 15: la firma no la puede expresar, y sin ella el diseño no significa lo que dice.

29.7 Dos defectos que aparecen al ejecutarlo

Los dos son de este capítulo, los dos parecían correctos escritos, y los dos son de la misma familia: **una cuenta que nadie había hecho**.

Detengámonos a pensar · Uno: mil veinticuatro balas por tecla

La primera versión dejaba apilar la misma mejora tantas veces como la recogieras. Suena inofensivo.

`TiroDoble` **duplica** las balas de un disparo. Dos doble son cuatro. Diez doble son **mil veinticuatro balas por tecla**, cada una resolviendo colisiones contra cincuenta y cinco invasores.

El autómata recogía recompensas bien, así que la partida de prueba se quedó dos minutos calculando en lugar de medio segundo. No hubo excepción ni error: solo un juego que dejó de responder.

La regla que lo arregla es de juego y de diseño a la vez: **la misma mejora no se apila dos veces**. Se recoge igual, pero no suma.

Y lo que me interesa de esto: la salida exponencial no estaba en ninguna línea sospechosa. Estaba en la *composición* de dos decisiones razonables.

Detengámonos a pensar · Dos: siete balas en el aire con un arma de seis

El capítulo 31 deja cinco invariantes vigilados en cada turno de treinta partidas. Uno dice: *nunca hay más balas en el aire de las que el arma permite*. Llevaba veinte capítulos sin saltar.

Saltó en el turno 759: siete balas donde caben seis. La causa, escrita, resulta muy simpática a la vista y desastrosa a nivel de jugabilidad:

```
if mundo.balas_en_vuelo() >= self._arma.balas_maximas:
    return []                      # <- se pregunta ANTES de saber cuantas
return self._arma.disparar(...)    # <- y el tiro doble suelta dos
```

Con cinco balas en el aire y seis permitidas, la comprobación pasa —cinco es menos que seis— y entonces entran dos. Siete.

Se arregla contando primero y preguntando después: **la ráfaga entra entera o no entra**.

Veintiocho capítulos enseñando a repartir responsabilidades, y el defecto estaba en el orden de dos líneas. Por eso este libro destaca muchísimo la búsqueda continua de la calidad.

29.8 Lo que costó la vuelta siguiente

La pregunta que este libro le hace a todo: *si mañana quiero una cuarta mejora, ¿cuánto cuesta?*

Medido quitando `TiroDoble` del juego terminado y contando lo que hace falta para volver a ponerlo:

Una cuarta mejora para el jugador	
Archivos que hay que tocar	1
Sitios que hay que tocar	2
Líneas que hay que escribir	18

Dos sitios, los dos donde vive el arma: la clase, y su nombre en la tupla `MEJORAS` de la que reparte la fábrica. Ni el cañón, ni la partida, ni la recompensa, ni la vista.

Es la cifra más baja del libro, y hay que decir por qué: porque esta parte del diseño se hizo *después* de veintisiete capítulos discutiendo dónde va cada cosa. No es que componer sea mágico. Es que si llegamos a este punto, hemos aprendido a preguntar.

Si esto fuera un equipo · Dar a alguien una capacidad nueva sin cambiarle el puesto

La forma cara de que alguien haga algo nuevo es rehacerle el puesto. La barata es que el puesto ya admitiera capacidades y se le añada una.

La diferencia no se ve el primer día: se ve el tercero, cuando hay que hacerlo otra vez y no hay que volver a negociar la descripción del puesto.

El jugador ya mejora durante la partida. Falta que la invasión no sea siempre la misma.

Capítulo 30

Oleadas que no se repiten

El jugador ya mejora durante la partida; la invasión sigue siendo, oleada tras oleada, cincuenta y cinco bichos que bajan igual.

El reto de este juego, enunciado en el capítulo 18, no era demostrar que no se puede perder: era que las mecánicas funcionen, que la velocidad sea la adecuada y que el juego aguante horas. **Un juego que en la oleada nueve hace exactamente lo mismo que en la primera no aguanta horas.**

30.1 El dolor: una clase por nivel

Quiero que a partir de cierta oleada algunos invasores disparen por su cuenta. Y que otros se salgan de la retícula. Y que los de arriba, más adelante, se reparen solos además de llevar escudo.

Lo que quiero	La clase que pediría
Un pulpo que dispara solo	`PulpoArtillero`
Un calamar que zigzaguea	`CalamarZigzag`
Un acorazado con escudo	`AcorazadoConEscudo`
. . . y con regeneración	`AcorazadoConEscudoYRegeneracion`

30.2 La progresión, en una tabla y en un solo sitio

La segunda pregunta, y es la que decide si esto se puede afinar: *¿quién debería saber que la oleada seis trae invasores que zigzaguean?*

No el invasor, que no sabe en qué oleada está. No la formación, que solo sabe mover al conjunto. No la partida, que ya tiene diecisiete métodos.

Lo sabe **la fábrica del capítulo 20**, que es la única pieza del juego cuyo oficio es componer una oleada a partir de su número. Y lo sabe en una tabla:

Código 30.1: Toda la dificultad del juego. Una línea por dificultad, y el orden es la prioridad.

```
PROGRESION = (
    Regla(8, "escudo y regeneracion", lambda s: s.arriba,
          lambda n: Combinada(Escudo(1), Regeneracion(cada=90))),
    Regla(5, "escudo", lambda s: s.arriba,
          lambda n: Escudo(1)),
    Regla(4, "artillero", lambda s: s.abajo and s.columna % 4 == 0,
          lambda n: Artillero(cada=max(10, 30 - 2 * n))),
    Regla(6, "zigzag", lambda s: s.fila % 2 == 1 and s.columna % 3 == 0,
```

```
            lambda n: Zigzag(cada=max(4, 10 - n // 2), amplitud=2)),
    )
```

Una regla dice tres cosas: **desde qué oleada**, **a quién** y **qué se le pone**. Nada más. Y el resultado es este:

Oleada	Qué aparece	Qué cambia para quien juega
1–2	—	Aprender a moverse y apuntar
3	Acorazados arriba	La fila de arriba aguanta dos impactos
4	Artilleros abajo	Cae fuego que no viene de la estrategia
5	Escudos arriba	El primer impacto de arriba no cuenta
6	Zigzag	La retícula deja de ser una retícula
8	Escudo y regeneración	Lo de arriba se cura si le das tiempo

Detengámonos a pensar · Subir la dificultad no toca el juego: toca la tabla

Esto es lo que hay que retener del capítulo, y es comprobable: los números de la izquierda de esa tabla son **los únicos sitios** donde se decide la curva de dificultad.

Ni la partida, ni el invasor, ni la formación, ni la vista, ni el bucle saben que existen las oleadas difíciles. La partida le pide a la fábrica «la oleada siete» y recibe una formación. Qué trae dentro no es asunto suyo.

Y por eso afinar es barato, que es lo que un juego necesita: afinar no se hace una vez, se hace cincuenta.

30.3 Artillero: disparar sin esperar turno

El capítulo 21 dejó el disparo de la invasión en manos de una estrategia: cada turno, ella elige *un* tirador entre los de abajo.

Un artillero no espera su turno.

Código 30.2: Once líneas, y cambian cómo se siente el juego a partir de la oleada cuatro.

```
class Artillero(Habilidad):
    def al_avanzar(self, portador, mundo):
        self._cuenta += 1
        if self._cuenta < self.cada:
            return
        self._cuenta = 0
        x, y = portador.posicion_de_disparo()
        mundo.soltar_bomba(x, y)
```

Dos detalles que no son detalles.

Se pide soltar la bomba; no se inverviene en la lista. La habilidad no hace `mundo.proyectiles.append(...)`. Llama a `soltar_bomba`, que es un método de la partida y que

comprueba si caben más bombas. Que quepan tres a la vez es una regla del juego, y tiene que valer igual para la estrategia y para el artillero.

Dispara más seguido conforme avanza la oleada: `cada=max(10, 30 - 2*n)`. La dificultad no crece a saltos, crece dentro de la propia habilidad.

30.4 Zigzag: salirse de la retícula

Un invasor no decide cuándo avanza la formación —eso es del conjunto, y es el corazón del capítulo 11—. Pero sí puede moverse *además*.

Código 30.3: El límite del mundo no lo negocia una habilidad.

```
def al_avanzar(self, portador, mundo):
    ...
    destino = portador.caja().desplazada(dx=self._sentido)
    if not destino.dentro_de(mundo.limites):
        self._sentido = -self._sentido
        return
    self._desviacion += self._sentido
    portador.desplazar(dx=self._sentido)
```

Esa comprobación es la parte importante. Una habilidad puede hacer casi cualquier cosa con su portador, y por eso hay que decidir qué **no** puede: sacarlo del tablero, no.

Compruébalo · El invariante, vigilado

```
def test_el_zigzag_no_saca_a_nadie_del_tablero(self):
    h = Zigzag(cada=1, amplitud=3)
    for _ in range(200):
        h.al_avanzar(invasor, p)
        self.assertTrue(invasor.caja().dentro_de(p.limites))
```

Doscientos pasos, comprobando en cada uno. Sin esto, la habilidad sería una promesa.

30.5 Combinada: escudo y regeneración sin una clase nueva

Quiero que a partir de la oleada ocho los de arriba lleven escudo **y** regeneración. Con herencia, eso es `InvasorConEscudoYRegeneracion`; con una habilidad más, ocho clases.

Código 30.4: Varias habilidades en una. Es una lista, no una clase por combinación.

```
class Combinada(Habilidad):
    def absorbe_impacto(self, portador):
        # `any` se para en la primera que se lo come: un impacto gasta una
        # carga, no todas.
        return any(h.absorbe_impacto(portador) for h in self._partes)
```

```
6
7       def al_avanzar(self, portador, mundo):
8           for h in self._partes:
9               h.al_avanzar(portador, mundo)
```

Combinar capacidades sin inventar un puesto por combinación

En el juego

Escudo y regeneración a la vez son una lista de dos, y las dos siguen siendo las mismas clases que usan los búnkeres y los invasores por separado.
La sexta habilidad no multiplicará nada: será una clase más.

En la vida real

Cuando alguien necesita saber de dos cosas, la organización sana le da la segunda formación. La enferma crea un puesto nuevo con nombre compuesto, y otro para la siguiente combinación.
Al tercer nombre compuesto, nadie sabe ya quién hace qué.

30.6 Lo que costó subir la dificultad, medido

Medí lo que cuesta añadir **una habilidad más** a la invasión, quitando `Zigzag` del juego terminado y contando lo que hace falta para reponerlo.

Salieron **siete sitios**. Y seis de ellos estaban en la fábrica.

Detengámonos a pensar · El defecto del capítulo 14, cometido en el capítulo que presume de resolverlo

La primera versión daba a la fábrica un parámetro por habilidad: `oleada_escudos`, `oleada_artilleros`, `oleada_zigzag`...
Funcionaba perfectamente. Y añadir una habilidad obligaba a tocar seis sitios en esa clase: la constante, la importación, el parámetro en `__slots__`, el parámetro en el constructor, la asignación y la regla.
Con la progresión en una tabla —la del apartado «la progresión, en una tabla y en un solo sitio»— una habilidad nueva es una línea en la tabla y una clase en su módulo.

Una habilidad más para la invasión	Con parámetros	Con la tabla
Archivos que hay que tocar	2	2
Sitios que hay que tocar	**7**	**3**

Y lo que cuesta el juego entero de esta parte: el juego pasó de 1 070 a 1 737 líneas de código, de 42 a 59 clases y de 21 a 23 módulos.

30.7 Y lo que encareció, que también hay que decirlo

Sería cómodo cerrar en la sección anterior. Estas tres cifras dicen lo contrario y son igual de medidas.

Uno: un cambio viejo se ha vuelto más caro. «Dos balas en el aire» costaba 9 sitios y ahora cuesta 13. La razón es directa: ahora hay tres mejoras que calculan cuántas balas caben, y todas tienen algo que decir sobre ese número. **La flexibilidad nueva se paga en el sitio donde antes solo había un atributo.**

Dos: el juego consume más. Medido sobre diez partidas por versión, con la misma vara para todas:

Versión repartida	Pico de memoria	Microsegundos por turno
Sin lo de esta parte	31 018 B	139,7
Con lo de esta parte	96 192 B	227,4
Lo que cuesta la funcionalidad	×3,1	×1,6

Tres veces la memoria. Y conviene separar dos cosas que es muy fácil sumar y no son lo mismo: **una parte de lo que gasta este juego es el precio del reparto, y otra es el precio de la funcionalidad**.

Tres: el juego es más difícil, y eso no siempre es mejor. Sobre cien partidas del autómata:

Versión repartida	Puntuación media	Despejan oleada	Oleada máxima
Sin lo de esta parte	2 191	65 %	16
Con lo de esta parte	7 211	97 %	12

Las recompensas hacen que casi todas las partidas pasen de la primera oleada, y que se puntúe tres veces más. Y a la vez, **la oleada máxima alcanzada baja de 16 a 12**: llegar lejos es más difícil que antes, porque a partir de la ocho la invasión se cura.

Sin programar · Cómo se sabe que una curva de dificultad funciona

No por jugar una partida y que te parezca bien. Por dos números que tienen que moverse en direcciones distintas:

Cuántos pasan del principio. Si es bajo, el juego expulsa a quien empieza.

Hasta dónde llega el mejor. Si es bajo, el juego se acaba para quien se queda.

Si los dos suben a la vez, casi seguro has hecho el juego más fácil y no mejor.

Con esto, el juego está terminado y jugado. La parte siguiente vuelve a la pregunta incómoda del libro: qué de todo esto se puede demostrar.

VIII

Lo que realmente cambió

Capítulo 31

Comprobarlo cuando no se puede recorrer todo

El libro anterior podía recorrer el tres en raya validando todos sus estados posibles: 1 784 partidas, cero derrotas, en menos de un segundo. No era una estimación ni una muestra: era un recuento, y con él se podía *demostrar* que aquel algoritmo no perdía.

Aquí eso no está disponible. El azar hace que sea imposible recorrer todos los estados posibles del juego.

31.1 El espacio de estados de este juego, calculado

El estado de una partida es todo lo que hace falta saber para continuarla.

Cuadro 31.1: Cuántos valores puede tomar cada parte del estado. No se cuentan la puntuación ni el número de oleada: no cambian lo que puede pasar a continuación.

Parte del estado	**Valores posibles**
Invasores vivos o muertos	$\approx 10^{16}$
Desplazamiento de la formación	110
Sentido de avance	2
Contador de cadencia	10
Columna del cañón	42
Vidas	3
Bala propia (o ninguna)	1 057
Bombas en vuelo (hasta 3)	$\approx 10^{8}$
Estado de los búnkeres	256
OVNI (o ninguno)	89
Total (cota superior)	$\approx 10^{37}$

Detengámonos a pensar · Qué significa 10^{37}

Supón una máquina capaz de examinar **mil millones de estados por segundo**, que es muy optimista. Recorrerlos todos llevaría del orden de $1{,}5 \times 10^{21}$ **años**.

Y conviene decir que es una **cota superior**: muchas de esas combinaciones no son alcanzables jugando —no puede haber una bala en la fila del cañón y a la vez el cañón intacto en ese sitio, por ejemplo—. Aunque solo fuera alcanzable una millonésima parte, seguirían siendo 10^{31}.

La conclusión no depende de la precisión del cálculo. Depende del exponente.

31.2 Lo que se pierde al perder la exhaustividad

Conviene ser exacto sobre qué se ha perdido, porque no es todo.

La diferencia, dicha sin suavizar

Lo que podía el libro anterior	Lo que puedo aquí
Demostrar. «No existe ninguna secuencia de jugadas del rival que lleve a perder.» Recorrido completo, cero derrotas, certeza. Cuando algo se recorre en su totalidad, un resultado negativo significa algo: no encontrar el fallo *es* no haberlo.	**No encontrar.** «En 30 partidas y 60 000 turnos no ocurrió.» No encontrar un fallo y no haberlo son cosas distintas, y aquí solo puedo afirmar la primera.

Lo que *no* se ha perdido es tan importante como lo perdido:

El reto también cambió. El capítulo 18 lo dejó dicho: aquí no hay que garantizar que no se pueda perder, sino que las mecánicas funcionen y que el juego aguante. Y eso, curiosamente, se comprueba mejor jugando mucho que enumerando todo.

31.3 Determinismo: fijar el azar para poder repetir

La primera de las tres herramientas, y la que sostiene a las otras dos.

Código 31.1: La prueba más importante del juego. Si esta falla, ninguna de las demás significa nada.

```
    def test_la_misma_semilla_da_la_misma_partida(self):
        for s in (0, 7, 23):
            with self.subTest(semilla=s):
                a, b = _jugar(s), _jugar(s)
                self.assertEqual(
                    (a.puntuacion, a.oleada, a.turno, a.desenlace),
                    (b.puntuacion, b.oleada, b.turno, b.desenlace))
```

Detengámonos a pensar · Por qué esta va de primera

Sin determinismo, un fallo que aparece una vez no se puede volver a ver, y un fallo que no se puede volver a ver no se puede arreglar: solo se puede esperar a que vuelva.
Y hay una segunda prueba, aparentemente absurda, que compruebo justo después: que **semillas distintas den partidas distintas**. Si las dos pasaran porque el juego siempre hace lo mismo, el determinismo no sería una propiedad: sería que el azar está roto.
Las dos juntas dicen lo que hace falta: el juego es impredecible y repetible a la vez.

Y esto es posible por el capítulo 26 y por nada más: la semilla entra por el constructor. Si la partida fabricara su propio azar, este capítulo no existiría.

31.4 Propiedades en lugar de casos

La segunda herramienta. En lugar de comprobar que la partida número 4 712 termina bien, se comprueba que **en ninguna partida ocurre lo que no puede ocurrir**.

Código 31.2: Cinco invariantes vigilados en cada turno de treinta partidas. Uno de los cinco.

```
        def bombas(p):
            n = sum(1 for x in p.proyectiles
                    if x.vivo and x.bando is Bando.INVASION)
            self.assertLessEqual(n, BOMBAS_SIMULTANEAS,
                                 f"turno {p.turno}: {n} bombas")
```

Cuadro 31.2: Las propiedades que vigila el juego, y qué protege cada una. Se comprueban en cada turno de 30 partidas, en una sola pasada.

Propiedad	Qué fallo atraparía
Nunca más de 3 bombas a la vez	Una regla del juego rota por un camino nuevo
Nunca más balas de las que permite el arma	El cañón disparando sin límite
Nada vivo se sale del tablero	Un invasor invisible que impide despejar la oleada
Los proyectiles muertos no se acumulan	Una fuga que degradaría una sesión larga
La puntuación nunca baja	Una resta donde debería haber una suma

Detengámonos a pensar · Una propiedad no es un caso de prueba, y la diferencia importa

Un caso de prueba dice: «con esta entrada, esta salida». Comprueba un punto.
Una propiedad dice: «pase lo que pase, esto no». Comprueba una *región*, y la comprueba en cada turno de cada partida que se juegue, incluidas las que se jueguen dentro de un año con reglas que hoy no existen.
Y tiene un efecto secundario: **escribir la propiedad obliga a saber qué es imposible**. «La puntuación nunca baja» parece obvia hasta que alguien añade una penalización, y entonces la prueba te obliga a decidir si es una penalización legítima o un error.

31.5 Invariantes que no pueden romperse

La tercera herramienta ya está contada: son los del capítulo 6, garantizados en el nacimiento y protegidos por la frontera del capítulo 7.

Lo que añade este capítulo es que **también se comprueban**. Están en el archivo de

pruebas de contratos: que un invasor no pueda nacer en la fila -3, que una formación vacía no exista, que un cañón nazca con vidas.

Y una que no es un invariante de clase sino de contrato, la del capítulo 15:

Código 31.3: La cláusula que la firma no puede expresar, comprobada sobre todas las habilidades que existan.

```
    def test_toda_habilidad_termina_cediendo(self):
        for clase in _todas_las_subclases(Habilidad):
            with self.subTest(habilidad=clase.__name__):
                h = clase()
                cede = any(not h.absorbe_impacto(None) for _ in range(1000))
                self.assertTrue(cede, ...)
```

Fíjate en que recorre las subclases: la habilidad que alguien escriba mañana queda comprobada sin tocar la prueba.

31.6 Lo que este capítulo *no* demuestra

Cuadro 31.3: Lo que se ha comprobado y lo que eso significa.

Lo comprobado	Lo que significa
15 pruebas, 30 partidas, 5 invariantes por turno	Que en esas 30 partidas no ocurrió. No que no pueda ocurrir
21 partidas encadenadas, memoria plana	Que no hay fuga *en ese recorrido*. Una fuga lenta necesitaría más
Determinismo en 3 semillas	Que esas 3 se repiten. Es una comprobación puntual de una propiedad general
Toda partida alcanza un desenlace	En 30 semillas. Con la 31 quizás no

Si esto fuera un equipo · Auditar todo o vigilar lo que no puede pasar

Cuando un proceso es pequeño se revisa entero. Cuando crece, revisarlo entero deja de ser posible y hay que elegir.

La salida mala es revisar una muestra y llamarla garantía. La buena es distinta: definir qué **no puede pasar nunca** y vigilar eso en todo, en lugar de vigilarlo todo en una muestra.

Es la diferencia entre auditar el 5 % de los expedientes y comprobar que ningún expediente sale incompleto.

Con esto queda dicho qué se puede afirmar de este juego y con qué fuerza.

Lo que falta es la factura. El capítulo 18 prometió poner juntas las dos cifras que tiran en direcciones opuestas: lo que ahorró dimensionar el dato y lo que costaron los ocho patrones.

Y el capítulo 13 dejó pendiente algo peor: la medición del coste del cambio no favorece a la orientación a objetos tanto como se supone.

Capítulo 32

El diseño se mide por el cambio siguiente

Este es el capítulo donde el libro paga sus facturas. Hay tres pendientes:

1. El capítulo 18 dejó planteada la tensión entre las dos cifras que tiran en direcciones opuestas: lo que ahorró dimensionar y lo que cuestan los ocho patrones.
2. El capítulo 14 midió que añadir un tipo de invasor cuesta cinco sitios en la versión repartida y seis en el monolito, lo que claramente no es una victoria.
3. Y llevo treinta y un capítulos diciendo que repartir sale a cuenta.

Las tres se saldan con la misma tabla.

32.1 Las cifras, medidas y no estimadas

Todo lo que sigue sale de ejecutar las dos versiones y medirlas con la misma vara.

Cuadro 32.1: Tamaño de las dos versiones. La segunda fila descuenta la documentación, porque el juego repartido está mucho más comentado y contarlo como código sería hacer trampa.

	Monolito	Repartido
Unidades (archivos)	1	23
Líneas de código	271	1 737
Ídem, sin documentación	253	1 255
Funciones y métodos	2	200
Clases	0	59

Cuadro 32.2: Consumo, sobre 10 partidas completas por versión. Se mide el **pico** de memoria durante la partida, y el tiempo es el mejor de tres cronometrajes. Las cifras de memoria se repiten exactas entre ejecuciones; las de tiempo varían un 3 %, y por eso las proporciones de tiempo se dan redondeadas.

	Pico de memoria	Microsegundos por turno
Monolito	6 770 B	28,9
Repartido, sin la Parte VII	31 018 B	139,7
Repartido en objetos	96 192 B	227,4

Detengámonos a pensar · Dos precios que es muy fácil sumar, y no son lo mismo

Cuando la Parte VII le puso al juego un jugador humano, recompensas y oleadas que cambian, el juego repartido **dejó de jugar al mismo juego** que las otras dos versiones. Dura más, puntúa más y gasta más. Comparar su consumo con el del monolito sin decir eso sería comparar dos juegos distintos y llamarlo diseño.

Así que se mide también *apagando* lo nuevo, que se puede hacer sin tocar el juego porque la fábrica lo admite como parámetro (capítulo 30). Y entonces se leen dos precios distintos:

	Memoria	Tiempo por turno
El precio del **reparto**	×4,6	≈ ×5
El precio de la **funcionalidad**	×3,1	≈ ×1,6
Lo que ves en la tabla de arriba	×14,3	≈ ×8

No se suman: se multiplican. Y sobre todo, **no se atribuyen al mismo culpable**. Quien mire la última fila y concluya «las clases salen carísimas» está cobrándole al diseño el precio de tres funcionalidades que el monolito no tiene.

32.2 Los tres cambios del capítulo 2, ahora

La segunda factura. El capítulo 2 midió lo que cuesta hacer tres cambios sobre el monolito. Aquí están los mismos tres sobre el juego repartido, medidos con el mismo criterio.

Cuadro 32.3: Sitios distintos que hay que tocar por cambio. «Sin la raíz» excluye la raíz de composición, que es donde se expone la opción al usuario y no mide diseño: el monolito no tiene interfaz que actualizar, así que incluirla sería compararlo con ventaja.

Cambio pedido	Monolito	Repartido	Sin la raíz
Un cuarto tipo de invasor	6	5	5
Dos balas en el aire	4	13	7
Los diez mejores marcadores	6	12	1
Una cuarta mejora para el jugador	—	2	2
Una habilidad más para la invasión	—	3	3

32.3 Lo que el reparto costó en recursos

La tercera factura: la tensión que anuncié en el capítulo 18 entre dimensionar y aplicar patrones.

Cuadro 32.4: De dónde salen los 31 kilobytes del pico de la versión repartida sin la Parte VII, que es la comparable con el monolito. Los objetos de una oleada están medidos aparte en el capítulo 19; el resto es lo que añaden las piezas del reparto y los objetos que van y vienen durante el turno.

Qué	Bytes	Capítulo
55 invasores como objetos con `__slots__`	11 208	19
Los mismos como listas paralelas —lo que hacía el monolito—	3 128	19
Diferencia atribuible a los objetos	8 080	
Resto: fábrica, estrategia, sucesos, repositorio, vista, control, y los objetos que se crean y descartan en cada turno	≈ 20 000	20–27

O sea: de los 24 kilobytes de diferencia, **un tercio son los objetos y dos tercios es todo lo demás**. El capítulo 19 ahorró un 25 % con `__slots__`, y los ocho patrones se comieron ese ahorro varias veces.

Detengámonos a pensar · Entonces, ¿el capítulo 19 fue una pérdida de tiempo?

No, y la razón es la que aquel capítulo daba: **lo que importa no es el ahorro, es la escala**.

Treinta kilobytes de pico en una máquina de hoy no son nada, y por eso las dos decisiones —objetos y patrones— son asumibles. Si en lugar de 55 invasores hubiera 55 000, la cuenta de aquel capítulo daría 11 megabytes solo en invasores, y entonces habría que rehacer la decisión.

La regla no cambia: *cuenta cuántos hay y multiplica*. Lo que cambia es el resultado.

Lo que sí sería un error es lo contrario: aplicar los ocho patrones sin haber medido nunca y descubrir en producción que el bucle no llega a tiempo.

32.4 Qué se pudo borrar

El libro anterior dejó una medida que aquí conviene aplicar: *una transformación se mide por lo que permite eliminar*.

Aplicada a este juego, la respuesta es incómoda: **no se borró nada**. El juego repartido tiene todo lo que tenía el monolito —y bastante más— y 1 466 líneas de código por encima de él.

Detengámonos a pensar · Y sin embargo hay algo que sí desapareció

Desapareció preguntar de qué tipo es algo. El capítulo 10 lo midió: cero `isinstance` en el juego entero, frente a seis tramos del bucle del monolito rotulados con comentarios

para no perderse.

Se repartió el estado compartido. Cincuenta y dos variables vivas a la vez pasaron a ser atributos de **59 clases**.

`Partida` tiene **dieciséis**. Es, con diferencia, la que más, y es exactamente la clase que el capítulo 13 señaló con cinco razones para cambiar y que nunca llegamos a partir del todo. Las siguientes son `Invasor` con siete y `FabricaDeOleadas` con seis.

Se pasó de 52 variables en una función a 16 atributos en la clase central y el resto repartido en 58 clases pequeñas. Es una mejora de algo más de tres veces, no una desaparición, y la clase que sigue concentrando estado es la que el libro lleva señalando desde el capítulo 3.

Y desapareció tener que leer el juego entero para cambiar una cosa.

Ninguna de las tres se ve en un recuento de líneas. Por eso el balance de este capítulo no puede ser una tabla sola.

32.5 Defender un diseño explicándolo

Cuadro 32.5: Las cuatro operaciones que exige afinar un juego, y si obligan a tocar el código fuente. Esta es la medida que sí distingue las dos versiones.

Operación	Monolito	Repartido
Vigilar un invariante en cada turno	Hay que editar `jugar()`	Se pasa un vigilante
Cambiar cómo dispara la invasión	Hay que editar el bucle	Se pasa otra estrategia
Mirar la oleada 7 sin jugar seis	Imposible	`fabrica.crear(7)`
Probar el guardado sin tocar el disco	Hay que editar	Se pasa `EnMemoria`
Operaciones que exigen editar	**4 de 4**	**0 de 4**

Detengámonos a pensar · El valor real del diseño

Las tres magnitudes que se miden fácil —líneas, memoria, tiempo— favorecen al monolito. Y no importa por gusto: importa porque el capítulo 18 dejó dicho cuál es el reto de este juego. No es demostrar que no se pierde: es que las mecánicas funcionen, que la velocidad sea la adecuada, que aguante horas.

Eso no se resuelve una vez. Se afina, y afinar es un bucle de mirar, cambiar y volver a mirar. Las cuatro operaciones de la tabla son ese bucle.

En el monolito, cada vuelta del bucle empieza editando el juego. Y un bucle que empieza editando el juego es un bucle que se da pocas veces, porque cada vuelta arriesga romper algo.

El diseño no abarata el cambio. Abarata la vuelta siguiente.

32.6 Lo que el reparto no hizo

No hizo el juego más pequeño. Es cinco veces más grande sin contar documentación.

No lo hizo más rápido. Es **casi ocho veces** más lento por turno —y **unas cinco** descontando lo que cuesta la funcionalidad de la Parte VII—, y esa proporción crecería con más entidades: cada indirección se paga por objeto y por turno.

No abarató los cambios medidos en sitios. Empate técnico.

No lo hizo más correcto. Las dos versiones juegan al mismo juego con las mismas reglas. El capítulo 31 no pudo demostrar que ninguna falle.

Y no apareció solo. Treinta capítulos, siete correcciones que la medición me obligó a hacer y un defecto de diseño que encontró una prueba. Ese coste es real y no está en ninguna tabla.

Si esto fuera un equipo · Lo que cuesta y lo que permite

Repartir el trabajo en una organización tampoco lo hace más barato: añade coordinación, reuniones y traspasos, y eso se paga todos los días.

Lo que compra es lo mismo que aquí: poder cambiar una parte sin parar el resto, y poder mirar dentro sin desmontarlo.

Y como aquí, la trampa es justificarlo diciendo que es más eficiente. No lo es. Es más *sostenible*, que es otra cosa y hay que decirla por su nombre.

Con las tres facturas pagadas, queda una pregunta que este capítulo ha dejado más viva que nunca. Si el reparto cuesta cinco veces más código, 14 veces más memoria y no abarata los cambios medidos, cabe preguntarse si hacía falta.

El capítulo siguiente hace esa pregunta, y la responde construyendo las versiones y comparándolas.

Capítulo 33

¿Hacía falta la orientación a objetos?

El libro anterior tenía un capítulo así: se preguntaba si el cuadrado mágico que le daba nombre era realmente necesario, construía las versiones sin él y respondía que sí se podía quitar.

Este es el equivalente, y la pregunta es más incómoda porque afecta al título.

33.1 La objeción del lector curioso

Va así, y es buena:

> El capítulo anterior mide que el juego repartido es cinco veces más grande, gasta 4,6 veces más memoria y va unas cinco veces más lento —y eso ya descontando lo que cuesta la funcionalidad de la Parte VII—. Y que el coste de un cambio es un empate.
>
> El juego de una sola pieza era un desastre, de acuerdo. Pero *¿la alternativa era la orientación a objetos, o simplemente escribirlo bien?*

Es la pregunta que este libro tiene que responder, y no se responde opinando.

Inténtalo primero · Antes de que la responda

Imagina una tercera versión: sin clases, sin herencia, sin polimorfismo. Solo funciones cortas y datos bien organizados.

¿Cuánto crees que se parecería al monolito, y cuánto al juego repartido? Escribe tu apuesta antes de mirar la tabla.

33.2 Tres versiones del mismo juego

La construí, y se ejecuta como las otras dos.

Cuadro 33.1: Qué es cada versión. Las dos primeras implementan las mismas reglas; la de objetos lleva además el jugador humano, las recompensas y las oleadas que cambian de la Parte VII, y por eso la tabla siguiente la mide también con eso apagado.

Versión	**Qué es**
Monolito	El juego entero en una función de 265 líneas. Capítulo 2
Funciones y datos	Sin clases, sin herencia, sin polimorfismo. Estado en tuplas con nombre e inmutables; funciones cortas, cada una con una responsabilidad
Repartido en objetos	Los 23 módulos y 59 clases de este libro

La segunda no es el monolito con otro nombre. Aplica casi todo lo que este libro defiende —responsabilidad única, nada de estado compartido, contratos claros— **sin usar una sola clase**.

Cuadro 33.2: Las tres versiones, medidas con la misma vara por los guiones de comparación y de coste del cambio. «Sitios» son los tramos separados que hay que tocar para cada uno de los tres cambios del capítulo 2. La cuarta columna es la de objetos con la Parte VII apagada, que es la única comparable con las dos primeras: sin ella la tabla estaría comparando dos juegos distintos (capítulo 32).

	Monolito	**Funciones**	**Objetos**	**Ídem, sin la Parte VII**
Archivos	1	1	23	—
Líneas, sin documentación	253	**240**	1 255	—
Funciones y métodos	2	18	200	—
Clases	0	4	59	—
Pico de memoria	**6 770 B**	595 984 B	96 192 B	31 018 B
Microsegundos por turno	**28,9**	43,4	227,4	139,7
Sitios: cuarto tipo de invasor	6	**2**	5	—
Sitios: dos balas en el aire	4	**2**	13	—
Sitios: diez mejores marcadores	6	**5**	12	—
Puntuación media, 100 partidas	1 609	1 731	7 211	2 191

33.3 La respuesta a la pregunta incómoda de este capítulo: para el juego tal como se pidió

Detengámonos a pensar · La versión sin clases gana casi en todo

Es la más pequeña. 240 líneas frente a 253 del monolito y 1 255 del juego repartido. Menos código que la versión que este libro presentó como insostenible.

Y es la más barata de cambiar. Dos sitios para añadir un tipo de invasor, dos para las dos balas, cinco para los marcadores. Gana a las otras dos en los tres cambios, y en dos de ellos de forma muy significativa.

Así que la respuesta a la pregunta del capítulo es: **NO**. Para este juego, tal como está pedido, la orientación a objetos no hacía falta. Un programador que escriba funciones cortas sobre datos inmutables llega a algo más pequeño y más fácil de cambiar.

Diecisiete capítulos de conceptos y principios, ocho de patrones, y la respuesta honesta a «¿hacía falta?» es que NO.

Detengámonos a pensar · Y pierde estrepitosamente en una cosa

Mira la fila del pico de memoria: **596 kilobytes**, casi noventa veces el monolito y veinte veces el juego repartido comparable.

Cada turno produce un estado nuevo en lugar de modificar el que hay:

```
return j._replace(invasores=tuple(i._replace(x=i.x + j.sentido)
                                  for i in j.invasores))
```

Cincuenta y cinco tuplas nuevas por paso de la formación, más el estado completo. La inmutabilidad —que es lo que hace que las funciones sean cortas y no compartan nada— se paga en memoria, y aquí se paga muy cara.

No es un defecto de mi implementación: es el precio del estilo. Y es la razón por la que la comparación no tiene un ganador limpio.

33.4 Y sin embargo, por qué insisto en recomendar la orientación a objetos

Ahora, la parte que la tabla no dice.

Vuelvo a la medida del capítulo 32: las cuatro operaciones que exige afinar un juego, y si obligan a editar el código.

Cuadro 33.3: Las cuatro operaciones sobre las tres versiones. Contado abriendo cada una e intentándolo.

Operación	Monolito	Funciones	Objetos
Vigilar un invariante cada turno	editar	**no**	**no**
Cambiar cómo dispara la invasión	editar	editar	**no**
Mirar la oleada 7 sin jugar seis	editar	**no**	**no**
Probar el guardado sin tocar disco	editar	editar	**no**
Exigen editar	**4 de 4**	**2 de 4**	**0 de 4**

Detengámonos a pensar · Dos de cuatro, analicemos de dónde surgen

La versión de funciones gana dos gratis, y no por casualidad.

Vigilar un invariante sale gratis porque `avanzar(j, azar)` devuelve el estado en lugar de modificarlo: desde fuera se puede mirar cada turno sin tocar nada. Es una ventaja del estilo inmutable, no de las clases, y el juego de objetos la consigue por otra vía.

Mirar la oleada 7 sale gratis porque `crear_oleada(7)` es una función suelta. La fábrica del capítulo 20 hace lo mismo con más ceremonia.

Y las otras dos exigen editar por el mismo motivo: `disparo_rival` y el guardado están *escritos dentro*. Para sustituirlos habría que pasarlos como parámetros —y en cuanto se pasa una función como parámetro y se le exige un contrato, se está haciendo lo mismo que una estrategia, solo que sin nombre.

33.5 Un reparto parcial no rinde

Y de aquí sale el criterio que este libro estaba buscando, que es el hermano del que dejó el anterior.

Detengámonos a pensar · Lo que rinde no son las clases: son los principios

Mira otra vez la tabla de las tres versiones y pregúntate qué separa a la primera de las otras dos.

No son las clases: la segunda no tiene ninguna que importe. Lo que la primera no tiene es **separación**. Una función de 265 líneas con 52 variables vivas falla por eso, y no por no usar objetos.

Y lo que separa a la segunda de la tercera tampoco son las clases en sí. Es la **sustituibilidad**: poder cambiar una pieza sin editar. Dos de las cuatro operaciones.

Los principios rinden sin clases. Las clases, sin principios, no rinden.

La versión de funciones cumple responsabilidad única, no comparte estado y tiene contratos claros. Cumple casi todo el capítulo 13 y buena parte del 17, sin una sola clase. Un juego con 59 clases y una función de 265 líneas dentro de una de ellas habría sido peor que las tres.

Detengámonos a pensar · Entonces, ¿cuándo compensa la orientación a objetos?

Imagina a otra persona tocando este código. O a ti mismo, meses después, tras olvidar por qué está escrito así. La pregunta deja de ser cuánto ocupa, y pasa a ser cuánto cuesta entenderlo para cambiarlo sin romperlo. Esa es la parte que la tabla no mide.

El monolito del libro anterior atendió el problema que tenía: un juego pequeño, una persona, un sitio donde cambiarlo todo. Este libro no contradice aquello —enseña a reconocer cuándo compensa cada criterio—. Donde la sostenibilidad del código importa, la orientación a objetos paga memoria y líneas a cambio de poder sustituir una pieza sin tocar las demás. Es el precio del gobierno del código, y minimizarlo es trabajo del diseño: es lo que estos capítulos han enseñado.

Con eso delante, cuándo compensa es más estrecho de lo que este libro parecía prometer:

Cuando vas a sustituir piezas. Si el programa necesita que se pueda cambiar una parte sin editarlo —para probarlo, para configurarlo, para soportar dos entornos—, las clases con contrato son el mecanismo que mejor lo da.

Cuando el estado es grande y muta mucho. La inmutabilidad de la versión de funciones cuesta 596 KB en un juego de 55 invasores. Con estado grande, esa vía se cierra.

Y cuando hay muchas cosas parecidas pero no iguales. Seis clases de entidad que responden a la misma orden es donde el polimorfismo del capítulo 10 paga solo.

Si esto fuera un equipo · Estructura o método

Cuando algo no funciona en una organización, la reacción habitual es cambiar la estructura: crear un departamento, un comité, un puesto.

Y a veces lo que fallaba no era la estructura: era que nadie sabía quién decidía qué. Repartir eso arregla el problema *sin* tocar el organigrama, igual que la versión de funciones arregla el monolito sin una sola clase.

Queda una última cosa antes de cerrar esta parte, y es la contraria de esta. Este capítulo ha enseñado dónde compensa cada criterio. El siguiente enseña qué pasa cuando se aplica todo lo de este libro sin ninguno: qué aspecto tiene un juego con todos los patrones puestos y ninguna razón para tenerlos.

Capítulo 34

Cuándo esto no funciona

Un libro que solo cuenta cuándo su idea funciona no está enseñando una idea: está vendiéndola.

El anterior tenía este capítulo y decía por qué: si sales creyendo que todo problema esconde una transformación, perderás tiempo buscando donde no hay nada, y después de tres veces abandonarás el método entero.

34.1 Clases que solo son funciones con sombrero

El primero, y el más común.

Código 34.1: No guarda nada y no puede tener una segunda implementación.

```
class CalculadorDeCadencia:
    def __init__(self, divisor):
        self._divisor = divisor

    def calcular(self, vivos):
        return 1 + vivos // self._divisor
```

Eso es una función con un archivo alrededor, y es la hermana de la que borré en el capítulo 13: aquella reenviaba a otro objeto, esta calcula una línea. La prueba de aquel capítulo descarta a las dos en un segundo: *¿podría haber una segunda implementación?* No. La cadencia se calcula de una manera.

Detengámonos a pensar · Las tres señales, y basta con una

No tiene estado propio, o el que tiene es una constante disfrazada de parámetro.
Tiene un solo método público, y se llama como la clase.
No imaginas la segunda implementación. Es la definitiva, y es la que salvó a `EstrategiaDisparo` —tiene tres— y condenó a esta.
Y hay una prueba práctica todavía más rápida. Mira cómo se usa:

```
CalculadorDeCadencia(6).calcular(55)
```

Se crea, se usa y se tira en la misma línea. La conclusión es clara: era una función disfrazada de clase.

34.2 Repartir de más: cuando cada decisión está en otro archivo

El capítulo 33 dejó una tabla que este capítulo hereda: el juego repartido tiene 1 255 líneas donde la versión de funciones tiene 240.

Buena parte de esa diferencia es lo que compra la sustituibilidad. Y otra parte, no.

Cómo se distinguen

Un reparto que rinde

Para entender qué pasa al morir un invasor abres el invasor y lees diez líneas. La habilidad y la publicación de sucesos son piezas con nombre, y solo las abres si te interesan.
Cada archivo se puede leer solo y significa algo.

Un reparto que solo dispersa

Para entender qué pasa al morir un invasor abres cinco archivos, y en cada uno hay tres líneas que llaman al siguiente.
Ninguno significa nada por separado, y la lógica que buscabas no está en ninguno: está en el recorrido.
Se reconoce porque al explicárselo a alguien tienes que abrir los cinco.

Detengámonos a pensar · La medida que sale del propio juego

El capítulo 17 midió el acoplamiento y encontró un patrón sano: lo que no depende de nada es de lo que depende todo.
La señal contraria es la que hay que buscar: **módulos que dependen de muchos y de los que dependen muchos**. En este juego no hay ninguno, y si lo hubiera sería el primer sitio donde mirar.
Y hay una segunda medida, más tosca y muy útil: **¿cuántos archivos hay que abrir para seguir una operación de principio a fin?** Para el impacto de una bala en este juego son tres: el proyectil, la partida y el invasor. Con siete, empezaría a sospechar.

34.3 Optimizar de más: el byte que costó una tarde y ahorró un kilobyte

Detengámonos a pensar · La regla, y es la misma de siempre

Sin medir, no se optimiza. Y con la medida delante, la pregunta no es «¿cuánto ahorro?», sino **«¿ese ahorro escala con algo?»**.
Un ahorro que no escala es una preferencia estética con aspecto de ingeniería.
Y hay un coste que casi nunca se cuenta: el código empaquetado a mano no se puede leer sin descifrarlo, y eso se paga cada vez que alguien lo abra, para siempre.

34.4 SOLID aplicado a rajatabla, y lo que produce

En la Parte V incumplí tres de los cinco principios a propósito y lo dije cada vez. Este es el sitio de explicar por qué ignorar un principio también es una decisión de arquitectura que se toma conscientemente.

Cuadro 34.1: Cada principio, llevado al extremo, produce su propio desastre. La columna de la derecha es el que yo he visto más veces.

Principio	Llevado al extremo
Responsabilidad única	Clases de un método que solo reenvían. Cinco archivos para una operación
Abierto/cerrado	Puntos de extensión para cosas que nadie ha pedido nunca, con una sola implementación cada uno
Sustitución de Liskov	Jerarquías que no se tocan por miedo, y comportamiento duplicado para no arriesgar
Segregación de interfaces	Un contrato por método. La idea se pierde y hay que reconstruirla leyendo la lista
Inversión de dependencias	Una interfaz por cada clase, y una fábrica para cada interfaz

Detengámonos a pensar · Los cinco extremos tienen la misma forma

Fíjate en la columna de la derecha: los cinco producen **indirección sin sustitución**. Una pieza más que atravesar y nada que se pueda cambiar por ella.
De ahí sale la prueba que vale para los cinco a la vez:

> **¿Existe, o puede existir, una segunda cosa al otro lado de esta indirección?**

Si sí, la indirección compra algo. Si no, solo la pagas.
Es la misma pregunta que descartó `CalculadorDeCadencia` y la misma que justificó `EnMemoria`. Un principio no dice qué hacer: dice qué mirar.

34.5 Lo que este juego no sirve para enseñar

El libro anterior terminó con una lista así, y era de las páginas más útiles. Space Invaders enseña lo que el tres en raya no podía, y tiene sus propios huecos.

Cuadro 34.2: Lo que este caso no puede enseñar sin forzarlo, y por qué.

Concepto	**Por qué aquí no**
Concurrencia	El bucle es de un solo hilo y por turnos. Igual que el tres en raya: sigue sin poder enseñarse
Persistencia de verdad	Diez marcadores en un JSON. No hay consultas, ni esquema, ni datos que no quepan en memoria
Sistemas distribuidos	Todo ocurre en un proceso. Nada que coordinar, nada que pueda fallar a medias
Jerarquías profundas	Y aquí es una virtud: el capítulo 9 enseña a no construirlas, no a manejarlas
Rendimiento a escala	55 invasores. Los problemas que aparecen con cien mil entidades no salen aquí
Interfaces complejas	La vista pide caja y símbolo. Nada de animación, ni de estado propio de la interfaz

34.6 El riesgo de ver patrones donde no los hay

Hay un efecto secundario de aprender un catálogo, y este libro puede provocarlo: empezar a nombrar patrones en todas partes.

Nombrar no es aplicar. Que una clase reciba otra por el constructor no la convierte en inyección de dependencias si lo que recibe es una clase concreta —está dicho en el capítulo 17—.

Un patrón sin su dolor es decoración. Los ocho capítulos de la Parte VI empiezan por el dolor a propósito. Si no puedes nombrar qué te dolía, no necesitas el patrón: necesitas un nombre para lo que ya hacías.

Y el catálogo no es el objetivo. La versión de funciones del capítulo 33 no aplica ni uno de los ocho, y gana en tres de las cuatro medidas.

34.7 Cómo saber que te has equivocado

Cuadro 34.3: Señales de que el reparto se ha ido de las manos.

Lo que observas	Lo que probablemente significa
Para explicar una operación abres cinco archivos	Repartiste el recorrido, no las responsabilidades
Una interfaz con una sola implementación, y no ves la segunda	Indirección sin sustitución
Clases cuyo nombre acaba en «-ador» o «-or» y tienen un método	Eran funciones
Al añadir algo tocas la abstracción *y* la implementación	La abstracción no aisló nada
Escribir una prueba exige montar seis objetos	Las dependencias van en la dirección equivocada
Nadie del equipo sabe dónde va lo nuevo	El reparto no comunica

Detengámonos a pensar · Y una cautela sobre todo lo anterior, con fuente

Hay que aclarar algo sobre una expresión que uso con frecuencia: he hablado todo el libro de código «difícil de leer» como si la dificultad fuera una propiedad del código. No lo es del todo.

Un estudio de 2023 midió qué factores influyen en la carga cognitiva de quien programa, con seguimiento ocular sobre 216 programadores. Los que más pesaron fueron la **edad** (18,0 %) y la **distancia lingüística** entre la lengua materna de quien lee y el inglés (15,4 %) (Issever et al., 2023).

Lo que ese estudio sostiene: que lo difícil de leer depende también de *quién* lee, y no solo de qué está escrito.

Lo que no sostiene: no midió el efecto del número de variables, ni del tamaño de una función, ni de ninguna propiedad del código. No respalda ninguna de las afirmaciones de este libro sobre diseño.

Cuando decidas si un reparto está bien, acuérdate de que la respuesta depende de quién va a mantenerlo. Un diseño elegante que solo tú entiendes no es un diseño elegante.

Si esto fuera un equipo · Reorganizar por reorganizar

Toda organización con un problema tiene la tentación de crear una estructura nueva. Y a veces la estructura nueva solo añade un paso.

La señal es la misma que en el código: si al crear el puesto nuevo nadie puede decir qué

decisión se toma ahí que antes no se tomaba en ningún sitio, no se ha repartido nada. Y la prueba también: *¿podría ocupar ese puesto otra persona con otro método?* Si no, no es un puesto.

Con esto se cierra la Parte VIII. El juego está construido, medido y discutido, incluida la posibilidad de que no hiciera falta construirlo así.

Queda lo que atraviesa todo el libro y hasta ahora solo ha ido asomando en los recuadros *prompt*. Hoy no diseñas solo: tienes al lado una herramienta que escribe un Space Invaders entero en un minuto. Y en un libro sobre diseño, eso no es una comodidad: es un problema nuevo.

IX

Capacidades aumentadas con Inteligencia Artificial

Capítulo 35

Un juego entero en un prompt

Te ha dado prompts en casi todos los capítulos, y en todos te ha pedido que no le pidas el juego. Esta parte explica por qué no es una contradicción, y por qué aquí el asunto es más serio que en el libro anterior.

35.1 Lo que la herramienta hace extraordinariamente bien

Escribe un Space Invaders que funciona. Se lo pides en una frase y en menos de un minuto tienes código que se ejecuta, con sus invasores, su formación que baja, sus búnkeres y su marcador. Probablemente bien estructurado, con clases y nombres razonables.

Y hace algo más difícil que eso, que es lo que este libro sí ha aprovechado: traduce, explica código ajeno, genera casos de prueba, recorre exhaustivamente lo que es pequeño y encuentra el error de sintaxis que llevas veinte minutos mirando.

Detengámonos a pensar · Dónde rinde mejor, y qué tienen en común esas tareas

«Escribe una función que reciba los proyectiles y devuelva los que siguen dentro del tablero» es una petición donde la decisión ya está tomada. La herramienta la ejecuta mejor y más rápido que tú.

«Hazme un Space Invaders» no es eso. Es delegar la decisión, y con ella todo lo que este libro ha tratado de enseñar.

35.2 El error de diseño que un programa que funciona esconde

En aquel libro, si le pedías el algoritmo del tres en raya y salía mal, **se notaba**: el programa perdía la partida. El error tenía un síntoma inmediato, visible y comprobable en la primera partida.

Aquí no ocurre nada de eso. Un juego mal diseñado **funciona perfectamente**: se juega, los invasores bajan y el marcador sube. El daño no aparece hasta el primer cambio, y para entonces ya nadie recuerda de dónde salió el código.

La responsabilidad sigue siendo tuya, aunque el código no lo hayas escrito tú. Y hay un riesgo que agrava el daño silencioso: la herramienta no suele cuestionar lo que le pides. Tiende a cumplir el encargo como mejor sepa y lo más rápido que pueda, aunque el diseño que entregue no sea el que te conviene.

Este libro no rechaza usarla —el recuadro *prompt* la ha convocado en casi todos los capítulos—. Lo que pide es que el criterio —qué se delega, qué no y para qué— esté en tu cabeza y no en la suya.

Detengámonos a pensar · Lo que se sabe medido sobre esto

No es una impresión mía. Ghosh Paul et al. (2025) compararon código generado por cuatro modelos con soluciones de referencia escritas por profesionales, midiendo la incidencia de «malos olores» de código en tareas Java.

El código generado presentó, de media, un **63,34 % más de malos olores**: un 42,28 % con Falcon, un 62,07 % con Gemini Pro, un 65,05 % con ChatGPT y un 84,97 % con Codex.

Y el dato que más importa para este libro: el aumento se reparte en un 73,35 % de malos olores *de implementación* y un 21,42 % *de diseño*, y los autores encuentran que las tareas más complejas y los temas de **orientación a objetos** correlacionan con más malos olores.

Es decir: justo donde este libro pone el foco es donde el código generado flaquea más.

Y lo que ese trabajo no sostiene: no dice que el código generado no funcione —funciona—, no mide otros lenguajes distintos de Java, y evalúa modelos que a estas alturas ya no son los últimos. Tómalo como lo que es: una medida de una situación que cambia deprisa.

35.3 Por qué aquí el riesgo es mayor que en el primer libro

Porque el fallo no tiene síntoma. Ya está dicho: un mal diseño se ejecuta igual de bien que uno bueno.

Porque el juez está lejos. En el libro anterior, el juez era el recorrido exhaustivo: cero derrotas o no. Aquí el juez es el capítulo 32, y aparece cuando hay que cambiar algo, semanas después.

Y porque el aspecto es bueno. El código generado tiene clases con nombres correctos, métodos cortos y documentación. Se parece mucho a un buen diseño, y la única manera de distinguirlos es la que este libro ha usado durante treinta y cuatro capítulos: preguntar quién sabe qué, y medir el coste del cambio siguiente.

Los dos caminos desde el mismo punto

Pedir el juego

«Hazme un Space Invaders en Python.» En un minuto tienes un juego que funciona. Puede que con una clase que lo sabe todo, puede que con quince bien repartidas. No lo sabes, porque no lo decidiste. Y no lo vas a saber hasta que hagas un análisis sobre él o te pidan un cambio.

Pedir el reparto

«Estas son las reglas. Cuestióname el planteamiento antes de escribir nada. Después propón la lista mínima de clases, con una frase por clase y sin usar la palabra “y”.» Tardas más. Y lo que sale lo decidiste tú, con la herramienta discutiéndolo.

Cuando llegue el cambio, sabrás dónde va.

35.4 Lo que sigue siendo humano

Puedes delegar la escritura, la traducción, buena parte de la comprobación y casi todo el trabajo mecánico. Lo que no se delega:

Notar que algo no encaja. Que una clase tenga cinco razones para cambiar no lo señala ninguna herramienta si el programa funciona.

Decidir el criterio. Este libro eligió medir el coste del cambio siguiente. Otra persona habría medido el rendimiento y habría escrito la versión de funciones del capítulo 33. Las dos son defendibles, y elegir es responsabilidad de quien arquitecta el software.

Responder por el resultado. Aunque el código lo escriba una máquina, la responsabilidad sobre él sigue siendo tuya.

Si esto fuera un equipo · Delegar la ejecución sin delegar el criterio

Delegar un trabajo a alguien competente sale bien mientras el criterio de aceptación esté claro.

Cuando además se delega el criterio —«hazlo como te parezca»—, el resultado puede ser excelente y no ser el que hacía falta, y nadie podrá decir que estuvo mal hecho.

Compruébalo · El experimento de los dos caminos

Pídele a tu asistente un juego pequeño de una frase. Guárdalo.

Después pídele el mismo juego siguiendo el prompt del capítulo siguiente: cuestionar primero, listar clases con una frase, sin la palabra «y».

Compara los dos con la medida del capítulo 32: pide un cambio —un tipo de enemigo nuevo— y cuenta los sitios que hay que tocar en cada uno.

Queda lo concreto: cómo se escribe una petición que te obligue a decidir en lugar de ahorrarte la decisión. Los prompts de este libro están construidos de una manera muy determinada, y en el siguiente capítulo profundizaremos en el porqué.

Capítulo 36

Prompts que reparten

Este capítulo explica el criterio, para que puedas mejorar la forma de escribir los tuyos.

El libro anterior dejó una regla: *pide piezas de un plan que tú tienes; no pidas el plan.* Sigue valiendo, y para diseño no basta. Aquí hacen falta tres cosas más.

36.1 No pidas el juego, pide el reparto

La primera, que es la del libro anterior aplicada a lo que nos ocupa.

Dos maneras de pedir lo mismo

Pedir el resultado

«Hazme un Space Invaders en Python, con clases bien organizadas.»

Recibes un juego que funciona. No sabes quién sabe qué, no elegiste el reparto, y si mañana hay que cambiar algo no sabrás dónde va.

Y como funciona, no tienes ningún motivo para mirarlo.

Pedir el reparto

«Estas son las reglas. Propón la lista mínima de clases, con una frase por clase y sin usar la palabra «y». Señálame las que no pasen esa prueba.»

Recibes una propuesta de reparto que puedes discutir, y una lista de las que no convencen ni al propio asistente.

El código viene después, y ya sabes dónde va cada cosa.

36.2 Que cuestione el escenario antes de escribir nada

Un asistente al que le das un enunciado empieza a resolverlo. No pregunta si el enunciado está bien, porque nadie se lo ha pedido, y porque su trabajo aparente es producir.

Detengámonos a pensar · Por qué esto importa más en diseño que en un algoritmo

Un algoritmo mal especificado da un resultado que se puede comprobar: el tres en raya pierde la partida.

Un diseño mal especificado da un diseño que funciona y que resuelve el problema equivocado. Y no hay nada que lo delate.

Casi todos los problemas de diseño que resolvimos en este libro venían de un planteamiento que no habíamos mirado bien: que el OVNI fuera un invasor, que las habilidades fueran subclases, que el límite de turnos fuera del control. Los tres se habrían discutido en dos minutos si alguien nos hubiera preguntado.

Pedir que te cuestionen es la parte más barata del proceso y la que más valor retorna.

36.3 Que haga lo mínimo pedido

Un asistente tiende a añadir: si le pides una clase, te da la clase, su jerarquía y tres subclases.

Lo que no pediste no lo decidiste. Las decisiones de diseño son tuyas.

Lo añadido parece decidido. Un lector posterior —incluido tú dentro de tres meses— no distingue lo que se pensó de lo que vino de regalo.

Y lo añadido es justo donde aparecen los malos olores. El capítulo 35 traía la medida: el aumento de malos olores en código generado se concentra en las tareas complejas y en los temas de orientación a objetos (Ghosh Paul et al., 2025).

Detengámonos a pensar · Cómo se pide, literalmente

La fórmula que uso en todos los prompts de este libro, y que puedes copiar:

> *«No añadas nada que no te haya pedido. Si crees que falta algo, dímelo en una lista aparte en vez de añadirlo.»*

Las dos frases hacen falta. La primera pone el límite; la segunda deja salida, porque si no la das, lo añadido aparece igual y además sin avisar.

36.4 El agente inteligente eres tú: la planeación y el objetivo son tuyos

Detengámonos a pensar · Quién piensa y quién construye

La planeación y el objetivo los define la persona. La herramienta propone, construye, critica y compara —y hace muy bien las cuatro cosas—, pero no tiene autoridad para decidir por ti.

Un asistente que cuestiona tu planteamiento, hace lo mínimo y te devuelve lo que le pediste es un colaborador excelente. Uno que decide por ti es una delegación sin control, y el problema no es que se equivoque: es que cuando acierte tampoco sabrás por qué.

36.5 Anatomía de un prompt de diseño

Cuadro 36.1: Las seis piezas. Los ejemplos son literales de los prompts de los capítulos 3 y 11.

Pieza	Qué hace	Ejemplo del libro
Contexto	Sitúa el trabajo	«Voy a construir un Space Invaders y quiero decidir el reparto ANTES de escribir código»
Objeción	Le obliga a discutir	«Cuestióname el planteamiento. ¿Hay alguna regla ambigua? Pregúntamelo antes de seguir»
Encargo	Dice qué producir	«Propón la lista mínima de clases»
Restricción	Fija tu criterio	«Una frase de qué hace, sin usar la palabra «y»»
Devolución	Te obliga a pensar	«Señálame las que no pasen esa prueba, y por qué»
Límite	Impide el regalo	«No añadas nada que no te haya pedido»

36.6 Cómo pedirle que critique tu diseño

Un asistente al que le enseñas tu diseño encontrará razones para que sea bueno. No es adulación: le has dado un punto de partida y trabaja desde él.

Prompt · Que refute tu reparto

```
Te paso un diseño y quiero que intentes demostrarme que está mal repartido.

[Pega las clases, con una frase de qué hace cada una.]

No busques razones para que esté bien. Busca:

1. Una clase con más de una razón para cambiar. Dime cuáles son y QUIÉN pediría
   cada una: un rol, no una persona.
2. Una indirección que no permita sustituir nada: una interfaz o una clase
   intermedia con una sola implementación posible.
3. Un sitio donde para seguir una operación haya que abrir más de tres
   archivos.
4. Una clase sin estado propio y con un solo método público.

Para cada hallazgo dime si lo arreglarías moviendo la responsabilidad o
BORRANDO la pieza. No propongas añadir nada.
```

```
Si no encuentras ninguno, dime exactamente qué has revisado y qué no has
podido revisar. No me digas que está bien: dime qué has mirado.
```

Detengámonos a pensar · Y una cautela antes de aceptar lo que te devuelva

La crítica también se equivoca. Si te señala una clase como innecesaria y tú puedes nombrar la segunda implementación que justifica su indirección, la equivocada es la crítica, no la clase. Si te propone unir dos piezas que de verdad cambian por motivos distintos, lo que te está devolviendo es el monolito con otra cara.

Lo que te entrega el asistente es una lista de sospechosos, no un veredicto. La prueba del capítulo 34 —*¿existe, o puede existir, una segunda cosa al otro lado de esta indirección?*— sigue siendo tuya en cada hallazgo.

36.7 Verificar lo que te responde cuando todo compila

En el libro anterior, verificar era comprobar que el algoritmo no perdía. Aquí el código siempre funciona, así que verificar es otra cosa.

Cuadro 36.2: Qué comprobar de un diseño que te devuelven, y cómo. Ninguna de las cinco exige ejecutar el programa.

Si te devuelve...	**Comprueba...**
Una clase	Que puedas decir qué hace en una frase sin «y»
Una interfaz	Que exista, o puedas imaginar, una segunda implementación
Una jerarquía	Que el hijo pueda ocupar el sitio del padre sin avisar a nadie
Un patrón, con su nombre	Qué dolor concreto resuelve. Si no hay dolor, es decoración
Una cifra	Reprodúcela. Este libro corrigió siete propias por hacerlo

Compruébalo · Los dos diseños, uno al lado del otro

Pídele a tu asistente el mismo componente dos veces.

La primera, de una frase. La segunda, con las seis piezas de este capítulo.

Pon los dos resultados uno al lado del otro y aplícales la tabla de arriba, fila por fila. No mires cuál es más bonito: cuenta cuántas filas pasa cada uno.

Y aquí termina lo que este libro tenía que enseñarte sobre diseño y sobre cómo pedirlo.

Apéndice A

Glosario

Los términos aparecen con el significado que tienen *en este libro*, que no siempre es el único posible. Cada uno remite a la definición numerada donde se introduce, para que puedas leer el contexto en lugar de fiarte del resumen.

Las definiciones no se reescriben aquí: se remite a ellas. Un libro que se cita a sí mismo se verifica como se verifica una fuente, y la manera de no contradecirse es no tener dos versiones de lo mismo.

Los diez conceptos

Clase Descripción de una familia de cosas: qué sabe cada una, qué sabe hacer y qué tiene que cumplir para ser válida. No es ninguna de esas cosas: es su descripción. Definición 3.1.

Objeto Una de esas cosas, ya existiendo, con sus valores propios. Definición 3.2.

Atributo Dato que un objeto guarda porque le pertenece y porque no se puede deducir de lo que ya tiene. La segunda condición es la que más atributos elimina. Definición 4.4.

Método Algo que un objeto sabe hacer consigo mismo. Se le pide; no se le hace. Definición 5.1.

Constructor Contrato de entrada de una clase: dice qué hace falta para que uno de sus objetos exista, y rechaza lo que no cumpla. Definición 6.2.

Encapsulamiento Decisión de qué parte de un objeto es asunto de los demás y qué parte es solo suya. En Python es un contrato, no un muro. Definición 7.1.

Abstracción Descripción que conserva lo que decide y descarta lo demás. Definición 8.1.

Herencia Declaración de que una clase es un caso particular de otra. No es una declaración de parecido: es una declaración de sustituibilidad. Definición 9.1.

Polimorfismo Una misma orden produce comportamientos distintos según a quién se le dé, sin que quien la da tenga que saber a quién. Definición 10.1.

Composición Construir un objeto a partir de otros que él tiene y usa. Las capacidades se montan al crear, no se declaran en la jerarquía. Definición 11.1.

Interfaz Lista de lo que hay que saber hacer para ocupar un sitio. Dice qué se promete y nada de cómo. Definición 12.1.

Clase abstracta Contrato que además trae trabajo hecho: implementa lo común y deja abierto lo que cada caso resuelve a su manera. Definición 12.2.

Los tres escalones del dato

Dato Valor sin contexto. No significa nada por sí solo. Definición 4.1.

Información Dato con su contexto: qué es, de quién, en qué unidad. Definición 4.2.

Conocimiento Lo que permite decidir: relaciona varias informaciones y produce una acción. Definición 4.3.

Los cinco principios

Responsabilidad única Una clase debe tener una sola *razón para cambiar*. Una razón no es una función: es alguien que pediría el cambio. Definición 13.1.

Abierto/cerrado Abierta a la extensión y cerrada a la modificación. Se comprueba con un diff. Definición 14.1.

Sustitución de Liskov El hijo debe poder ocupar el sitio del padre sin que nadie se entere. Pide menos, promete más; nunca al revés. Definición 15.1.

Segregación de interfaces Nadie debe verse obligado a depender de métodos que no usa. Un contrato por grupo de clientes. Definición 16.1.

Inversión de dependencias Lo importante no depende de lo accesorio: los dos dependen de un contrato, y el contrato pertenece a lo importante. Definición 17.1.

Los ocho patrones

Factory Objeto cuyo único oficio es construir otros. Quien la usa pide lo que quiere y no sabe cómo se compone. Definición 20.1.

Strategy Objeto que encapsula una manera de hacer algo, para poder sustituirla sin tocar a quien la usa. Definición 21.1.

Observer Quien publica no sabe quién escucha, ni cuántos, ni si hay alguno. Definición 22.1.

Adapter Traduce lo que habla una pieza ajena al idioma que entiende la tuya. Traduce; no decide, no arregla, no recuerda. Definición 23.1.

Repository Contrato para guardar y recuperar, escrito con el vocabulario del problema y no con el del almacén. Definición 24.1.

Singleton Clase que impide que exista más de un objeto suyo. El único patrón que este libro incluye para desaconsejarlo. Definición 25.1.

Inyección de dependencias Un objeto recibe las piezas que necesita en lugar de fabricarlas. Es una *técnica*, no el principio de inversión de dependencias. Definición 26.1.

Modelo, vista y control El modelo es lo que el programa es; la vista lo muestra y no decide; el control traduce lo que llega de fuera. Definición 27.1.

Términos del método

Invariante de clase Algo cierto de todos los objetos de una clase desde que nacen. No es una comprobación: es una promesa que no se puede incumplir. Definición 6.1.

Raíz de composición El único sitio del programa que conoce todas las piezas a la vez, porque es quien las monta. Que nadie dependa de él es lo que permite que el resto no conozca a nadie. Capítulo 17.

Doble de prueba Implementación de un contrato hecha para poder probar: el repositorio en memoria, el reloj falso, la estrategia sin azar. Legítimo mientras cumpla el mismo contrato y nada más. Capítulo 26.

Propiedad Afirmación de la forma «pase lo que pase, esto no». Se comprueba en cada turno de muchas partidas, frente a un caso de prueba, que comprueba un punto. Capítulo 31.

Determinismo Que la misma semilla produzca la misma partida. No es lo contrario del azar: es lo que permite volver a ver un fallo. Capítulo 31.

Sitio Unidad con la que este libro mide el coste de un cambio: un tramo separado del archivo, contado con un diff sin contexto. El capítulo 32 explica por qué resultó ser una medida peor de lo que parecía.

Indirección sin sustitución Una pieza más que atravesar sin que haya nada que se pueda poner en su lugar. Es la forma que toman los cinco principios llevados al extremo. Capítulo 34.

Referencias

Cabral, R., Kalinowski, M., Baldassarre, M. T., Villamizar, H., Escovedo, T., & Lopes, H. (2024). Investigating the Impact of SOLID Design Principles on Machine Learning Code Understanding [Versión de los autores consultada en arXiv:2402.05337]. *Proceedings of the IEEE/ACM 3rd International Conference on AI Engineering — Software Engineering for AI (CAIN '24)*. https://doi.org/10.1145/3644815.3644957

Ghosh Paul, D., Zhu, H., & Bayley, I. (2025). Investigating the Smells of LLM Generated Code [Preprint. Versión extendida de un trabajo presentado en ICCBDCS 2025]. https://arxiv.org/abs/2510.03029

Issever, D., Catalbas, M. C., & Duran, F. (2023). Examining Factors Influencing Cognitive Load of Computer Programmers. *Brain Sciences*, *13*(8), 1132. https://doi.org/10.3390/brainsci13081132

Nana Tchakoute, R., & Tadonki, C. (2026). Energy-Aware Computing in the Year 2026 [Preprint. Panorámica sobre eficiencia energética en el continuo de computación]. https://arxiv.org/abs/2605.24569